

MAKALAH
PERANCANGAN DAN IMPLEMENTASI BASIS DATA RELASIONAL PADA
SISTEM RESERVASI HOTEL DENGAN MENGGUNAKAN MYSQL



Oleh:

- | | |
|----------------------------------|-----------|
| 1. Fania Hasna Rasyida | K1D024001 |
| 2. Frisca Putri Auliya | K1D024008 |
| 3. Desti Cahya Utami | K1D024009 |
| 4. Fatih Maulana Luthf Rabbaanii | K1D024022 |
| 5. Amelia Wulandari | K1D024027 |

Disusun untuk memenuhi tugas terstruktur :

Mata kuliah : Basis Data

Dosen : Lutfiah Maharani Siniwi S.Stat., M.Stat.

KEMENTERIAN PENDIDIKAN TINGGI, SAINS, DAN TEKNOLOGI

UNIVERSITAS JENDERAL SOEDIRMAN

FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM

PROGRAM STUDI STATISTIKA

PURWOKERTO

2026

KATA PENGANTAR

Puji syukur panjatkan ke hadirat Tuhan Yang Maha Esa atas segala rahmat, karunia, dan hidayah-Nya sehingga kami dapat menyelesaikan laporan project Mata Kuliah Basis Data yang berjudul “Perancangan dan Implementasi Sistem Basis Data Relasional pada Sistem Reservasi Hotel” dengan baik dan tepat waktu. Laporan ini disusun sebagai salah satu syarat untuk memenuhi tugas Project Akhir pada Mata Kuliah Basis Data serta sebagai bentuk penerapan konsep-konsep yang telah dipelajari selama perkuliahan.

Dalam penyusunan project ini melakukan perancangan dan implementasi sebuah sistem basis data relasional yang mendukung proses bisnis reservasi hotel. Sistem yang dirancang mencakup pengelolaan data tamu, kamar, tipe kamar, reservasi, pembayaran, check-in, check-out, fasilitas, pegawai, serta ulasan tamu. Melalui project ini, dapat memperoleh pengalaman dalam menganalisis kebutuhan sistem, merancang *Entity Relationship Diagram* (ERD), melakukan normalisasi data, mengimplementasikan database menggunakan SQL, serta mengembangkan berbagai objek basis data seperti *query*, *view*, *function*, *stored procedure*, dan *trigger*.

Penyusunan laporan ini tidak terlepas dari bantuan, dukungan, serta bimbingan berbagai pihak. Oleh karena itu, kami menyampaikan ucapan terima kasih kepada dosen pengampu Mata Kuliah Basis Data yang telah memberikan arahan dan bimbingan selama proses perkuliahan dan pengerjaan project.

Penyusun menyadari bahwa laporan ini masih memiliki keterbatasan dan belum sepenuhnya sempurna. Oleh karena itu, penyusun sangat mengharapkan kritik dan saran yang membangun guna menjadi bahan evaluasi dan perbaikan di masa mendatang. Besar harapan kami agar laporan ini dapat memberikan manfaat, menambah wawasan, serta menjadi referensi dalam memahami proses perancangan dan implementasi sistem basis data relasional.

Purwokerto, 15 Juni 2026

Penyusun

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah.....	2
1.3 Tujuan	2
1.4 Manfaat	2
1.4.1 Manfaat Teoritis.....	3
1.4.2 Manfaat Praktis	3
BAB II ANALISIS KEBUTUHAN.....	4
2.2.1 Manager	5
2.2.2 Admin	5
2.2.3 Resepsionis	5
2.2.4 Tamu	6
2.3 Daftar Kebutuhan Fungsional	6
2.4 Daftar Business Rules	7
BAB III PERANCANGAN	10
3.1 Entity Relationship Diagram (ERD)	10
3.2 Pemetaan ERD Menjadi Skema Relasional (Logical Design).....	10
3.3 Normalisasi	12
3.3.1 Data Tidak Normal (0NF).....	12
3.3.2 First Normal Form (1NF).....	13
3.3.3 Second Normal Form (2NF)	13
3.3.4 Third Normal Form (3NF)	14
3.3.5 Ketergantungan Fungsional	15
3.4 Data Dictionary	15
BAB IV IMPLEMENTASI	20
4.1 Implementasi Struktur Database (DDL)	20
4.1.1 Implementasi Tabel Tipe Kamar	20
4.1.2 Implementasi Tabel Kamar	22
4.1.3 Implementasi Tabel Tamu	23
4.1.4 Implementasi Tabel Pegawai	25
4.1.5 Implementasi Tabel Fasilitas	26
4.1.6 Implementasi Tabel Reservasi	28
4.1.7 Implementasi Tabel Pembayaran	30
4.1.8 Implementasi Tabel Check_in	33
4.1.9 Implementasi Tabel Check_out	34
4.1.10 Implementasi Tabel Ulasan.....	36

4.1.11 Implementasi Tabel Audit Log	38
4.2 Implementasi Data Manipulation Language (DDL)	40
4.2.1 Implementasi Data Tabel Tipe Kamar	40
4.2.2 Implementasi Data Tabel Kamar	41
4.2.3 Implementasi Data Tabel Tamu	43
4.2.4 Implementasi Data Tabel Pegawai	45
4.2.5 Implementasi Data Tabel Pegawai	46
4.2.6 Implementasi Data Tabel Reservasi	47
4.2.7 Implementasi Data Tabel Pembayaran	49
4.2.8 Implementasi Data Tabel Check-in	51
4.2.9 Implementasi Data Tabel Check-out	53
4.2.9 Implementasi Data Tabel Ulasan	55
BAB V PENGUJIAN QUERY	58
5.1 Query SELECT	58
5.1.1 Query 1 - Daftar Kamar Beserta Tipe (Sederhana)	58
5.1.2 Query 2 - Daftar Tamu (Sederhana)	59
5.1.3 Query 3 - Pembayaran via Transfer (Sederhana)	60
5.1.4 Query 4 - Detail Lengkap Reservasi (JOIN >= 3 Tabel)	62
5.1.5 Query 5 - Riwayat Pembayaran Lengkap (JOIN >= 3 Tabel)	64
5.1.6 Query 6 - Detail Check-In dan Pegawai (JOIN >= 3 Tabel)	66
5.1.7 Query 7 - Ulasan Tamu Beserta Detail Kamar (JOIN >= 3 Tabel)	68
5.1.8 Query 8 - Tamu di Atas Rata-rata Pembayaran (Subquery)	70
5.1.9 Query 9 - Ranking Tipe Kamar dengan CTE dan Window Function	72
5.1.10 Query 10 - Pegawai dengan Transaksi Terbanyak (Agregat + GROUP BY + HAVING)	74
5.2 VIEW	76
5.2.1 VIEW v_reservasi_lengkap	76
5.2.2 VIEW v_statistik_kamar	79
5.3 STORED PROCEDURE	81
5.3.1 sp_buat_reservasi	81
5.4 TRIGGER	83
5.4.1 trg_after_checkout_update_kamar	83
5.5 FUNCTION	86
5.5.1 fn_hitung_total_tagihan	86
BAB VI PENUTUP	89
6.1 Kesimpulan	89
6.2 Saran	89
LAMPIRAN	90

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi yang semakin pesat membawa dampak besar terhadap berbagai sektor industri, termasuk industri perhotelan. Hotel sebagai salah satu usaha jasa akomodasi yang sangat dinamis dituntut untuk mampu memberikan pelayanan yang cepat, akurat, dan berkualitas tinggi kepada setiap tamunya. Untuk memenuhi tuntutan tersebut, diperlukan sebuah sistem pengelolaan informasi yang terstruktur, efisien, dan dapat diandalkan.

Tanpa adanya sistem yang baik, pengelolaan operasional hotel yang kompleks seperti proses reservasi kamar, pencatatan data tamu, pengelolaan check-in dan check-out, serta penanganan pembayaran akan sangat rentan terhadap berbagai permasalahan. Permasalahan tersebut antara lain adalah terjadinya double booking karena tidak adanya pencatatan terpusat, lambatnya pelayanan akibat pencarian data yang tidak efisien, terjadinya inkonsistensi dan redundansi data, serta sulitnya manajemen dalam mengakses laporan dan informasi secara *real-time*.

Sistem basis data relasional hadir sebagai solusi yang tepat untuk mengatasi permasalahan tersebut. Dengan menggunakan model basis data relasional, seluruh data yang berkaitan satu sama lain dapat disimpan secara terstruktur dalam bentuk tabel-tabel yang terhubung melalui relasi yang jelas. Pendekatan ini memungkinkan proses penyimpanan, pengambilan, pembaruan, dan penghapusan data dilakukan secara efisien serta meminimalisasi terjadinya anomali data.

Dalam konteks sistem reservasi hotel, terdapat berbagai entitas data yang saling berinteraksi. Seorang tamu yang ingin menginap akan melakukan reservasi untuk memesan kamar tertentu sesuai dengan tipe kamar yang diinginkan. Proses reservasi tersebut kemudian akan menghasilkan proses check-in yang dilayani oleh pegawai hotel, dan diakhiri dengan proses check-out serta pembayaran. Di samping itu, hotel juga menyediakan berbagai fasilitas pendukung yang dapat digunakan oleh tamu, serta tamu dapat memberikan ulasan atas pelayanan yang diterima. Seluruh interaksi antar entitas ini membutuhkan sebuah sistem basis data yang mampu merepresentasikan dan mengelola hubungan-hubungan tersebut secara akurat. Oleh karena itu, diperlukan sebuah perancangan basis data yang komprehensif agar dapat mendukung seluruh operasional hotel secara terintegrasi.

Berdasarkan permasalahan dan kebutuhan di atas, penyusun merancang dan mengimplementasikan sebuah sistem basis data reservasi hotel. Sistem ini dirancang untuk mencakup minimal 10 entitas utama, yaitu Tamu, Kamar, Tipe_Kamar, Reservasi, Check_In, Check_Out, Pembayaran, Fasilitas, Pegawai, dan Ulasan. Perancangan sistem dilakukan secara menyeluruh mulai dari tahap identifikasi kebutuhan, pembuatan *Entity Relationship Diagram* (ERD), normalisasi dari 1NF hingga 3NF, implementasi DDL dan DML, hingga pembuatan objek basis data lanjutan seperti *view*, *stored procedure*, *trigger*, dan *function*.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, rumusan masalah dalam perancangan sistem basis data reservasi hotel ini adalah sebagai berikut:

1. Bagaimana merancang *Entity Relationship Diagram* (ERD) dan skema relasional yang mampu merepresentasikan seluruh entitas, relasi, dan kardinalitas dalam sistem reservasi hotel secara tepat?
2. Bagaimana mengimplementasikan basis data relasional sistem reservasi hotel menggunakan DDL dan DML yang memenuhi kaidah normalisasi hingga bentuk normal ketiga (3NF)?
3. Bagaimana membuat *query*, *view*, *stored procedure*, *trigger*, dan *function* yang mendukung kebutuhan operasional sistem reservasi hotel?

1.3 Tujuan

Berdasarkan rumusan masalah yang telah disusun, tujuan dari perancangan sistem basis data reservasi hotel ini adalah sebagai berikut:

1. Merancang *Entity Relationship Diagram* (ERD) dan skema relasional yang mampu merepresentasikan seluruh entitas, relasi, dan kardinalitas dalam reservasi hotel secara tepat.
2. Mengimplementasikan sistem basis data reservasi hotel menggunakan DDL dan DML yang memenuhi kaidah normalisasi hingga bentuk normal ketiga (3NF).
3. Membuat *query*, *view*, *stored procedure*, *trigger*, dan *function* untuk mendukung kebutuhan operasional sistem reservasi hotel secara efektif dan efisien.

1.4 Manfaat

Adapun manfaat yang diharapkan dari perancangan sistem basis data reservasi hotel ini adalah sebagai berikut:

1.4.1 Manfaat Teoritis

1. Menambah wawasan dan pemahaman mengenai konsep perancangan basis data relasional, mulai dari identifikasi kebutuhan data, pembuatan Entity Relationship Diagram (ERD), normalisasi, hingga implementasi basis data menggunakan MYSQL.
2. Menjadi sarana penerapan teori yang telah dipelajari pada mata kuliah Basis Data, khususnya dalam penggunaan Data Definition Language (DDL), Data Manipulation Language (DML), view, stored procedure, trigger, dan function.
3. Menjadi referensi bagi mahasiswa atau peneliti lain yang ingin mempelajari dan mengembangkan sistem basis data pada bidang perhotelan maupun sistem informasi yang memiliki karakteristik serupa.

1.4.2 Manfaat Praktis

1. Membantu proses pengelolaan data reservasi hotel agar lebih terstruktur, terintegrasi, dan mudah diakses.
2. Mengurangi risiko terjadinya kesalahan pencatatan data, duplikasi data, serta inkonsistensi informasi yang dapat mengganggu operasional hotel.
3. Mendukung proses pelayanan hotel, seperti reservasi kamar, check-in, check-out, pembayaran, dan pengelolaan fasilitas, sehingga dapat dilakukan dengan lebih cepat dan efisien.
4. Memudahkan pihak manajemen hotel dalam memperoleh informasi dan laporan yang dibutuhkan untuk mendukung pengambilan keputusan.
5. Menjadi model dasar yang dapat dikembangkan lebih lanjut menjadi sistem informasi hotel yang lebih lengkap dan sesuai dengan kebutuhan operasional di dunia nyata.

BAB II

ANALISIS KEBUTUHAN

2.1 Deskripsi Studi Kasus dan Ruang Lingkup Sistem

Sistem Informasi Reservasi Hotel merupakan sistem yang dirancang untuk membantu proses pengelolaan data dan transaksi yang terjadi pada sebuah hotel. Sistem ini digunakan untuk mengelola berbagai informasi yang berkaitan dengan operasional hotel, seperti data tamu, data kamar, data tipe kamar, data fasilitas, data pegawai, reservasi, proses check-in dan check-out, pembayaran, serta ulasan dari tamu. Seluruh data tersebut disimpan dan dikelola secara langsung dalam satu basis data sehingga memudahkan proses pengelolaan informasi.

Dalam kegiatan operasional hotel, proses pencatatan yang masih dilakukan secara manual sering kali menimbulkan berbagai permasalahan, seperti kesalahan pencatatan data, ketidaksesuaian informasi, kesulitan dalam mengetahui ketersediaan kamar, serta lamanya proses pembuatan laporan. Selain itu, penyimpanan data secara manual juga berisiko menyebabkan kehilangan data dan duplikasi informasi. Oleh karena itu, diperlukan suatu sistem informasi yang mampu mengelola seluruh data dan transaksi secara terstruktur, sehingga dapat meningkatkan efisiensi kerja serta mendukung pelayanan yang lebih baik kepada pelanggan.

Ruang lingkup sistem yang akan dirancang mencakup pengelolaan seluruh informasi yang berkaitan dengan kegiatan operasional hotel, mulai dari pengelolaan data tamu, kamar, tipe kamar, fasilitas, pegawai, reservasi, check-in, check-out, pembayaran, dan ulasan pelanggan. Selain itu, sistem juga menyediakan fasilitas untuk menampilkan dan mencetak laporan transaksi sebagai bahan evaluasi serta pendukung pengambilan keputusan bagi pihak manajemen hotel.

2.2 Identifikasi Pengguna

Identifikasi pengguna dilakukan untuk mengetahui pihak-pihak yang berinteraksi secara langsung dengan Sistem Informasi Reservasi Hotel. Setiap pengguna memiliki tugas dan hak akses yang berbeda sesuai dengan perannya dalam mengatur operasional hotel. Pembagian hak akses ini bertujuan untuk menjaga keamanan data serta memastikan setiap proses dapat berjalan secara efektif dan teratur.

Pada sistem yang dirancang, terdapat empat jenis pengguna, yaitu Manajer, Admin, Resepsionis, dan Tamu.

2.2.1 Manager

Manager merupakan pengguna yang memiliki wewenang untuk memantau dan mengevaluasi operasional hotel melalui informasi yang tersedia pada sistem. Manajer tidak terlibat langsung dengan transaksi, tetapi berperan dalam pengambilan keputusan berdasarkan laporan yang dihasilkan oleh sistem.

Hak akses manajer meliputi :

1. Melihat data reservasi
2. Melihat data Pembayaran
3. Melihat tingkat penggunaan kamar
4. Melihat data ulasan pelanggan
5. Melihat dan mencetak laporan operasional hotel

2.2.2 Admin

Admin merupakan pengguna yang bertanggung jawab dalam mengelola data utama yang digunakan oleh sistem. Admin memiliki hak akses untuk melakukan pengelolaan data agar informasi yang tersimpan dalam basis data tetap lengkap dan konsisten.

Hak akses admin meliputi :

1. Mengelola data pegawai
2. Mengelola data kamar
3. Mengelola data tipe kamar
4. Mengelola data fasilitas
5. Menggunakan data pengguna sistem
6. Menggunakan data ulasan
7. Melihat dan mencetak laporan

2.2.3 Resepsionis

Resepsionis merupakan pengguna yang berinteraksi langsung dengan tamu hotel dalam proses pelayanan. Resepsionis bertugas mengelola transaksi yang berkaitan dengan reservasi dan administrasi tamu selama menginap.

Hak akses resepsionis meliputi:

1. Mengelola data tamu
2. Melakukan proses reservasi kamar
3. Melakukan proses check-in
4. Melakukan proses check-out
5. Mencatat data pembayaran

6. Melihat status ketersediaan kamar
7. Mencetak bukti reservasi dan pembayaran

2.2.4 Tamu

Tamu merupakan pengguna yang memanfaatkan sistem untuk memperoleh informasi mengenai layanan hotel dan melakukan reservasi secara mandiri. Selain itu, tamu juga dapat memberikan ulasan terhadap pelayanan hotel setelah proses menginap selesai.

Hak akses Tamu meliputi:

1. Melakukan registrasi akun.
2. Melihat informasi kamar dan fasilitas hotel
3. Melakukan reservasi kamar
4. Melihat status reservasi
5. Memberikan ulasan atau penilaian terhadap pelayanan hotel

Dengan adanya empat jenis pengguna tersebut, setiap aktivitas dalam Sistem Informasi Reservasi Hotel dapat dilakukan sesuai dengan tugas dan wewenang masing-masing pengguna, sehingga pengelolaan data dan proses informasi hotel dapat berjalan secara lebih efektif, aman, dan teratur.

2.3 Daftar Kebutuhan Fungsional

Kebutuhan fungsional terkait data merupakan fungsi-fungsi yang harus dimiliki oleh sistem dalam mengelola, menyimpan, memperbarui, dan menampilkan data yang berkaitan dengan operasional hotel. Fungsi-fungsi tersebut dirancang agar seluruh proses bisnis dapat berjalan secara efektif dan terintegrasi.

Berikut adalah kebutuhan fungsional pada sistem informasi reservasi hotel :

1. Sistem dapat menyimpan, mengubah, menghapus dan menampilkan data tamu
2. Sistem dapat menyimpan, mengubah, menghapus dan menampilkan data pegawai
3. Sistem dapat menyimpan, mengubah, menghapus dan menampilkan data kamar
4. Sistem dapat menyimpan, mengubah, menghapus dan menampilkan data tipe kamar
5. Sistem dapat menyimpan, mengubah, menghapus dan menampilkan data fasilitas hotel
6. Sistem dapat menyimpan, mengubah, menghapus, dan menampilkan data reservasi kamar
7. Sistem dapat menyimpan dan menampilkan data check-in tamu
8. Sistem dapat menyimpan dan menampilkan data check-out tamu
9. Sistem dapat menyimpan, mengubah, dan menampilkan data pembayaran

10. Sistem dapat menyimpan, mengubah, menghapus, dan menampilkan data ulasan pelanggan
11. Sistem dapat menampilkan informasi ketersediaan kamar berdasarkan data reservasi dan status kamar
12. Sistem dapat memperbarui status kamar secara otomatis setelah proses check-in dan check-out
13. Sistem dapat menghitung total biaya menginap berdasarkan tipe kamar dan lama menginap
14. Sistem dapat menampilkan riwayat reservasi dan pembayaran setiap tamu
15. Sistem dapat menghasilkan laporan data reservasi, pembayaran, dan penggunaan kamar berdasarkan periode tertentu
16. Sistem menyediakan fasilitas pencarian data berdasarkan nama tamu, nomor reservasi, nomor kamar, atau data lainnya yang relevan
17. Sistem mampu menjaga hubungan antar data sehingga informasi yang tersimpan tetap konsisten dan saling terhubung

2.4 Daftar *Business Rules*

Tabel 1. Daftar *Business Rules*

No	Entitas 1	Relasi	Entitas 2	Kardinalitas
1	TAMU	melakukan	RESERVASI	1:M
2	RESERVASI	memesan	KAMAR	M:1
3	KAMAR	memiliki	TIPE_KAMAR	M:1
4	RESERVASI	melakukan	PEMBAYARAN	1:1
5	RESERVASI	menghasilkan	CHECK_IN	1:1
6	RESERVASI	menghasilkan	CHECK_OUT	1:1
7	PEGAWAI	melayani	CHECK_IN	1:M
8	PEGAWAI	melayani	CHECK_OUT	1:M
9	KAMAR	menyediakan	FASILITAS	1:M
10	RESERVASI	memberikan	ULASAN	1:1

Sistem reservasi hotel dibangun berdasarkan sejumlah aturan bisnis yang mengatur hubungan antar entitas dalam basis data. Setiap tamu dapat melakukan satu atau lebih reservasi sesuai dengan kebutuhan menginapnya. Namun, setiap reservasi hanya dapat dibuat oleh satu tamu. Aturan ini memungkinkan sistem menyimpan riwayat pemesanan yang dilakukan oleh setiap pelanggan secara terstruktur.

Setiap reservasi digunakan untuk memesan satu kamar tertentu. Sebaliknya, satu kamar dapat dipesan melalui banyak reservasi pada waktu yang berbeda. Dengan aturan ini, sistem dapat mengelola penggunaan kamar dan memantau ketersediaannya selama periode operasional hotel.

Setiap kamar harus termasuk ke dalam satu tipe kamar tertentu, seperti Standard, Deluxe, atau Suite. Sementara itu, satu tipe kamar dapat dimiliki oleh banyak kamar. Pengelompokan ini memudahkan pengelolaan informasi kamar berdasarkan karakteristik, kapasitas, dan harga yang dimiliki.

Setiap reservasi menghasilkan satu transaksi pembayaran yang digunakan untuk mencatat proses pelunasan biaya menginap. Oleh karena itu, setiap pembayaran hanya dapat terkait dengan satu reservasi, dan setiap reservasi hanya memiliki satu data pembayaran yang tercatat dalam sistem.

Selain pembayaran, setiap reservasi juga menghasilkan satu proses check-in dan satu proses check-out. Data check-in digunakan untuk mencatat kedatangan tamu, sedangkan data check-out digunakan untuk mencatat tamu setelah masa menginap selesai. Dengan demikian, setiap reservasi memiliki satu catatan check-in dan satu catatan check-out yang saling berkaitan.

Dalam pelaksanaan operasional hotel, setiap proses check-in dan check-out dilayani oleh pegawai yang bertugas. Seorang pegawai dapat melayani banyak proses check-in maupun check-out, sedangkan setiap transaksi check-in dan check-out hanya ditangani oleh satu pegawai yang bertanggung jawab pada saat proses berlangsung.

Setiap kamar dapat menyediakan berbagai fasilitas untuk menunjang kenyamanan tamu selama menginap. Hubungan antara entitas Kamar dan Fasilitas bersifat one-to-many (1:M), yang berarti satu kamar dapat memiliki banyak fasilitas, sedangkan setiap fasilitas hanya terkait dengan satu kamar. Hubungan ini memungkinkan sistem mengelola data fasilitas yang tersedia pada masing-masing kamar secara terstruktur dan konsisten.

Setelah menyelesaikan masa menginapnya, tamu dapat memberikan ulasan terhadap pelayanan hotel berdasarkan reservasi yang telah dilakukan. Setiap reservasi hanya dapat menghasilkan satu ulasan, dan setiap ulasan hanya terkait dengan satu reservasi tertentu. Ulasan yang diberikan dapat berupa rating dan komentar yang digunakan sebagai bahan evaluasi kualitas pelayanan hotel.

Aturan-aturan bisnis tersebut menjadi dasar dalam perancangan basis data dan memastikan bahwa seluruh proses reservasi, pembayaran, pelayanan tamu, serta

pengelolaan kamar dapat berjalan secara konsisten, terintegrasi, dan sesuai dengan kebutuhan operasional hotel.

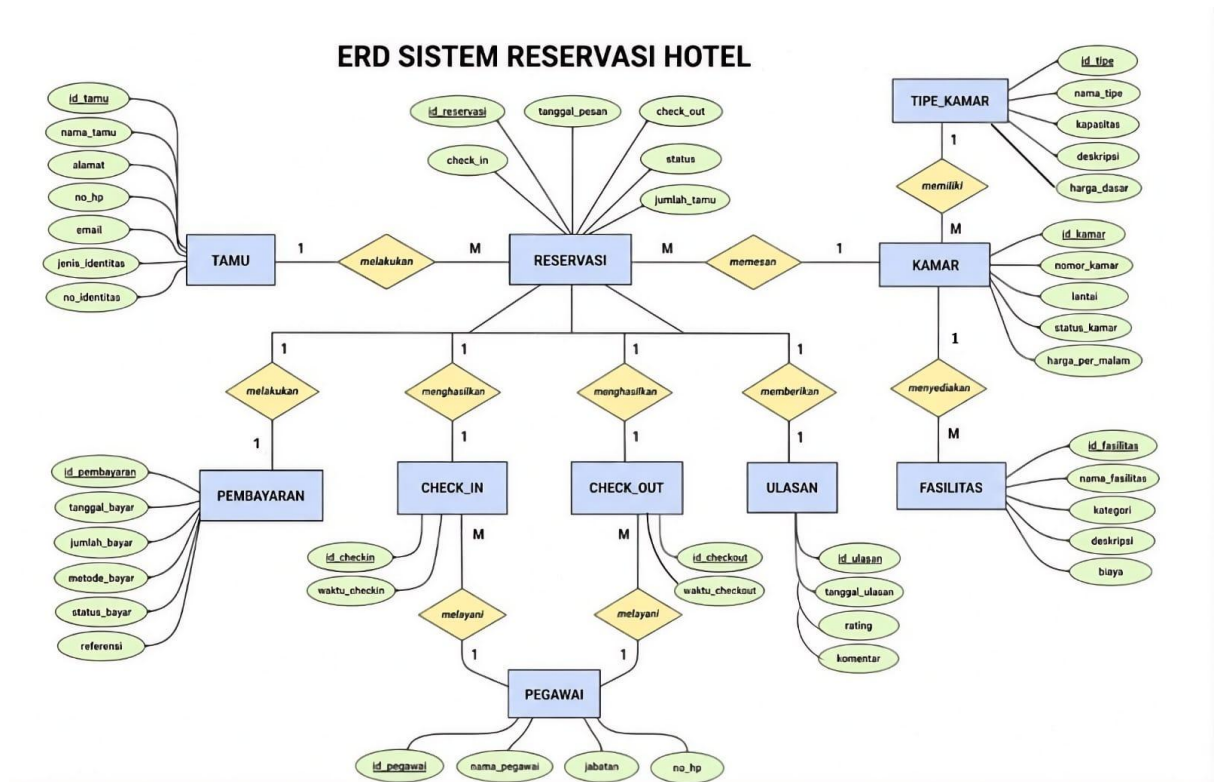
Untuk Business Rules pada sistem reservasi hotel yang lain yaitu:

1. Kamar tidak bisa dipakai dua tamu sekaligus. Kamar yang sedang ditempati tamu tidak boleh dipesan atau di-check-in oleh tamu lain, sampai tamu sebelumnya sudah checkout.
2. Tanggal check-in harus sebelum check-out. Tanggal check-in wajib lebih awal dari tanggal check-out, dengan minimal menginap 1 malam. Reservasi tidak bisa dibuat untuk hari yang sudah lewat.
3. Check-in hanya boleh jika sudah bayar. Tamu hanya bisa melakukan check-in jika pembayaran sudah lunas (atau sudah membayar uang muka sesuai kebijakan hotel). Reservasi yang belum dibayar tidak boleh diproses check-in.
4. Ulasan hanya bisa diberikan setelah checkout. Tamu hanya bisa memberikan ulasan jika reservasinya sudah selesai (sudah checkout). Tamu yang masih menginap atau yang membatalkan reservasi tidak bisa memberi ulasan.

BAB III

PERANCANGAN

3.1 Entity Relationship Diagram (ERD)



Gambar 3.1 Entity Relationship Diagram (ERD)

3.2 Pemetaan ERD Menjadi Skema Relasional (Logical Design)

Tahap logical design dilakukan dengan mengubah setiap entitas pada Entity Relationship Diagram (ERD) menjadi tabel relasional yang dapat diimplementasikan pada sistem manajemen basis data. Setiap tabel memiliki primary key sebagai identitas unik dan foreign key sebagai penghubung antar tabel sesuai dengan hubungan yang telah ditentukan pada ERD.

Entitas Tamu dipetakan menjadi tabel TAMU yang memiliki atribut id_tamu sebagai primary key, serta atribut nama_tamu, alamat, no_hp, email, jenis_identitas, dan no_identitas. Tabel ini digunakan untuk menyimpan seluruh informasi pelanggan yang melakukan reservasi hotel.

Entitas Tipe_Kamar dipetakan menjadi tabel TIPE_KAMAR dengan id_tipe sebagai primary key. Tabel ini menyimpan informasi mengenai jenis kamar yang tersedia di

hotel, meliputi nama_tipe, kapasitas, deskripsi, dan harga_dasar. Setiap tipe kamar dapat dimiliki oleh beberapa kamar yang berbeda.

Entitas Kamar dipetakan menjadi tabel KAMAR yang memiliki id_kamar sebagai primary key. Selain menyimpan informasi nomor_kamar, lantai, status_kamar, dan harga_per_malam, tabel ini juga memiliki atribut id_tipe sebagai foreign key yang menghubungkan setiap kamar dengan tipe kamar yang dimilikinya. Hubungan antara TIPE_KAMAR dan KAMAR bersifat one-to-many (1:M), sehingga satu tipe kamar dapat dimiliki oleh banyak kamar.

Entitas Reservasi dipetakan menjadi tabel RESERVASI dengan id_reservasi sebagai primary key. Tabel ini menyimpan data transaksi pemesanan yang meliputi tanggal_pesan, check_in, check_out, status, dan jumlah_tamu. Selain itu, tabel RESERVASI memiliki foreign key id_tamu dan id_kamar yang menghubungkan reservasi dengan tamu yang melakukan pemesanan serta kamar yang dipesan. Dengan demikian, satu tamu dapat memiliki banyak reservasi dan satu kamar dapat digunakan dalam banyak reservasi pada waktu yang berbeda.

Entitas Pembayaran dipetakan menjadi tabel PEMBAYARAN yang memiliki id_pembayaran sebagai primary key. Tabel ini berisi informasi tanggal_bayar, jumlah_bayar, metode_bayar, status_bayar, dan referensi. Untuk menghubungkan pembayaran dengan reservasi, digunakan atribut id_reservasi sebagai foreign key. Karena hubungan antara RESERVASI dan PEMBAYARAN bersifat one-to-one (1:1), setiap reservasi hanya memiliki satu data pembayaran dan setiap pembayaran hanya terkait dengan satu reservasi.

Entitas Check_In dipetakan menjadi tabel CHECK_IN yang memiliki id_checkin sebagai primary key dan atribut waktu_checkin. Tabel ini juga memiliki foreign key id_reservasi dan id_pegawai untuk menghubungkan data check-in dengan reservasi yang bersangkutan serta pegawai yang melayani proses check-in. Hubungan antara RESERVASI dan CHECK_IN bersifat one-to-one (1:1), sedangkan hubungan antara PEGAWAI dan CHECK_IN bersifat one-to-many (1:M).

Entitas Check_Out dipetakan menjadi tabel CHECK_OUT dengan id_checkout sebagai primary key dan atribut waktu_checkout. Tabel ini memiliki foreign key id_reservasi dan id_pegawai untuk menunjukkan reservasi yang diselesaikan dan pegawai yang melayani proses check-out. Sama seperti check-in, hubungan antara RESERVASI dan CHECK_OUT bersifat one-to-one (1:1), sedangkan hubungan antara PEGAWAI dan CHECK_OUT bersifat one-to-many (1:M).

Entitas Pegawai dipetakan menjadi tabel PEGAWAI yang memiliki id_pegawai sebagai primary key. Tabel ini menyimpan informasi pegawai berupa nama_pegawai, jabatan, dan no_hp. Data pegawai digunakan sebagai referensi pada proses check-in dan check-out untuk mengetahui petugas yang memberikan layanan kepada tamu.

Entitas Ulasan dipetakan menjadi tabel ULASAN yang memiliki id_ulasan sebagai primary key. Tabel ini menyimpan informasi tanggal_ulasan, rating, dan komentar yang diberikan oleh tamu setelah menyelesaikan masa menginapnya. Atribut id_reservasi digunakan sebagai foreign key untuk menghubungkan ulasan dengan reservasi tertentu. Hubungan antara RESERVASI dan ULASAN bersifat one-to-one (1:1).

Entitas Fasilitas dipetakan menjadi tabel FASILITAS yang memiliki id_fasilitas sebagai primary key. Tabel ini menyimpan informasi nama_fasilitas, kategori, deskripsi, dan biaya. Selain itu, tabel FASILITAS memiliki atribut id_kamar sebagai foreign key yang menghubungkan fasilitas dengan kamar yang menyediakannya. Hubungan antara KAMAR dan FASILITAS bersifat one-to-many (1:M), yang berarti satu kamar dapat memiliki banyak fasilitas, sedangkan setiap fasilitas hanya terkait dengan satu kamar.

Hasil pemetaan tersebut menghasilkan sepuluh tabel utama, yaitu TAMU, TIPE_KAMAR, KAMAR, RESERVASI, PEMBAYARAN, CHECK_IN, CHECK_OUT, PEGAWAI, ULASAN, dan FASILITAS. Struktur relasional ini telah mampu merepresentasikan seluruh kebutuhan data pada sistem reservasi hotel serta menjaga integritas data melalui penggunaan primary key dan foreign key pada setiap hubungan antar tabel.

3.3 Normalisasi

3.3.1 Data Tidak Normal (0NF)

Pada tahap ini seluruh data masih disimpan dalam satu tabel besar yang menggabungkan data reservasi, tamu, kamar, tipe kamar, pembayaran, pegawai, fasilitas, check-in, check-out, dan ulasan. Struktur seperti ini menyebabkan terjadinya redundansi. Struktur tabel 0NF dapat digambarkan sebagai berikut:

Tabel 2. Data tidak normal

0NF(id_reservasi, tanggal_pesan, id_check_in, check_out, jumlah_tamu, id_tamu, nama_tamu, id_kamar, nomor_kamar, id_tipe, nama_tipe, id_reservasi.1, tanggal_pesan.1, check_in.1, check_out.1, jumlah_tamu.1, id_tamu.1, nama_tamu.1,

```
alamat, no_hp, email, jenis_identitas, no_identitas, id_kamar.1, nomor_kamar.1,
lantai, status_kamar, harga_per_malam, id_tipe.1, nama_tipe.1, kapasitas,
deskripsi_tipe, harga_dasar, id_pembayaran, tanggal_bayar, jumlah_bayar,
metode_bayar, status_bayar, referensi, id_checkin, waktu_checkin, id_checkout,
waktu_checkout, id_pegawai, nama_pegawai, jabatan, no_hp_pegawai, id_fasilitas,
nama_fasilitas, kategori, deskripsi_fasilitas, biaya, id_ulasan, tanggal_ulasan, rating,
komentar, status_reservasi
)
```

3.3.2 First Normal Form (1NF)

Pada tahap 1NF, setiap atribut telah memiliki nilai yang bersifat atomik dan tidak ada lagi kelompok data yang berulang dalam satu kolom. Namun, seluruh data masih berada dalam satu tabel sehingga redundansi data masih terjadi. Struktur tabel 1NF dapat digambarkan sebagai berikut :

Tabel 3. First Normal Form (1NF)

```
1NF(
id_reservasi, tanggal_pesan, id_check_in, check_out, jumlah_tamu, status_reservasi,
id_tamu, nama_tamu, alamat, no_hp, email, jenis_identitas, no_identitas, id_kamar,
nomor_kamar, lantai, status_kamar, harga_per_malam, id_tipe, nama_tipe, kapasitas,
deskripsi_tipe, harga_dasar, id_pembayaran, tanggal_bayar, jumlah_bayar,
metode_bayar, status_bayar, referensi, id_checkin, waktu_checkin, id_checkout,
waktu_checkout, id_pegawai, nama_pegawai, jabatan, no_hp_pegawai, id_fasilitas,
nama_fasilitas, kategori, deskripsi_fasilitas, biaya, id_ulasan, tanggal_ulasan, rating,
komentar
)
```

3.3.3 Second Normal Form (2NF)

Pada tahap 2NF, dilakukan pemisahan tabel untuk menghilangkan ketergantungan parsial. Setiap atribut non-key telah bergantung sepenuhnya pada primary key masing-masing tabel. Hasil normalisasi menghasilkan beberapa beberapa tabel sebagai berikut :

Tabel 4. Second Normal Form (2NF)

Nama Tabel	Atribut
Tamu	id_tamu, nama_tamu, alamat, no_hp, email, jenis_identitas, no_identitas
Tipe_Kamar	id_tipe, nama_tipe, kapasitas, deskripsi_tipe, harga_dasar
Kamar	id_kamar, nomor_kamar, lantai, status_kamar, harga_per_malam, id_tipe
Reservasi	id_reservasi, tanggal_pesan, check_in, check_out, jumlah_tamu, status_reservasi, id_tamu, id_kamar
Pembayaran	id_pembayaran, tanggal_bayar, jumlah_bayar, metode_bayar, status_bayar, referensi, id_reservasi
Check_In	id_checkin, waktu_checkin, id_reservasi, id_pegawai
Check_Out	id_checkout, waktu_checkout, id_reservasi, id_pegawai
Pegawai	id_pegawai, nama_pegawai, jabatan, no_hp
Fasilitas	id_fasilitas, nama_fasilitas, kategori, deskripsi_fasilitas, biaya
Ulasan	id_ulasan, tanggal_ulasan, rating, komentar, id_reservasi

3.3.4 Third Normal Form (3NF)

Pada tahap 3NF dilakukan penghilangan ketergantungan transitif sehingga setiap atribut non-key hanya bergantung pada primary key tabel masing-masing.

Tabel 5. Third Normal Form (3NF)

Nama Tabel	Atribut
Tamu	id_tamu, nama_tamu, alamat, no_hp, email, jenis_identitas, no_identitas
Tipe_Kamar	id_tipe, nama_tipe, kapasitas, deskripsi_tipe, harga_dasar
Kamar	id_kamar, nomor_kamar, lantai, status_kamar, id_tipe
Reservasi	id_reservasi, tanggal_pesan, check_in, check_out, jumlah_tamu, status_reservasi, id_tamu, id_kamar
Pembayaran	id_pembayaran, tanggal_bayar, jumlah_bayar, metode_bayar, status_bayar, referensi, id_reservasi
Check_In	id_checkin, waktu_checkin, id_reservasi, id_pegawai
Check_Out	id_checkout, waktu_checkout, id_reservasi, id_pegawai

Pegawai	id_pegawai, nama_pegawai, jabatan, no_hp
Fasilitas	id_fasilitas, nama_fasilitas, kategori, deskripsi_fasilitas, biaya
Ulasan	id_ulasan , tanggal_ulasan, rating, komentar, id_reservasi
Reservasi_Fasilitas	id_reservasi, id_fasilitas

3.3.5 Ketergantungan Fungsional

Ketergantungan fungsional (functional dependency) digunakan untuk menunjukkan hubungan antara atribut penentu (determinan) dengan atribut yang ditentukan dalam suatu tabel. Ketergantungan fungsional ini menjadi dasar dalam proses normalisasi untuk mengurangi redundansi data dan menjaga konsistensi informasi pada basis data. Berdasarkan hasil normalisasi, ketergantungan fungsional pada Sistem Informasi Reservasi Hotel adalah sebagai berikut:

Tabel 6. Ketergantungan Fungsional

id_tamu → nama_tamu, alamat, no_hp, email, jenis_identitas, no_identitas
id_pegawai → nama_pegawai, jabatan, no_hp_pegawai
id_tipe → nama_tipe, kapasitas, deskripsi_tipe, harga_dasar
id_kamar → nomor_kamar, lantai, status_kamar, harga_per_malam, id_tipe
id_reservasi → tanggal_pesan, jumlah_tamu, status_reservasi, id_tamu, id_kamar
id_pembayaran → tanggal_bayar, jumlah_bayar, metode_bayar, status_bayar, referensi
id_checkin → waktu_checkin
id_checkout → waktu_checkout
id_fasilitas → nama_fasilitas, kategori, deskripsi_fasilitas, biaya
id_ulasan → tanggal_ulasan, rating, komentar

Berdasarkan ketergantungan fungsional tersebut, setiap atribut *non-key* bergantung sepenuhnya pada *primary key* tabel masing-masing. Dengan demikian, struktur basis data yang dihasilkan telah memenuhi Bentuk Normal Ketiga (*Third Normal Form/3NF*) dan mampu meminimalkan redundansi data serta menjaga integritas data.

3.4 Data Dictionary

Tabel 7. Data Tabel Tamu

Nama Kolom	Tipe Data	Constraint	Deskripsi
id_tamu	VARCHAR(7)	PK, NOT NULL	Kode unik tamu
nama_tamu	VARCHAR(100)	NOT NULL	Nama lengkap tamu
alamat	VARCHAR(200)	NULL	Alamat tamu
no_hp	VARCHAR(20)	NOT NULL	Nomor telepon tamu
email	VARCHAR(100)	UNIQUE, NULL	Alamat email tamu
jenis_identitas	ENUM('KTP','SIM','Passport')	NOT NULL, DEFAULT 'KTP'	Jenis identitas tamu
no_identitas	VARCHAR(20)	NOT NULL	Nomor identitas tamu

Tabel 8. Data Tabel Tipe Kamar

Nama Kolom	Tipe Data	Constraint	Deskripsi
id_tipe	VARCHAR(5)	PK, NOT NULL	Kode unik tipe kamar
nama_tipe	VARCHAR(50)	UNIQUE, NOT NULL	Nama tipe kamar
kapasitas	TINYINT	NOT NULL, CHECK (kapasitas > 0)	Jumlah maksimum tamu yang dapat ditampung
deskripsi_tipe	VARCHAR(200)	NULL	Deskripsi tipe kamar
harga_dasar	DECIMAL(12,2)	NOT NULL, CHECK	Harga dasar kamar per malam

		(harga_dasar ≥ 0)	
--	--	-------------------	--

Tabel 9. Data Tabel Kamar

Nama Kolom	Tipe Data	Constraint	Deskripsi
id_kamar	VARCHAR(5)	PK, NOT NULL	Kode unik kamar
nomor_kamar	VARCHAR(10)	NOT NULL	Nomor kamar hotel
lantai	TINYINT	NOT NULL, CHECK (lantai > 0)	Nomor lantai tempat kamar berada
status_kamar	ENUM('Tersedia', 'Terisi', 'Dipesan', 'Maintenance')	NOT NULL, DEFAULT 'Tersedia'	Status ketersediaan kamar
id_tipe	VARCHAR(5)	FK, NOT NULL	Kode tipe kamar yang dimiliki kamar

Tabel 10. Data Tabel Reservasi

Nama Kolom	Tipe Data	Constraint	Deskripsi
id_reservasi	VARCHAR(5)	PK, NOT NULL	Kode unik reservasi
tanggal_pesan	DATE	NOT NULL	Tanggal pemesanan kamar dilakukan

check_in	DATE	NOT NULL	Tanggal rencana check-in
check_out	DATE	NOT NULL	Tanggal rencana check-out
jumlah_tamu	TINYINT	NOT NULL, CHECK (jumlah_tamu > 0)	Jumlah tamu yang menginap
status_reservasi	ENUM('Dipesan','Check-In','Selesai','Batal')	NOT NULL, DEFAULT 'Dipesan'	Status reservasi
id_tamu	VARCHAR(7)	FK, NOT NULL	Kode tamu yang melakukan reservasi
id_kamar	VARCHAR(5)	FK, NOT NULL	Kode kamar yang dipesan

Tabel 11. Data Tabel Pembayaran

Nama Kolom	Tipe Data	Constraint	Deskripsi
id_pembayaran	VARCHAR(7)	PK, NOT NULL	Kode unik pembayaran
tanggal_bayar	DATE	NOT NULL	Tanggal pembayaran dilakukan
jumlah_bayar	DECIMAL(12,2)	NOT NULL, CHECK (jumlah_bayar > 0)	Nominal pembayaran

metode_bayar	ENUM('Transfer','QRIS','Cash','Kartu Kredit')	NOT NULL	Metode pembayaran yang digunakan
status_bayar	ENUM('Lunas','Pending','Gagal','Refund')	NOT NULL, DEFAULT 'Pending'	Status pembayaran
referensi	VARCHAR(20)	NULL	Nomor referensi transaksi pembayaran
id_reservasi	VARCHAR(5)	FK, UNIQUE, NOT NULL	Kode reservasi yang terkait dengan pembayaran

Tabel 12. Data Tabel Check-In

Nama Kolom	Tipe Data	Constraint	Deskripsi
id_checkin	VARCHAR(7)	PK, NOT NULL	Kode unik check-in
waktu_checkin	DATETIME	NOT NULL	Waktu tamu melakukan check-in
id_reservasi	VARCHAR(5)	FK, UNIQUE, NOT NULL	Kode reservasi yang melakukan check-in
id_pegawai	VARCHAR(5)	FK, NOT NULL	Kode pegawai yang melayani proses check-in

Tabel 13. Data Tabel Check-Out

Nama Kolom	Tipe Data	Constraint	Deskripsi
id_checkout	VARCHAR(7)	PK, NOT NULL	Kode unik check-out

waktu_checkout	DATETIME	NOT NULL	Waktu tamu melakukan check-out
id_reservasi	VARCHAR(5)	FK, UNIQUE, NOT NULL	Kode reservasi yang melakukan check-out
id_pegawai	VARCHAR(5)	FK, NOT NULL	Kode pegawai yang melayani proses check-out

Tabel 14. Data Tabel Pegawai

Nama Kolom	Tipe Data	Constraint	Deskripsi
id_pegawai	VARCHAR(5)	PK, NOT NULL	Kode unik pegawai
nama_pegawai	VARCHAR(100)	NOT NULL	Nama lengkap pegawai
jabatan	VARCHAR(50)	NOT NULL	Jabatan atau posisi pegawai
no_hp	VARCHAR(20)	NOT NULL	Nomor telepon pegawai

Tabel 15. Data Tabel Fasilitas

Nama Kolom	Tipe Data	Constraint	Deskripsi
id_fasilitas	VARCHAR(5)	PK, NOT NULL	Kode unik fasilitas
nama_fasilitas	VARCHAR(100)	UNIQUE, NOT NULL	Nama fasilitas hotel
kategori	VARCHAR(50)	NOT NULL	Kategori fasilitas
deskripsi_fasilitas	VARCHAR(200)	NULL	Deskripsi fasilitas
biaya	DECIMAL(10,2)	NOT NULL, DEFAULT 0, CHECK (biaya ≥ 0)	Biaya penggunaan fasilitas

Tabel 16. Data Tabel Ulasan

Nama Kolom	Tipe Data	Constraint	Deskripsi
id_ulasan	VARCHAR(5)	PK, NOT NULL	Kode unik ulasan
tanggal_ulasan	DATE	NOT NULL	Tanggal ulasan diberikan
rating	TINYINT	NOT NULL, CHECK (rating BETWEEN 1 AND 5)	Nilai penilaian tamu terhadap layanan hotel
komentar	VARCHAR(500)	NULL	Komentar atau masukan dari tamu
id_reservasi	VARCHAR(5)	FK, UNIQUE, NOT NULL	Kode reservasi yang memberikan ulasan

BAB IV

IMPLEMENTASI

4.1 Implementasi Struktur *Database* (DDL)

Tahap awal dalam implementasi sistem reservasi hotel adalah pembuatan *database* yang berfungsi sebagai media penyimpanan seluruh data yang digunakan dalam sistem. *Database* yang dibuat diberi nama *hotel_reservasi*. Setelah *database* berhasil dibuat, langkah berikutnya adalah mengaktifkan *database* tersebut agar dapat digunakan dalam proses pembuatan tabel dan pengelolaan data.

Tabel 17. *Script Pembuatan Database*

```
-- Membuat Database baru
CREATE DATABASE hotel_reservasi;
-- Memanggil database yang akan digunakan
USE hotel_reservasi;
```

Perintah `CREATE DATABASE` digunakan untuk membuat sebuah *database* baru dengan nama *hotel_reservasi*. *Database* ini berfungsi sebagai wadah untuk menyimpan seluruh tabel dan data yang berkaitan dengan sistem reservasi hotel.

Perintah `USE` digunakan untuk memilih dan mengaktifkan *database* *hotel_reservasi* sebagai *database* yang akan digunakan pada sesi kerja saat ini. Setelah perintah tersebut dijalankan, seluruh perintah SQL yang dieksekusi selanjutnya, seperti pembuatan tabel (`CREATE TABLE`), pengisian data (`INSERT`), perubahan data (`UPDATE`), maupun pengambilan data (`SELECT`), akan diterapkan pada *database* *hotel_reservasi*.

4.1.1 Implementasi Tabel Tipe Kamar

Tabel *tipe_kamar* digunakan untuk menyimpan informasi mengenai jenis kamar yang tersedia pada hotel. Tabel ini memiliki atribut *id_tipe* sebagai primary key yang berfungsi sebagai identitas unik setiap tipe kamar. Selain itu terdapat atribut *nama_tipe*, *kapasitas*, *deskripsi_tipe*, dan *harga_dasar*.

Tabel 18. *Script Pembuatan Tabel TIPE_KAMAR*

```
-- 1. TABEL TIPE_KAMAR
-- Membuat tabel dengan nama tipe_kamar
CREATE TABLE tipe_kamar (
    id_tipe    VARCHAR(5)  NOT NULL,
```

```
nama_tipe  VARCHAR(50) NOT NULL,  
kapasitas  TINYINT    NOT NULL CHECK (kapasitas > 0),  
deskripsi_tipe VARCHAR(200) NULL,  
harga_dasar DECIMAL(12,2) NOT NULL CHECK (harga_dasar >= 0),  
CONSTRAINT pk_tipe_kamar PRIMARY KEY (id_tipe),  
CONSTRAINT uq_nama_tipe UNIQUE (nama_tipe)  
);
```

Perintah CREATE TABLE tipe_kamar digunakan untuk membuat sebuah tabel baru bernama tipe_kamar pada database. Tabel ini berfungsi sebagai tempat penyimpanan data terkait kategori kamar yang tersedia dalam sistem reservasi hotel.

Atribut id_tipe didefinisikan dengan tipe data VARCHAR(5) dan diberi constraint NOT NULL, yang berarti atribut tersebut wajib diisi dan tidak boleh bernilai kosong. Atribut ini digunakan sebagai identitas unik untuk setiap tipe kamar yang tersedia.

Atribut nama_tipe menggunakan tipe data VARCHAR(50) dengan constraint NOT NULL. Atribut ini digunakan untuk menyimpan nama dari setiap tipe kamar, seperti *Standard, Deluxe, Superior, Family, dan Suite*.

Atribut kapasitas menggunakan tipe data TINYINT yang berfungsi untuk menyimpan jumlah maksimum tamu yang dapat menempati suatu tipe kamar. Constraint CHECK (kapasitas > 0) diterapkan untuk memastikan bahwa nilai kapasitas yang dimasukkan harus lebih besar dari nol sehingga data yang tersimpan tetap valid.

Atribut deskripsi_tipe menggunakan tipe data VARCHAR(200) dan diperbolehkan bernilai kosong (NULL). Atribut ini digunakan untuk menyimpan informasi tambahan mengenai karakteristik atau fasilitas yang dimiliki oleh suatu tipe kamar.

Atribut harga_dasar menggunakan tipe data DECIMAL(12,2) untuk menyimpan nilai tarif kamar dengan tingkat presisi hingga dua angka di belakang koma. Constraint CHECK (harga_dasar >= 0) diterapkan untuk memastikan bahwa harga kamar tidak bernilai negatif.

Constraint PRIMARY KEY pada atribut id_tipe digunakan untuk menjamin bahwa setiap tipe kamar memiliki kode yang unik serta dapat digunakan sebagai identitas utama pada tabel. Selain itu, constraint UNIQUE pada atribut nama_tipe diterapkan untuk mencegah terjadinya duplikasi nama tipe kamar dalam *database*.

4.1.2 Implementasi Tabel Kamar

Tabel kamar digunakan untuk menyimpan informasi mengenai seluruh kamar yang tersedia pada hotel. Data yang disimpan meliputi identitas kamar, nomor kamar, lokasi lantai, status ketersediaan kamar, serta tipe kamar yang dimiliki. Tabel ini memiliki hubungan dengan tabel tipe_kamar melalui atribut id_tipe yang berfungsi sebagai *foreign key*.

Tabel 19. Script Pembuatan Tabel Kamar

```
-- 2. TABEL KAMAR
CREATE TABLE kamar (
  id_kamar    VARCHAR(5) NOT NULL,
  nomor_kamar VARCHAR(10) NOT NULL,
  lantai      TINYINT    NOT NULL CHECK (lantai > 0),
  status_kamar ENUM('Tersedia','Terisi','Dipesan','Maintenance') NOT NULL
  DEFAULT 'Tersedia', -- jika kamar tsb tidak diisi, maka secara otomatis memberi
  status "tersedia"
  id_tipe     VARCHAR(5) NOT NULL,
  CONSTRAINT pk_kamar    PRIMARY KEY (id_kamar),
  CONSTRAINT fk_kamar_tipe FOREIGN KEY (id_tipe) REFERENCES
  tipe_kamar(id_tipe)
);
-- Membuat indeks untuk mempercepat pencarian berdasarkan status kamar
CREATE INDEX idx_kamar_status ON kamar(status_kamar);
CREATE INDEX idx_kamar_tipe  ON kamar(id_tipe);
```

Perintah CREATE TABLE kamar digunakan untuk membuat tabel kamar yang berfungsi sebagai tempat penyimpanan data kamar hotel. Tabel ini menjadi salah satu tabel utama dalam sistem reservasi karena digunakan untuk mencatat informasi mengenai kondisi dan ketersediaan kamar.

Atribut id_kamar didefinisikan menggunakan tipe data VARCHAR(5) dan diberikan constraint NOT NULL. Atribut ini digunakan sebagai kode unik yang membedakan setiap kamar yang tersedia pada hotel.

Atribut nomor_kamar menggunakan tipe data VARCHAR(10) dengan constraint NOT NULL. Atribut ini digunakan untuk menyimpan nomor kamar yang digunakan oleh pihak hotel sebagai identitas fisik kamar.

Atribut lantai menggunakan tipe data TINYINT dan diberi constraint CHECK (lantai > 0). Constraint tersebut diterapkan untuk memastikan bahwa nomor lantai yang dimasukkan memiliki nilai lebih besar dari nol sehingga data yang tersimpan sesuai dengan kondisi sebenarnya.

Atribut status_kamar menggunakan tipe data ENUM yang terdiri atas nilai Tersedia, Terisi, Dipesan, dan Maintenance. Penggunaan tipe data ini bertujuan untuk membatasi pilihan status yang dapat digunakan sehingga menjaga konsistensi data. Selain itu, atribut ini diberikan nilai default Tersedia menggunakan query DEFAULT, sehingga apabila status kamar tidak diinputkan saat penambahan data, sistem secara otomatis akan menetapkan status kamar sebagai tersedia.

Atribut id_tipe menggunakan tipe data VARCHAR(5) dengan constraint NOT NULL. Atribut ini digunakan untuk menyimpan kode tipe kamar yang dimiliki oleh setiap kamar.

Constraint PRIMARY KEY diterapkan pada atribut id_kamar untuk menjamin bahwa setiap kamar memiliki identitas yang unik dan tidak dapat diduplikasi. Constraint FOREIGN KEY pada atribut id_tipe digunakan untuk membentuk relasi antara tabel kamar dan tabel tipe_kamar. Dengan adanya relasi ini, setiap data kamar harus memiliki tipe kamar yang telah terdaftar pada tabel tipe_kamar, sehingga integritas data dapat terjaga.

Selain pembuatan tabel, dilakukan pula pembuatan indeks untuk meningkatkan kinerja proses pencarian data. Perintah CREATE INDEX idx_kamar_status ON kamar(status_kamar), digunakan untuk membuat indeks pada atribut status_kamar. Indeks ini berfungsi mempercepat proses pencarian kamar berdasarkan status tertentu, seperti kamar yang tersedia, terisi, dipesan, atau sedang dalam perawatan.

Perintah CREATE INDEX idx_kamar_tipe ON kamar(id_tipe), digunakan untuk membuat indeks pada atribut id_tipe. Indeks ini bertujuan mempercepat proses pencarian dan pengelompokan data kamar berdasarkan tipe kamar yang dimiliki.

4.1.3 Implementasi Tabel Tamu

Tabel tamu digunakan untuk menyimpan informasi mengenai pelanggan yang melakukan reservasi kamar pada hotel. Data yang disimpan meliputi identitas tamu, alamat, nomor telepon, alamat surat elektronik (*email*), jenis identitas, dan nomor identitas. Tabel ini berperan sebagai sumber data utama yang digunakan dalam proses reservasi dan pelayanan tamu selama menginap.

Tabel 20. *Script Pembuatan Tabel Tamu*

```

-- 3. TABEL TAMU
CREATE TABLE tamu (
    id_tamu VARCHAR(7) NOT NULL,
    nama_tamu VARCHAR(100) NOT NULL,
    alamat VARCHAR(200) NULL,
    no_hp VARCHAR(20) NOT NULL,
    email VARCHAR(100) NULL,
    jenis_identitas ENUM('KTP','SIM','Passport') NOT NULL DEFAULT 'KTP',
    no_identitas VARCHAR(20) NOT NULL,
    CONSTRAINT pk_tamu PRIMARY KEY (id_tamu),
    CONSTRAINT uq_tamu_identitas UNIQUE (jenis_identitas, no_identitas),
    CONSTRAINT uq_tamu_email UNIQUE (email)
);

CREATE INDEX idx_tamu_nama ON tamu(nama_tamu);

```

Perintah CREATE TABLE tamu digunakan untuk membuat tabel tamu yang berfungsi sebagai tempat penyimpanan seluruh data pelanggan yang menggunakan layanan hotel.

Atribut id_tamu menggunakan tipe data VARCHAR(7) dan diberikan constraint NOT NULL. Atribut ini berfungsi sebagai kode unik yang digunakan untuk mengidentifikasi setiap tamu yang terdaftar dalam sistem.

Atribut nama_tamu menggunakan tipe data VARCHAR(100) dengan constraint NOT NULL. Atribut ini digunakan untuk menyimpan nama lengkap tamu dan wajib diisi pada saat proses pendaftaran atau reservasi.

Atribut alamat menggunakan tipe data VARCHAR(200) dan diperbolehkan bernilai NULL. Atribut ini digunakan untuk menyimpan informasi alamat tempat tinggal tamu.

Atribut no_hp menggunakan tipe data VARCHAR(20) dengan constraint NOT NULL. Atribut ini digunakan untuk menyimpan nomor telepon yang dapat dihubungi oleh pihak hotel apabila diperlukan.

Atribut email menggunakan tipe data VARCHAR(100) dan diperbolehkan bernilai NULL. Atribut ini digunakan untuk menyimpan alamat surat elektronik milik tamu yang dapat dimanfaatkan sebagai sarana komunikasi maupun pengiriman informasi reservasi.

Atribut jenis_identitas menggunakan tipe data ENUM yang terdiri atas pilihan KTP, SIM, dan Passport. Penggunaan tipe data ini bertujuan untuk membatasi jenis identitas yang dapat digunakan oleh tamu sehingga data yang tersimpan menjadi lebih konsisten. Selain itu, atribut ini diberikan nilai default KTP menggunakan query DEFAULT, sehingga apabila jenis identitas tidak diinputkan, sistem secara otomatis akan menetapkan nilai KTP.

Atribut no_identitas menggunakan tipe data VARCHAR(20) dengan constraint NOT NULL. Atribut ini digunakan untuk menyimpan nomor identitas resmi milik tamu sesuai dengan jenis identitas yang dipilih.

Constraint PRIMARY KEY diterapkan pada atribut id_tamu untuk memastikan bahwa setiap tamu memiliki kode identitas yang unik dan tidak dapat diduplikasi. Constraint UNIQUE (jenis_identitas, no_identitas) digunakan untuk mencegah terjadinya duplikasi data identitas. Dengan adanya constraint ini, satu nomor identitas hanya dapat didaftarkan satu kali sesuai dengan jenis identitas yang digunakan. Constraint UNIQUE (email) diterapkan pada atribut email untuk memastikan bahwa satu alamat surat elektronik tidak digunakan oleh lebih dari satu tamu dalam sistem.

Perintah CREATE INDEX idx_tamu_nama ON tamu(nama_tamu), digunakan untuk membuat indeks pada atribut nama_tamu. Indeks ini berfungsi untuk meningkatkan kecepatan proses pencarian data tamu berdasarkan nama, terutama ketika jumlah data yang tersimpan dalam database semakin besar.

4.1.4 Implementasi Tabel Pegawai

Tabel pegawai digunakan untuk menyimpan informasi mengenai seluruh pegawai yang bekerja pada hotel dan terlibat dalam operasional sistem reservasi. Data pegawai yang disimpan meliputi kode pegawai, nama pegawai, jabatan, dan nomor telepon. Informasi pada tabel ini digunakan sebagai referensi dalam proses pelayanan tamu, khususnya pada kegiatan check-in dan check-out.

Tabel 21. *Script Pembuatan Tabel Pegawai*

```
-- 4. TABEL PEGAWAI  
CREATE TABLE pegawai (
```

```
id_pegawai VARCHAR(5) NOT NULL,  
nama_pegawai VARCHAR(100) NOT NULL,  
jabatan VARCHAR(50) NOT NULL,  
no_hp VARCHAR(20) NOT NULL,  
CONSTRAINT pk_pegawai PRIMARY KEY (id_pegawai)  
);
```

Perintah CREATE TABLE pegawai digunakan untuk membuat tabel pegawai yang berfungsi sebagai tempat penyimpanan data karyawan hotel yang bertugas dalam menjalankan berbagai aktivitas operasional.

Atribut id_pegawai menggunakan tipe data VARCHAR(5) dan diberikan constraint NOT NULL. Atribut ini berfungsi sebagai kode unik yang digunakan untuk mengidentifikasi setiap pegawai yang terdaftar dalam sistem.

Atribut nama_pegawai menggunakan tipe data VARCHAR(100) dengan constraint NOT NULL. Atribut ini digunakan untuk menyimpan nama lengkap pegawai dan wajib diisi agar identitas pegawai dapat dikenali dengan jelas.

Atribut jabatan menggunakan tipe data VARCHAR(50) dengan constraint NOT NULL. Atribut ini digunakan untuk menyimpan informasi mengenai posisi atau tugas pegawai dalam organisasi hotel, seperti resepsionis, *supervisor*, manajer, atau petugas *front office*.

Atribut no_hp menggunakan tipe data VARCHAR(20) dengan constraint NOT NULL. Atribut ini digunakan untuk menyimpan nomor telepon pegawai yang dapat digunakan sebagai sarana komunikasi apabila diperlukan.

Constraint PRIMARY KEY diterapkan pada atribut id_pegawai melalui deklarasi CONSTRAINT pk_pegawai PRIMARY KEY (id_pegawai). Constraint ini berfungsi untuk menjamin bahwa setiap pegawai memiliki kode identitas yang unik, tidak bernilai kosong, dan tidak dapat diduplikasi oleh data lainnya.

Berdasarkan implementasi yang telah dilakukan, tabel pegawai berhasil dibuat sebagai media penyimpanan data karyawan hotel. Tabel ini berperan penting dalam mendukung pengelolaan operasional hotel karena digunakan sebagai referensi pada berbagai proses bisnis, terutama pada pencatatan aktivitas check-in dan check-out tamu.

4.1.5 Implementasi Tabel Fasilitas

Tabel fasilitas digunakan untuk menyimpan informasi mengenai berbagai fasilitas yang disediakan oleh hotel untuk menunjang kenyamanan dan kebutuhan tamu selama

mengingat. Data yang disimpan meliputi kode fasilitas, nama fasilitas, kategori fasilitas, deskripsi fasilitas, serta biaya penggunaan fasilitas. Informasi pada tabel ini dapat digunakan sebagai referensi dalam pengelolaan layanan tambahan yang tersedia di hotel.

Tabel 22. *Script Pembuatan Tabel Fasilitas*

```
-- 5. TABEL FASILITAS
CREATE TABLE fasilitas (
  id_fasilitas    VARCHAR(5) NOT NULL,
  nama_fasilitas  VARCHAR(100) NOT NULL,
  kategori        VARCHAR(50) NOT NULL,
  deskripsi_fasilitas VARCHAR(200) NULL,
  biaya           DECIMAL(10,2) NOT NULL DEFAULT 0 CHECK (biaya >= 0),
  CONSTRAINT pk_fasilitas PRIMARY KEY (id_fasilitas),
  CONSTRAINT uq_nama_fasilitas UNIQUE (nama_fasilitas)
);
```

Perintah CREATE TABLE fasilitas digunakan untuk membuat tabel fasilitas yang berfungsi sebagai tempat penyimpanan data fasilitas yang dimiliki oleh hotel. Atribut id_fasilitas menggunakan tipe data VARCHAR(5) dan diberikan constraint NOT NULL. Atribut ini berfungsi sebagai kode unik yang digunakan untuk mengidentifikasi setiap fasilitas yang tersedia di hotel.

Atribut nama_fasilitas menggunakan tipe data VARCHAR(100) dengan constraint NOT NULL. Atribut ini digunakan untuk menyimpan nama fasilitas yang disediakan, seperti WiFi, kolam renang, pusat kebugaran, layanan laundry, dan fasilitas lainnya.

Atribut kategori menggunakan tipe data VARCHAR(50) dengan constraint NOT NULL. Atribut ini digunakan untuk mengelompokkan fasilitas berdasarkan jenis layanan yang diberikan, sehingga memudahkan proses pengelolaan dan pencarian data fasilitas.

Atribut deskripsi_fasilitas menggunakan tipe data VARCHAR(200) dan diperbolehkan bernilai NULL. Atribut ini digunakan untuk menyimpan informasi tambahan mengenai fasilitas yang tersedia, seperti deskripsi layanan atau ketentuan penggunaan fasilitas tersebut.

Atribut biaya menggunakan tipe data DECIMAL(10,2) dengan constraint NOT NULL. Atribut ini digunakan untuk menyimpan biaya penggunaan fasilitas dengan tingkat ketelitian hingga dua angka di belakang koma. Selain itu, atribut ini diberikan nilai default

0 melalui query DEFAULT, sehingga apabila biaya tidak diinputkan, sistem secara otomatis akan menetapkan nilai nol yang menunjukkan bahwa fasilitas tersebut dapat digunakan tanpa biaya tambahan.

Constraint CHECK (biaya $\geq$ 0) diterapkan untuk memastikan bahwa nilai biaya yang dimasukkan tidak bernilai negatif. Dengan demikian, validitas data dapat terjaga dan kesalahan input dapat diminimalkan. Constraint PRIMARY KEY diterapkan pada atribut id_fasilitas melalui deklarasi CONSTRAINT pk_fasilitas PRIMARY KEY (id_fasilitas). Constraint ini berfungsi untuk menjamin bahwa setiap fasilitas memiliki kode identitas yang unik dan tidak dapat diduplikasi. Constraint UNIQUE diterapkan pada atribut nama_fasilitas melalui deklarasi CONSTRAINT uq_nama_fasilitas UNIQUE (nama_fasilitas). Constraint ini bertujuan untuk mencegah adanya data fasilitas dengan nama yang sama sehingga konsistensi data dapat terjaga.

Berdasarkan implementasi yang telah dilakukan, tabel fasilitas berhasil dibuat sebagai media penyimpanan data fasilitas hotel. Penerapan *primary key*, *unique constraint*, *default value*, dan *check constraint* pada tabel ini membantu menjaga integritas data serta memastikan bahwa informasi fasilitas yang tersimpan sesuai dengan kebutuhan sistem reservasi hotel.

4.1.6 Implementasi Tabel Reservasi

Tabel reservasi merupakan tabel utama dalam sistem reservasi hotel yang digunakan untuk menyimpan data pemesanan kamar oleh tamu. Tabel ini mencatat informasi mengenai tanggal pemesanan, jadwal menginap, jumlah tamu, status reservasi, serta hubungan antara tamu dan kamar yang dipesan. Keberadaan tabel ini sangat penting karena menjadi pusat transaksi yang menghubungkan berbagai proses dalam sistem, seperti pembayaran, check-in, check-out, dan ulasan pelanggan.

Tabel 23. Script Pembuatan Tabel Reservasi

```
-- 6. TABEL RESERVASI
CREATE TABLE reservasi (
  id_reservasi  VARCHAR(5) NOT NULL,
  tanggal_pesan DATE      NOT NULL,
  check_in     DATE       NOT NULL,
  check_out    DATE       NOT NULL,
  jumlah_tamu  TINYINT    NOT NULL CHECK (jumlah_tamu > 0),
```

```

status_reservasi ENUM('Dipesan','Check-In','Selesai','Batal') NOT NULL
DEFAULT 'Dipesan',
id_tamu      VARCHAR(7) NOT NULL,
id_kamar     VARCHAR(5) NOT NULL,
CONSTRAINT pk_reservasi      PRIMARY KEY (id_reservasi),
CONSTRAINT fk_reservasi_tamu  FOREIGN KEY (id_tamu) REFERENCES
tamu(id_tamu),
CONSTRAINT fk_reservasi_kamar FOREIGN KEY (id_kamar) REFERENCES
kamar(id_kamar),
CONSTRAINT chk_tanggal_reservasi CHECK (check_out > check_in)
);

CREATE INDEX idx_reservasi_tamu  ON reservasi(id_tamu);
CREATE INDEX idx_reservasi_kamar ON reservasi(id_kamar);
CREATE INDEX idx_reservasi_status ON reservasi(status_reservasi);
CREATE INDEX idx_reservasi_checkin ON reservasi(check_in);

```

Perintah CREATE TABLE reservasi digunakan untuk membuat tabel reservasi yang berfungsi sebagai tempat penyimpanan seluruh data pemesanan kamar yang dilakukan oleh tamu hotel.

Atribut id_reservasi menggunakan tipe data VARCHAR(5) dan diberikan constraint NOT NULL. Atribut ini berfungsi sebagai kode unik yang digunakan untuk mengidentifikasi setiap transaksi reservasi yang tercatat dalam sistem.

Atribut tanggal_pesan menggunakan tipe data DATE dengan constraint NOT NULL. Atribut ini digunakan untuk menyimpan tanggal ketika reservasi dilakukan oleh tamu. Atribut check_in menggunakan tipe data DATE dengan constraint NOT NULL. Atribut ini digunakan untuk menyimpan tanggal kedatangan tamu sesuai dengan jadwal reservasi yang telah ditentukan.

Atribut check_out menggunakan tipe data DATE dengan constraint NOT NULL. Atribut ini digunakan untuk menyimpan tanggal keberangkatan tamu dari hotel setelah masa menginap berakhir.

Atribut jumlah_tamu menggunakan tipe data TINYINT dengan constraint CHECK (jumlah_tamu > 0). Constraint tersebut diterapkan untuk memastikan bahwa jumlah tamu

yang melakukan reservasi harus lebih besar dari nol sehingga data yang tersimpan tetap valid.

Atribut `status_reservasi` menggunakan tipe data `ENUM` yang terdiri atas nilai `Dipesan`, `Check-In`, `Selesai`, dan `Batal`. Penggunaan tipe data ini bertujuan untuk membatasi status reservasi yang dapat digunakan sehingga konsistensi data dapat terjaga. Selain itu, atribut ini diberikan nilai default `Dipesan` melalui query `DEFAULT`, sehingga setiap reservasi baru secara otomatis akan memiliki status awal sebagai reservasi yang telah dipesan.

Atribut `id_tamu` menggunakan tipe data `VARCHAR(7)` dengan constraint `NOT NULL`. Atribut ini digunakan untuk menyimpan identitas tamu yang melakukan reservasi. Atribut `id_kamar` menggunakan tipe data `VARCHAR(5)` dengan constraint `NOT NULL`. Atribut ini digunakan untuk menyimpan identitas kamar yang dipesan oleh tamu.

Constraint `PRIMARY KEY` diterapkan pada atribut `id_reservasi` untuk menjamin bahwa setiap transaksi reservasi memiliki identitas yang unik dan tidak dapat diduplikasi.

Constraint `FOREIGN KEY` pada atribut `id_tamu` digunakan untuk membentuk relasi antara tabel reservasi dan tabel tamu. Relasi ini memastikan bahwa setiap reservasi hanya dapat dibuat oleh tamu yang telah terdaftar dalam sistem.

Constraint `FOREIGN KEY` pada atribut `id_kamar` digunakan untuk membentuk relasi antara tabel reservasi dan tabel kamar. Dengan adanya relasi ini, setiap reservasi harus mengacu pada kamar yang telah tersedia dalam database.

Constraint `CHECK (check_out > check_in)` diterapkan untuk memastikan bahwa tanggal check-out harus lebih besar daripada tanggal check-in. Aturan ini digunakan untuk menjaga validitas data reservasi dan mencegah terjadinya kesalahan penginputan tanggal.

Selain pembuatan tabel, dilakukan pula pembuatan beberapa indeks untuk meningkatkan kinerja sistem dalam proses pencarian dan pengolahan data reservasi. Perintah `CREATE INDEX idx_reservasi_tamu ON reservasi(id_tamu)`, digunakan untuk mempercepat proses pencarian data reservasi berdasarkan identitas tamu. Perintah `CREATE INDEX idx_reservasi_kamar ON reservasi(id_kamar)`, digunakan untuk mempercepat pencarian data reservasi berdasarkan kamar yang dipesan. Perintah `CREATE INDEX idx_reservasi_status ON reservasi(status_reservasi)`, digunakan untuk mempercepat proses pencarian data berdasarkan status reservasi, seperti reservasi yang masih aktif, telah selesai, atau dibatalkan. Perintah `CREATE INDEX idx_reservasi_checkin ON reservasi(check_in)`, digunakan untuk mempercepat pencarian data reservasi berdasarkan tanggal check-in, terutama ketika sistem melakukan pengecekan jadwal kedatangan tamu.

Berdasarkan implementasi yang telah dilakukan, tabel reservasi berhasil dibuat beserta relasinya dengan tabel tamu dan tabel kamar. Penerapan *primary key*, *foreign key*, *check constraint*, *default value*, dan indeks pada tabel ini mendukung integritas data serta meningkatkan efisiensi pengelolaan transaksi reservasi dalam sistem hotel.

4.1.7 Implementasi Tabel Pembayaran

Tabel pembayaran digunakan untuk menyimpan informasi mengenai transaksi pembayaran yang dilakukan oleh tamu atas reservasi kamar yang telah dibuat. Tabel ini mencatat data pembayaran seperti tanggal pembayaran, jumlah pembayaran, metode pembayaran, status pembayaran, serta referensi transaksi. Selain itu, tabel pembayaran memiliki hubungan langsung dengan tabel reservasi sehingga setiap transaksi pembayaran dapat ditelusuri berdasarkan reservasi yang terkait.

Tabel 24. *Script Pembuatan Tabel Pembayaran*

```
-- 7. TABEL PEMBAYARAN
CREATE TABLE pembayaran (
  id_pembayaran VARCHAR(7) NOT NULL,
  tanggal_bayar DATE NOT NULL,
  jumlah_bayar DECIMAL(12,2) NOT NULL CHECK (jumlah_bayar > 0),
  metode_bayar ENUM('Transfer','QRIS','Cash','Kartu Kredit') NOT NULL,
  status_bayar ENUM('Lunas','Pending','Gagal','Refund') NOT NULL DEFAULT
'Pending',
  referensi VARCHAR(20) NULL,
  id_reservasi VARCHAR(5) NOT NULL,
  CONSTRAINT pk_pembayaran PRIMARY KEY (id_pembayaran),
  CONSTRAINT fk_pembayaran_reservasi FOREIGN KEY (id_reservasi)
REFERENCES reservasi(id_reservasi),
  CONSTRAINT uq_pembayaran_reservasi UNIQUE (id_reservasi)
);
CREATE INDEX idx_pembayaran_status ON pembayaran(status_bayar);
```

Perintah CREATE TABLE pembayaran digunakan untuk membuat tabel pembayaran yang berfungsi sebagai tempat penyimpanan seluruh data transaksi pembayaran yang dilakukan oleh tamu hotel.

Atribut `id_pembayaran` menggunakan tipe data `VARCHAR(7)` dan diberikan constraint `NOT NULL`. Atribut ini berfungsi sebagai kode unik yang digunakan untuk mengidentifikasi setiap transaksi pembayaran yang tercatat dalam sistem.

Atribut `tanggal_bayar` menggunakan tipe data `DATE` dengan constraint `NOT NULL`. Atribut ini digunakan untuk menyimpan tanggal dilakukannya pembayaran oleh tamu. Atribut `jumlah_bayar` menggunakan tipe data `DECIMAL(12,2)` dengan constraint `NOT NULL`. Atribut ini digunakan untuk menyimpan nominal pembayaran yang dilakukan oleh tamu dengan tingkat ketelitian hingga dua angka di belakang koma. Constraint `CHECK (jumlah_bayar > 0)` diterapkan untuk memastikan bahwa nilai pembayaran yang dimasukkan harus lebih besar dari nol sehingga data yang tersimpan tetap valid.

Atribut `metode_bayar` menggunakan tipe data `ENUM` yang terdiri atas pilihan Transfer, QRIS, Cash, dan Kartu Kredit. Penggunaan tipe data ini bertujuan untuk membatasi metode pembayaran yang dapat digunakan sehingga data yang tersimpan menjadi lebih konsisten dan terstandarisasi.

Atribut `status_bayar` menggunakan tipe data `ENUM` yang terdiri atas nilai Lunas, Pending, Gagal, dan Refund. Atribut ini digunakan untuk menunjukkan status dari suatu transaksi pembayaran. Selain itu, atribut ini diberikan nilai default Pending melalui query `DEFAULT`, sehingga setiap transaksi pembayaran baru secara otomatis akan memiliki status awal sebagai pembayaran yang masih menunggu penyelesaian.

Atribut `referensi` menggunakan tipe data `VARCHAR(20)` dan diperbolehkan bernilai `NULL`. Atribut ini digunakan untuk menyimpan nomor referensi transaksi, seperti nomor transfer atau kode pembayaran elektronik yang diperoleh dari sistem pembayaran.

Atribut `id_reservasi` menggunakan tipe data `VARCHAR(5)` dengan constraint `NOT NULL`. Atribut ini digunakan untuk menyimpan identitas reservasi yang terkait dengan transaksi pembayaran.

Constraint `PRIMARY KEY` diterapkan pada atribut `id_pembayaran` untuk menjamin bahwa setiap transaksi pembayaran memiliki identitas yang unik dan tidak dapat diduplikasi. Constraint `FOREIGN KEY` pada atribut `id_reservasi` digunakan untuk membentuk relasi antara tabel pembayaran dan tabel reservasi. Relasi ini memastikan bahwa setiap pembayaran harus terkait dengan data reservasi yang telah terdaftar dalam sistem. Constraint `UNIQUE (id_reservasi)` diterapkan untuk memastikan bahwa satu reservasi hanya dapat memiliki satu data pembayaran. Dengan demikian, tidak akan terjadi duplikasi transaksi pembayaran untuk reservasi yang sama.

Selain pembuatan tabel, dilakukan pula pembuatan indeks pada atribut `status_bayar` melalui perintah `CREATE INDEX idx_pembayaran_status ON pembayaran(status_bayar)`. Indeks tersebut digunakan untuk mempercepat proses pencarian dan pengelompokan data pembayaran berdasarkan status transaksi, seperti pembayaran yang telah lunas, masih menunggu konfirmasi, gagal diproses, atau telah dilakukan pengembalian dana (*refund*).

Berdasarkan implementasi yang telah dilakukan, tabel pembayaran berhasil dibuat beserta relasinya dengan tabel reservasi. Penerapan *primary key*, *foreign key*, *unique constraint*, *check constraint*, *default value*, dan indeks pada tabel ini mendukung integritas data serta meningkatkan efisiensi pengelolaan transaksi pembayaran dalam sistem reservasi hotel.

4.1.8 Implementasi Tabel Check_in

Tabel `check_in` digunakan untuk menyimpan informasi mengenai proses kedatangan tamu yang telah melakukan reservasi. Tabel ini mencatat waktu check-in, data reservasi yang terkait, serta pegawai yang melayani proses check-in. Dengan adanya tabel ini, setiap aktivitas kedatangan tamu dapat terdokumentasi dengan baik dan dapat digunakan sebagai bagian dari proses operasional hotel.

Tabel 25. Script Pembuatan Tabel Check_in

```
CREATE TABLE check_in (  
  id_checkin VARCHAR(7) NOT NULL,  
  waktu_checkin DATETIME NOT NULL,  
  id_reservasi VARCHAR(5) NOT NULL,  
  id_pegawai VARCHAR(5) NOT NULL,  
  CONSTRAINT pk_checkin PRIMARY KEY (id_checkin),  
  CONSTRAINT fk_checkin_reservasi FOREIGN KEY (id_reservasi)  
  REFERENCES reservasi(id_reservasi),  
  CONSTRAINT fk_checkin_pegawai FOREIGN KEY (id_pegawai)  
  REFERENCES pegawai(id_pegawai),  
  CONSTRAINT uq_checkin_reservasi UNIQUE (id_reservasi)  
);
```

Perintah `CREATE TABLE check_in` digunakan untuk membuat tabel `check_in` yang berfungsi sebagai tempat penyimpanan data proses check-in tamu pada hotel. Atribut `id_checkin` menggunakan tipe data `VARCHAR(7)` dan diberikan constraint `NOT NULL`.

Atribut ini berfungsi sebagai kode unik yang digunakan untuk mengidentifikasi setiap transaksi check-in yang terjadi dalam sistem.

Atribut waktu_checkin menggunakan tipe data DATETIME dengan constraint NOT NULL. Atribut ini digunakan untuk menyimpan informasi tanggal dan waktu secara lengkap saat tamu melakukan check-in di hotel. Penggunaan tipe data DATETIME memungkinkan sistem mencatat waktu kedatangan tamu secara lebih akurat.

Atribut id_reservasi menggunakan tipe data VARCHAR(5) dengan constraint NOT NULL. Atribut ini digunakan untuk menyimpan identitas reservasi yang terkait dengan proses check-in.

Atribut id_pegawai menggunakan tipe data VARCHAR(5) dengan constraint NOT NULL. Atribut ini digunakan untuk menyimpan identitas pegawai yang bertugas melayani dan memproses check-in tamu.

Constraint PRIMARY KEY diterapkan pada atribut id_checkin melalui deklarasi CONSTRAINT pk_checkin PRIMARY KEY (id_checkin). Constraint ini berfungsi untuk memastikan bahwa setiap data check-in memiliki identitas yang unik dan tidak dapat diduplikasi.

Constraint FOREIGN KEY pada atribut id_reservasi digunakan untuk membentuk relasi antara tabel check_in dan tabel reservasi. Relasi ini memastikan bahwa setiap proses check-in hanya dapat dilakukan untuk reservasi yang telah terdaftar dalam sistem.

Constraint FOREIGN KEY pada atribut id_pegawai digunakan untuk membentuk relasi antara tabel check_in dan tabel pegawai. Relasi ini memungkinkan sistem mengetahui pegawai yang bertanggung jawab dalam melayani proses check-in tamu.

Constraint UNIQUE (id_reservasi) diterapkan untuk memastikan bahwa satu data reservasi hanya dapat memiliki satu data check-in. Dengan demikian, sistem dapat mencegah terjadinya pencatatan check-in lebih dari satu kali untuk reservasi yang sama.

Berdasarkan implementasi yang telah dilakukan, tabel check_in berhasil dibuat dan terhubung dengan tabel reservasi serta tabel pegawai melalui *foreign key*. Penerapan *primary key*, *foreign key*, dan *unique constraint* pada tabel ini mendukung integritas data serta memastikan bahwa setiap proses check-in tercatat secara akurat dan tidak mengalami duplikasi dalam sistem reservasi hotel.

4.1.9 Implementasi Tabel Check_out

Tabel check_out digunakan untuk menyimpan informasi mengenai proses keberangkatan tamu dari hotel setelah masa menginap berakhir. Tabel ini mencatat waktu

check-out, data reservasi yang terkait, serta pegawai yang bertugas melayani proses check-out. Informasi yang tersimpan pada tabel ini dapat digunakan sebagai bukti bahwa tamu telah menyelesaikan masa menginapnya dan telah keluar dari hotel sesuai dengan prosedur yang berlaku.

Tabel 26. *Script Pembuatan Tabel Check_out*

```
-- 9. TABEL CHECK_OUT
CREATE TABLE check_out (
  id_checkout VARCHAR(7) NOT NULL,
  waktu_checkout DATETIME NOT NULL,
  id_reservasi VARCHAR(5) NOT NULL,
  id_pegawai VARCHAR(5) NOT NULL,
  CONSTRAINT pk_checkout PRIMARY KEY (id_checkout),
  CONSTRAINT fk_checkout_reservasi FOREIGN KEY (id_reservasi)
REFERENCES reservasi(id_reservasi),
  CONSTRAINT fk_checkout_pegawai FOREIGN KEY (id_pegawai)
REFERENCES pegawai(id_pegawai),
  CONSTRAINT uq_checkout_reservasi UNIQUE (id_reservasi)
);
```

Perintah CREATE TABLE check_out digunakan untuk membuat tabel check_out yang berfungsi sebagai tempat penyimpanan data proses check-out tamu dalam sistem reservasi hotel.

Atribut id_checkout menggunakan tipe data VARCHAR(7) dan diberikan constraint NOT NULL. Atribut ini berfungsi sebagai kode unik yang digunakan untuk mengidentifikasi setiap transaksi check-out yang tercatat dalam sistem.

Atribut waktu_checkout menggunakan tipe data DATETIME dengan constraint NOT NULL. Atribut ini digunakan untuk menyimpan informasi tanggal dan waktu secara lengkap saat tamu melakukan check-out dari hotel. Penggunaan tipe data DATETIME memungkinkan sistem mencatat waktu keberangkatan tamu secara akurat.

Atribut id_reservasi menggunakan tipe data VARCHAR(5) dengan constraint NOT NULL. Atribut ini digunakan untuk menyimpan identitas reservasi yang berkaitan dengan proses check-out.

Atribut `id_pegawai` menggunakan tipe data `VARCHAR(5)` dengan constraint `NOT NULL`. Atribut ini digunakan untuk menyimpan identitas pegawai yang bertanggung jawab dalam melayani dan memproses check-out tamu.

Constraint `PRIMARY KEY` diterapkan pada atribut `id_checkout` melalui deklarasi `CONSTRAINT pk_checkout PRIMARY KEY (id_checkout)`. Constraint ini berfungsi untuk memastikan bahwa setiap data check-out memiliki identitas yang unik dan tidak dapat diduplikasi.

Constraint `FOREIGN KEY` pada atribut `id_reservasi` digunakan untuk membentuk relasi antara tabel `check_out` dan tabel `reservasi`. Relasi ini memastikan bahwa setiap proses check-out hanya dapat dilakukan untuk reservasi yang telah terdaftar dalam sistem.

Constraint `FOREIGN KEY` pada atribut `id_pegawai` digunakan untuk membentuk relasi antara tabel `check_out` dan tabel `pegawai`. Relasi ini memungkinkan sistem mencatat pegawai yang bertanggung jawab dalam proses check-out tamu.

Constraint `UNIQUE (id_reservasi)` diterapkan untuk memastikan bahwa satu data reservasi hanya dapat memiliki satu data check-out. Dengan adanya constraint ini, sistem dapat mencegah terjadinya pencatatan check-out lebih dari satu kali untuk reservasi yang sama.

Berdasarkan implementasi yang telah dilakukan, tabel `check_out` berhasil dibuat dan terhubung dengan tabel `reservasi` serta tabel `pegawai` melalui *foreign key*. Penerapan *primary key*, *foreign key*, dan *unique constraint* pada tabel ini mendukung integritas data serta memastikan bahwa setiap proses check-out tercatat dengan baik, akurat, dan tidak mengalami duplikasi dalam sistem reservasi hotel.

4.1.10 Implementasi Tabel Ulasan

Tabel ulasan digunakan untuk menyimpan penilaian dan tanggapan yang diberikan oleh tamu setelah menyelesaikan masa menginap di hotel. Data yang tersimpan pada tabel ini meliputi tanggal pemberian ulasan, nilai rating, komentar, serta informasi reservasi yang terkait. Tabel ulasan berfungsi sebagai sarana evaluasi kualitas pelayanan dan fasilitas hotel berdasarkan pengalaman yang dirasakan oleh tamu.

Tabel 27. Script Pembuatan Tabel Ulasan

```
-- 10. TABEL ULASAN
CREATE TABLE ulasan (
  id_ulasan  VARCHAR(5) NOT NULL,
```

```

tanggal_ulasan DATE      NOT NULL,
rating      TINYINT      NOT NULL CHECK (rating BETWEEN 1 AND 5),
komentar    VARCHAR(500) NULL,
id_reservasi VARCHAR(5) NOT NULL,
CONSTRAINT pk_ulasan      PRIMARY KEY (id_ulasan),
CONSTRAINT fk_ulasan_reservasi FOREIGN KEY (id_reservasi) REFERENCES
reservasi(id_reservasi),
CONSTRAINT uq_ulasan_reservasi UNIQUE (id_reservasi)
);
CREATE INDEX idx_ulasan_rating ON ulasan(rating);

```

Perintah CREATE TABLE ulasan digunakan untuk membuat tabel ulasan yang berfungsi sebagai tempat penyimpanan data penilaian dan komentar yang diberikan oleh tamu setelah menggunakan layanan hotel.

Atribut id_ulasan menggunakan tipe data VARCHAR(5) dan diberikan constraint NOT NULL. Atribut ini berfungsi sebagai kode unik yang digunakan untuk mengidentifikasi setiap data ulasan yang tersimpan dalam sistem.

Atribut tanggal_ulasan menggunakan tipe data DATE dengan constraint NOT NULL. Atribut ini digunakan untuk menyimpan tanggal ketika tamu memberikan ulasan terhadap layanan hotel yang telah diterimanya.

Atribut rating menggunakan tipe data TINYINT dengan constraint NOT NULL. Atribut ini digunakan untuk menyimpan nilai penilaian yang diberikan oleh tamu terhadap kualitas pelayanan hotel. Constraint CHECK (rating BETWEEN 1 AND 5) diterapkan untuk memastikan bahwa nilai rating yang dimasukkan berada dalam rentang 1 sampai 5, sehingga sesuai dengan sistem penilaian yang digunakan.

Atribut komentar menggunakan tipe data VARCHAR(500) dan diperbolehkan bernilai NULL. Atribut ini digunakan untuk menyimpan tanggapan, kritik, maupun saran yang diberikan oleh tamu terkait pengalaman menginapnya di hotel.

Atribut id_reservasi menggunakan tipe data VARCHAR(5) dengan constraint NOT NULL. Atribut ini digunakan untuk menyimpan identitas reservasi yang menjadi dasar pemberian ulasan oleh tamu.

Constraint PRIMARY KEY diterapkan pada atribut `id_ulasan` melalui deklarasi `CONSTRAINT pk_ulasan PRIMARY KEY (id_ulasan)`. Constraint ini berfungsi untuk menjamin bahwa setiap ulasan memiliki identitas yang unik dan tidak dapat diduplikasi.

Constraint FOREIGN KEY pada atribut `id_reservasi` digunakan untuk membentuk relasi antara tabel ulasan dan tabel reservasi. Relasi ini memastikan bahwa setiap ulasan yang diberikan harus berkaitan dengan data reservasi yang telah terdaftar dalam sistem.

Constraint UNIQUE (`id_reservasi`) diterapkan untuk memastikan bahwa satu reservasi hanya dapat memiliki satu ulasan. Dengan demikian, sistem dapat mencegah terjadinya pemberian ulasan ganda terhadap reservasi yang sama.

Selain pembuatan tabel, dilakukan pula pembuatan indeks pada atribut `rating` melalui perintah `CREATE INDEX idx_ulasan_rating ON ulasan(rating)`. Indeks tersebut digunakan untuk meningkatkan kecepatan proses pencarian dan analisis data ulasan berdasarkan nilai `rating`. Dengan adanya indeks ini, sistem dapat lebih efisien dalam menampilkan ulasan dengan kategori penilaian tertentu, seperti ulasan dengan `rating` tinggi maupun `rating` rendah.

Berdasarkan implementasi yang telah dilakukan, tabel ulasan berhasil dibuat dan terhubung dengan tabel reservasi melalui *foreign key*. Penerapan *primary key*, *foreign key*, *unique constraint*, *check constraint*, dan indeks pada tabel ini mendukung integritas data serta meningkatkan efisiensi dalam pengelolaan dan analisis umpan balik pelanggan. Data yang tersimpan pada tabel ulasan dapat digunakan sebagai dasar evaluasi dan peningkatan kualitas layanan hotel di masa mendatang.

4.1.11 Implementasi Tabel Audit Log

Tabel `audit_log` digunakan untuk mencatat seluruh riwayat perubahan data yang terjadi pada tabel-tabel lain dalam sistem melalui mekanisme *trigger*. Tabel ini berfungsi sebagai media pemantauan aktivitas (*audit trail*) sehingga setiap operasi penambahan, perubahan, maupun penghapusan data dapat terdokumentasi dengan baik. Informasi yang tersimpan pada tabel ini dapat digunakan untuk keperluan pengawasan, pelacakan kesalahan, serta menjaga keamanan dan integritas data dalam sistem reservasi hotel.

Tabel 28. *Script* Pembuatan Tabel Audit Log

```
-- 11. TABEL AUDIT_LOG (untuk TRIGGER)
-- Untuk mencatat riwayat perubahan data yang terjadi pada tabel lain melalui trigger
CREATE TABLE audit_log (
```

```
id_log    INT          NOT NULL AUTO_INCREMENT,  
nama_tabel VARCHAR(50) NOT NULL,  
operasi   ENUM('INSERT','UPDATE','DELETE') NOT NULL,  
id_record VARCHAR(20) NOT NULL,  
keterangan VARCHAR(500) NULL,  
waktu_log DATETIME    NOT NULL DEFAULT CURRENT_TIMESTAMP,  
CONSTRAINT pk_audit_log PRIMARY KEY (id_log)  
);
```

Perintah CREATE TABLE audit_log digunakan untuk membuat tabel audit_log yang berfungsi sebagai tempat penyimpanan riwayat aktivitas perubahan data yang terjadi pada sistem.

Atribut id_log menggunakan tipe data INT dengan constraint NOT NULL dan AUTO_INCREMENT. Atribut ini berfungsi sebagai identitas unik untuk setiap catatan aktivitas yang tersimpan pada tabel audit log. Penggunaan AUTO_INCREMENT memungkinkan sistem menghasilkan nomor log secara otomatis setiap kali data baru ditambahkan sehingga tidak diperlukan pengisian nilai secara manual.

Atribut nama_tabel menggunakan tipe data VARCHAR(50) dengan constraint NOT NULL. Atribut ini digunakan untuk menyimpan nama tabel yang mengalami perubahan data, sehingga pengguna dapat mengetahui sumber aktivitas yang dicatat.

Atribut operasi menggunakan tipe data ENUM yang terdiri atas nilai INSERT, UPDATE, dan DELETE. Atribut ini digunakan untuk menunjukkan jenis operasi yang menyebabkan terbentuknya catatan log. Dengan penggunaan tipe data ENUM, nilai yang dapat disimpan menjadi lebih terkontrol dan konsisten.

Atribut id_record menggunakan tipe data VARCHAR(20) dengan constraint NOT NULL. Atribut ini digunakan untuk menyimpan identitas data yang mengalami perubahan sehingga proses pelacakan terhadap data yang dimodifikasi dapat dilakukan dengan lebih mudah.

Atribut keterangan menggunakan tipe data VARCHAR(500) dan diperbolehkan bernilai NULL. Atribut ini digunakan untuk menyimpan informasi tambahan mengenai aktivitas yang terjadi, seperti detail perubahan data atau deskripsi singkat mengenai operasi yang dilakukan.

Atribut waktu_log menggunakan tipe data DATETIME dengan constraint NOT NULL. Atribut ini berfungsi untuk mencatat waktu terjadinya aktivitas yang direkam dalam tabel audit log.

query DEFAULT CURRENT_TIMESTAMP diterapkan pada atribut waktu_log sehingga sistem secara otomatis akan mengisi tanggal dan waktu saat catatan log dibuat. Dengan demikian, setiap aktivitas yang tercatat memiliki informasi waktu yang akurat tanpa memerlukan input manual dari pengguna.

Constraint PRIMARY KEY diterapkan pada atribut id_log melalui deklarasi CONSTRAINT pk_audit_log PRIMARY KEY (id_log). Constraint ini berfungsi untuk menjamin bahwa setiap catatan log memiliki identitas yang unik dan tidak dapat diduplikasi.

Tabel audit_log tidak digunakan untuk menyimpan data operasional utama seperti reservasi atau pembayaran, melainkan digunakan sebagai tabel pendukung yang bekerja bersama *trigger*. Setiap kali terjadi operasi INSERT, UPDATE, atau DELETE pada tabel tertentu, *trigger* akan secara otomatis menambahkan catatan baru ke dalam tabel audit log.

Dengan adanya tabel ini, administrator sistem dapat melakukan pemantauan terhadap aktivitas yang terjadi dalam database, mengetahui riwayat perubahan data, serta melakukan investigasi apabila ditemukan kesalahan atau perubahan data yang tidak sesuai.

Berdasarkan implementasi yang telah dilakukan, tabel audit_log berhasil dibuat sebagai media pencatatan aktivitas perubahan data dalam sistem reservasi hotel. Penerapan *primary key*, *auto increment*, *default timestamp*, dan penggunaan tipe data ENUM mendukung proses audit data yang lebih terstruktur, sehingga keamanan, integritas, dan keterlacakan data dalam sistem dapat terjaga dengan baik.

4.2 Implementasi *Data Manipulation Language* (DML)

4.2.1 Implementasi Data Tabel Tipe Kamar

Setelah struktur tabel berhasil dibuat, langkah selanjutnya adalah melakukan pengisian data awal (*dummy data*) pada tabel tipe_kamar. Data ini digunakan untuk merepresentasikan berbagai jenis kamar yang tersedia pada hotel beserta kapasitas, deskripsi, dan harga dasar masing-masing tipe kamar.

Tabel 29. *Script* Implementasi Data Tabel Tipe Kamar

```
-- TIPE KAMAR (5 baris)
INSERT INTO tipe_kamar (id_tipe, nama_tipe, kapasitas, deskripsi_tipe, harga_dasar)
```

VALUES

```
('T001', 'Standard', 2, 'Kamar standar dengan fasilitas dasar', 350000),  
( 'T002', 'Deluxe', 2, 'Kamar deluxe dengan pemandangan kota', 550000),  
( 'T003', 'Superior', 3, 'Kamar superior luas untuk 3 tamu', 700000),  
( 'T004', 'Family', 4, 'Kamar keluarga dengan 2 tempat tidur', 950000),  
( 'T005', 'Suite', 4, 'Kamar suite mewah dengan ruang tamu', 1500000);
```

Perintah INSERT INTO digunakan untuk menambahkan data baru ke dalam tabel tipe_kamar. Pada sintaks tersebut, disebutkan nama tabel yang akan diisi, yaitu tipe_kamar, kemudian diikuti daftar atribut yang akan diberikan nilai, yaitu id_tipe, nama_tipe, kapasitas, deskripsi_tipe, dan harga_dasar.

query VALUES digunakan untuk menentukan nilai yang akan dimasukkan ke dalam setiap atribut. Dalam implementasi ini, terdapat lima data tipe kamar yang dimasukkan secara sekaligus dalam satu perintah SQL, sehingga proses pengisian data menjadi lebih efisien.

Data yang dimasukkan terdiri atas lima kategori kamar, yaitu:

- 1) *Standard*, dengan kapasitas maksimal 2 tamu dan harga dasar sebesar Rp350.000.
- 2) *Deluxe*, dengan kapasitas maksimal 2 tamu dan harga dasar sebesar Rp550.000.
- 3) *Superior*, dengan kapasitas maksimal 3 tamu dan harga dasar sebesar Rp700.000.
- 4) *Family*, dengan kapasitas maksimal 4 tamu dan harga dasar sebesar Rp950.000.
- 5) *Suite*, dengan kapasitas maksimal 4 tamu dan harga dasar sebesar Rp1.500.000.

Data tersebut disusun secara realistis untuk menggambarkan kondisi sebenarnya pada sistem reservasi hotel dan digunakan sebagai data awal untuk mendukung proses pengujian sistem.

Berdasarkan implementasi yang telah dilakukan, sebanyak lima data tipe kamar berhasil ditambahkan ke dalam tabel tipe_kamar. Data tersebut selanjutnya digunakan sebagai referensi bagi tabel kamar dalam menentukan jenis kamar yang tersedia pada hotel.

Tabel 30. Hasil Pengisian Data

ID Tipe	Nama Tipe	Kapasitas	Harga Dasar
T001	Standard	2	350.000

T002	Deluxe	2	550.000
T003	Superior	3	700.000
T004	Family	4	950.000
T005	Suite	4	1.500.000

4.2.2 Implementasi Data Tabel Kamar

Setelah data tipe kamar berhasil ditambahkan, langkah berikutnya adalah melakukan pengisian data pada tabel kamar. Data ini digunakan untuk merepresentasikan kamar-kamar yang tersedia di hotel beserta nomor kamar, lokasi lantai, status ketersediaan, dan tipe kamar yang dimiliki. Data kamar yang dimasukkan telah disesuaikan dengan tipe kamar yang tersedia pada tabel tipe_kamar sehingga hubungan antar tabel tetap terjaga.

Tabel 31. *Script Implementasi Data Tabel Tipe Kamar*

```
-- KAMAR (20 baris)
INSERT INTO kamar (id_kamar, nomor_kamar, lantai, status_kamar, id_tipe)
VALUES
('K001', '101', 1, 'Terisi', 'T001'),
('K002', '102', 1, 'Terisi', 'T001'),
('K003', '103', 1, 'Dipesan', 'T001'),
('K004', '201', 2, 'Terisi', 'T002'),
('K005', '202', 2, 'Terisi', 'T002'),
('K006', '203', 2, 'Dipesan', 'T002'),
('K007', '301', 3, 'Terisi', 'T003'),
('K008', '302', 3, 'Dipesan', 'T003'),
('K009', '401', 4, 'Terisi', 'T004'),
('K010', '501', 5, 'Dipesan', 'T005'),
('K017', '509', 5, 'Terisi', 'T005'),
('K024', '123', 1, 'Terisi', 'T001'),
('K031', '456', 4, 'Terisi', 'T004'),
('K038', '222', 2, 'Terisi', 'T002'),
('K045', '104', 1, 'Terisi', 'T001'),
('K052', '105', 1, 'Terisi', 'T001'),
```

```
('K059', '106', 1, 'Dipesan', 'T001'),  
( 'K066', '306', 3, 'Dipesan', 'T003'),  
( 'K073', '506', 5, 'Dipesan', 'T005'),  
( 'K080', '408', 4, 'Dipesan', 'T004');
```

Perintah INSERT INTO digunakan untuk menambahkan data baru ke dalam tabel kamar. Pada sintaks tersebut ditentukan atribut yang akan diisi, yaitu id_kamar, nomor_kamar, lantai, status_kamar, dan id_tipe. query VALUES digunakan untuk memasukkan sejumlah data kamar secara bersamaan dalam satu perintah SQL. Metode ini lebih efisien dibandingkan melakukan proses penyisipan data satu per satu.

Data yang dimasukkan terdiri atas 20 kamar yang tersebar pada beberapa lantai hotel. Setiap kamar memiliki kode unik (id_kamar) dan nomor kamar (nomor_kamar) yang digunakan sebagai identitas fisik kamar di hotel. Atribut lantai menunjukkan lokasi kamar berdasarkan tingkat bangunan hotel. Data yang dimasukkan mencakup kamar yang berada pada lantai 1 hingga lantai 5. Atribut status_kamar digunakan untuk menunjukkan kondisi atau ketersediaan kamar pada saat data dimasukkan. Dalam implementasi ini digunakan dua status, yaitu:

- 1) Terisi, yang menunjukkan bahwa kamar sedang digunakan oleh tamu.
- 2) Dipesan, yang menunjukkan bahwa kamar telah dipesan tetapi belum ditempati.

Atribut id_tipe digunakan sebagai *foreign key* yang menghubungkan tabel kamar dengan tabel tipe_kamar. Setiap kamar dikaitkan dengan salah satu tipe kamar yang telah terdaftar sebelumnya, yaitu *Standard*, *Deluxe*, *Superior*, *Family*, atau *Suite*.

Berdasarkan implementasi yang telah dilakukan, sebanyak 20 data kamar berhasil ditambahkan ke dalam tabel kamar. Data tersebut menggambarkan kondisi ketersediaan kamar hotel yang realistis serta menunjukkan hubungan antara tabel kamar dan tabel tipe_kamar melalui atribut id_tipe. Data kamar yang telah tersimpan selanjutnya dapat digunakan dalam proses reservasi, pengelolaan status kamar, serta pelacakan ketersediaan kamar pada sistem reservasi hotel.

4.2.3 Implementasi Data Tabel Tamu

Setelah struktur tabel tamu berhasil dibuat, langkah selanjutnya adalah melakukan pengisian data awal (*dummy data*) ke dalam tabel tersebut. Data yang dimasukkan digunakan untuk merepresentasikan informasi tamu yang melakukan reservasi hotel. Data ini terdiri atas identitas tamu, alamat, nomor telepon, alamat surat elektronik (*email*), jenis

identitas, dan nomor identitas. Seluruh data yang digunakan disusun secara realistis agar dapat mendukung proses pengujian sistem reservasi hotel.

Tabel 32. *Script Implementasi Data Tabel Tamu*

```
-- TAMU (20 baris)
INSERT INTO tamu (id_tamu, nama_tamu, alamat, no_hp, email, jenis_identitas,
no_identitas) VALUES
('TM001', 'Budi Santoso', 'Jakarta', '081234567801', 'budi@gmail.com', 'KTP',
'3171111111111110'),
('TM002', 'Siti Aminah', 'Bandung', '081234567802', 'siti@gmail.com', 'KTP',
'3272222222222220'),
('TM003', 'Andi Wijaya', 'Semarang', '081234567803', 'andi@gmail.com',
'KTP', '3373333333333330'),
('TM004', 'Rina Putri', 'Yogyakarta', '081234567804', 'rina@gmail.com', 'KTP',
'3474444444444440'),
('TM005', 'Dimas Pratama', 'Surabaya', '081234567805', 'dimas@gmail.com',
'KTP', '3575555555555550'),
('TM006', 'Dewi Lestari', 'Solo', '081234567806', 'dewi@gmail.com', 'KTP',
'3376666666666660'),
('TM007', 'Fajar Nugroho', 'Malang', '081234567807', 'fajar@gmail.com', 'KTP',
'3577777777777770'),
('TM008', 'Maya Sari', 'Bogor', '081234567808', 'maya@gmail.com', 'KTP',
'3278888888888880'),
('TM009', 'Rizki Akbar', 'Bekasi', '081234567809', 'rizki@gmail.com', 'KTP',
'3179999999999990'),
('TM010', 'Nanda Putri', 'Purwokerto', '081234567810', 'nanda@gmail.com',
'KTP', '3300000000000000'),
('TM011', 'Hendra Kusuma', 'Medan', '081234567811', 'hendra@gmail.com',
'KTP', '1271111111111110'),
('TM012', 'Laila Fitriani', 'Palembang', '081234567812', 'laila@gmail.com', 'KTP',
'1672222222222220'),
('TM013', 'Yusuf Maulana', 'Makassar', '081234567813', 'yusuf@gmail.com',
'KTP', '7373333333333330'),
('TM014', 'Putri Ayu', 'Denpasar', '081234567814', 'putri@gmail.com', 'KTP',
```

```
'5174444444444440'),
('TM015', 'Galih Saputra', 'Pekanbaru', '081234567815', 'galih@gmail.com', 'KTP',
'1475555555555550'),
('TM016', 'Anisa Rahma', 'Padang', '081234567816', 'anisa@gmail.com', 'KTP',
'1376666666666660'),
('TM017', 'Reza Pratama', 'Balikpapan', '081234567817', 'reza@gmail.com',
'SIM', '6477777777777770'),
('TM018', 'Cindy Oktavia', 'Banjarmasin', '081234567818', 'cindy@gmail.com',
'KTP', '6378888888888880'),
('TM019', 'Farhan Akbar', 'Manado', '081234567819', 'farhan@gmail.com',
'KTP', '7179999999999990'),
('TM020', 'Salsabila Putri', 'Pontianak', '081234567820', 'salsa@gmail.com', 'KTP',
'6100000000000000');
```

Perintah INSERT INTO digunakan untuk menambahkan data baru ke dalam tabel tamu. Pada sintaks tersebut, ditentukan atribut yang akan diisi yaitu id_tamu, nama_tamu, alamat, no_hp, email, jenis_identitas, dan no_identitas.

query VALUES digunakan untuk memasukkan sejumlah data tamu secara bersamaan dalam satu perintah SQL. Metode ini dipilih karena lebih efisien dibandingkan melakukan proses pengisian data satu per satu.

Data yang dimasukkan terdiri atas 20 data tamu yang berasal dari berbagai kota di Indonesia, seperti Jakarta, Bandung, Semarang, Yogyakarta, Surabaya, Solo, Malang, Bogor, Bekasi, Purwokerto, Medan, Palembang, Makassar, Denpasar, Pekanbaru, Padang, Balikpapan, Banjarmasin, Manado, dan Pontianak.

Setiap data tamu memiliki kode unik yang tersimpan pada atribut id_tamu. Atribut nama_tamu digunakan untuk menyimpan nama lengkap tamu, sedangkan atribut alamat digunakan untuk menyimpan kota asal tamu.

Atribut no_hp berisi nomor telepon yang dapat digunakan sebagai sarana komunikasi antara pihak hotel dan tamu. Sementara itu, atribut email digunakan untuk menyimpan alamat surat elektronik yang dapat dimanfaatkan untuk pengiriman informasi reservasi maupun konfirmasi pembayaran.

Atribut jenis_identitas digunakan untuk menunjukkan jenis dokumen identitas yang digunakan oleh tamu. Pada data yang dimasukkan, sebagian besar tamu menggunakan

identitas berupa KTP, sedangkan salah satu tamu menggunakan SIM sebagai identitas resmi.

Atribut `no_identitas` digunakan untuk menyimpan nomor identitas yang sesuai dengan jenis identitas yang dipilih. Seluruh nomor identitas dibuat unik sehingga memenuhi aturan *unique constraint* yang telah diterapkan pada tabel.

Berdasarkan implementasi yang telah dilakukan, sebanyak 20 data tamu berhasil ditambahkan ke dalam tabel tamu. Data tersebut digunakan sebagai data referensi dalam proses reservasi kamar, pembayaran, check-in, check-out, serta pemberian ulasan pada sistem reservasi hotel.

4.2.4 Implementasi Data Tabel Pegawai

Setelah struktur tabel pegawai berhasil dibuat, langkah selanjutnya adalah melakukan pengisian data pegawai yang bertugas dalam operasional hotel. Data pegawai digunakan untuk mendukung berbagai aktivitas layanan hotel, seperti proses check-in dan check-out tamu. Selain itu, data ini juga berfungsi sebagai referensi dalam pencatatan transaksi yang melibatkan petugas hotel.

Tabel 33. Script Implementasi Data Tabel Pegawai

```
INSERT INTO pegawai (id_pegawai, nama_pegawai, jabatan, no_hp) VALUES  
( 'P001', 'Ahmad Fauzi', 'Resepsionis', '081300000001'),  
( 'P002', 'Yuni Astuti', 'Resepsionis', '081300000002'),  
( 'P003', 'Dedi Saputra', 'Supervisor', '081300000003'),  
( 'P004', 'Rika Maharani', 'Front Office', '081300000004');
```

Perintah `INSERT INTO` digunakan untuk menambahkan data baru ke dalam tabel pegawai. Pada sintaks tersebut, ditentukan atribut yang akan diisi, yaitu `id_pegawai`, `nama_pegawai`, `jabatan`, dan `no_hp`. query `VALUES` digunakan untuk memasukkan beberapa data pegawai sekaligus dalam satu perintah SQL. Metode ini lebih efisien dibandingkan melakukan pengisian data satu per satu.

Atribut `id_pegawai` berisi kode unik yang digunakan untuk membedakan setiap pegawai dalam sistem. Kode tersebut digunakan sebagai identitas utama yang nantinya dapat direferensikan oleh tabel lain, seperti tabel `check_in` dan `check_out`. Atribut `nama_pegawai` digunakan untuk menyimpan nama lengkap pegawai yang bekerja di hotel. Data nama pegawai diperlukan untuk keperluan identifikasi dan administrasi operasional. Atribut `jabatan` digunakan untuk menyimpan posisi atau tugas pegawai dalam organisasi

hotel. Pada data yang dimasukkan terdapat beberapa jabatan, yaitu Resepsionis, *Supervisor*, dan *Front Office*. Atribut *no_hp* digunakan untuk menyimpan nomor telepon pegawai yang dapat digunakan sebagai sarana komunikasi internal maupun administrasi.

Berdasarkan implementasi yang telah dilakukan, sebanyak 4 data pegawai berhasil ditambahkan ke dalam tabel pegawai. Data tersebut terdiri atas dua pegawai dengan jabatan resepsionis, satu *supervisor*, dan satu petugas *front office*.

4.2.5 Implementasi Data Tabel Pegawai

Setelah struktur tabel fasilitas berhasil dibuat, langkah selanjutnya adalah melakukan pengisian data fasilitas yang tersedia di hotel. Data fasilitas digunakan untuk memberikan informasi mengenai layanan dan sarana yang dapat dimanfaatkan oleh tamu selama menginap. Keberadaan data fasilitas juga mendukung pengelolaan layanan hotel agar dapat memberikan kenyamanan dan kepuasan kepada pelanggan.

Tabel 34. *Script Implementasi Data Tabel Pegawai*

```
-- FASILITAS (5 baris)
INSERT INTO fasilitas (id_fasilitas, nama_fasilitas, kategori, deskripsi_fasilitas,
biaya) VALUES
('F001', 'WiFi',      'Internet', 'Akses internet gratis',      0),
('F002', 'Breakfast', 'Makanan', 'Sarapan pagi prasmanan', 50000),
('F003', 'Kolam Renang', 'Rekreasi', 'Akses kolam renang outdoor', 0),
('F004', 'Gym',      'Olahraga', 'Akses pusat kebugaran',      0),
('F005', 'Laundry',   'Layanan', 'Jasa cuci & setrika baju', 30000);
```

Perintah `INSERT INTO` digunakan untuk menambahkan data baru ke dalam tabel fasilitas. Pada sintaks tersebut ditentukan atribut yang akan diisi, yaitu *id_fasilitas*, *nama_fasilitas*, *kategori*, *deskripsi_fasilitas*, dan *biaya*. query `VALUES` digunakan untuk memasukkan beberapa data fasilitas sekaligus dalam satu perintah SQL. Cara ini dipilih karena lebih efisien dibandingkan memasukkan data satu per satu.

Atribut *id_fasilitas* digunakan untuk menyimpan kode unik yang berfungsi sebagai identitas setiap fasilitas yang tersedia di hotel. Kode ini memudahkan proses pengelolaan dan pencarian data fasilitas. Atribut *nama_fasilitas* digunakan untuk menyimpan nama fasilitas yang disediakan oleh hotel. Pada implementasi ini terdapat lima jenis fasilitas, yaitu WiFi, Breakfast, Kolam Renang, Gym, dan Laundry.

Atribut kategori digunakan untuk mengelompokkan fasilitas berdasarkan jenis layanannya. Kategori yang digunakan meliputi Internet, Makanan, Rekreasi, Olahraga, dan Layanan. Atribut deskripsi_fasilitas digunakan untuk menyimpan penjelasan singkat mengenai fasilitas yang tersedia sehingga pengguna dapat mengetahui fungsi dan manfaat dari fasilitas tersebut. Atribut biaya digunakan untuk menyimpan tarif penggunaan fasilitas. Nilai 0 menunjukkan bahwa fasilitas dapat digunakan secara gratis oleh tamu hotel, sedangkan nilai selain nol menunjukkan adanya biaya tambahan yang harus dibayarkan oleh tamu.

Berdasarkan implementasi yang telah dilakukan, sebanyak 5 data fasilitas berhasil ditambahkan ke dalam tabel fasilitas. Data tersebut menggambarkan berbagai layanan yang tersedia untuk menunjang kenyamanan tamu selama menginap.

4.2.6 Implementasi Data Tabel Reservasi

Setelah data tamu dan data kamar berhasil ditambahkan ke dalam sistem, langkah berikutnya adalah melakukan pengisian data pada tabel **reservasi**. Tabel ini merupakan tabel utama yang digunakan untuk mencatat seluruh transaksi pemesanan kamar yang dilakukan oleh tamu. Data reservasi mencakup informasi mengenai tanggal pemesanan, jadwal menginap, jumlah tamu, status reservasi, identitas tamu, serta kamar yang dipesan.

Tabel 35. *Script* Implementasi Data Tabel Reservasi

```
-- RESERVASI (20 baris)
INSERT INTO reservasi (id_reservasi, tanggal_pesan, check_in, check_out,
jumlah_tamu, status_reservasi, id_tamu, id_kamar) VALUES
('R001', '2026-06-01', '2026-06-05', '2026-06-07', 2, 'Selesai', 'TM001', 'K001'),
('R002', '2026-06-02', '2026-06-06', '2026-06-08', 2, 'Selesai', 'TM002', 'K004'),
('R003', '2026-06-03', '2026-06-10', '2026-06-12', 3, 'Selesai', 'TM003', 'K007'),
('R004', '2026-06-04', '2026-06-11', '2026-06-13', 2, 'Selesai', 'TM004', 'K002'),
('R005', '2026-06-05', '2026-06-15', '2026-06-17', 4, 'Selesai', 'TM005', 'K009'),
('R006', '2026-06-06', '2026-06-20', '2026-06-22', 2, 'Selesai', 'TM006', 'K005'),
('R007', '2026-06-07', '2026-06-25', '2026-06-27', 2, 'Dipesan', 'TM007', 'K006'),
('R008', '2026-06-08', '2026-06-28', '2026-06-30', 3, 'Dipesan', 'TM008', 'K008'),
('R009', '2026-06-09', '2026-07-01', '2026-07-03', 2, 'Dipesan', 'TM009', 'K003'),
('R010', '2026-06-10', '2026-07-05', '2026-07-07', 4, 'Dipesan', 'TM010', 'K010'),
('R011', '2026-06-11', '2026-07-08', '2026-07-10', 2, 'Selesai', 'TM001', 'K017'),
```

```

('R012', '2026-06-12', '2026-07-11', '2026-07-13', 2, 'Selesai', 'TM002', 'K024'),
('R013', '2026-06-13', '2026-07-14', '2026-07-16', 3, 'Selesai', 'TM003', 'K031'),
('R014', '2026-06-14', '2026-07-17', '2026-07-19', 2, 'Selesai', 'TM004', 'K038'),
('R015', '2026-06-15', '2026-07-20', '2026-07-22', 4, 'Selesai', 'TM005', 'K045'),
('R016', '2026-06-16', '2026-07-23', '2026-07-25', 2, 'Selesai', 'TM006', 'K052'),
('R017', '2026-06-17', '2026-07-26', '2026-07-28', 2, 'Dipesan', 'TM007', 'K059'),
('R018', '2026-06-18', '2026-07-29', '2026-07-31', 3, 'Dipesan', 'TM008', 'K066'),
('R019', '2026-06-19', '2026-08-01', '2026-08-03', 2, 'Dipesan', 'TM009', 'K073'),
('R020', '2026-06-20', '2026-08-05', '2026-08-07', 4, 'Selesai', 'TM010', 'K080');

```

Perintah INSERT INTO digunakan untuk menambahkan data baru ke dalam tabel reservasi. Pada sintaks tersebut ditentukan atribut yang akan diisi, yaitu id_reservasi, tanggal_pesan, check_in, check_out, jumlah_tamu, status_reservasi, id_tamu, dan id_kamar. query VALUES digunakan untuk memasukkan beberapa data reservasi sekaligus dalam satu perintah SQL sehingga proses pengisian data menjadi lebih efisien.

Atribut id_reservasi berisi kode unik yang digunakan sebagai identitas setiap transaksi reservasi yang dilakukan oleh tamu. Kode ini menjadi referensi utama bagi tabel lain seperti tabel pembayaran, check-in, check-out, dan ulasan. Atribut tanggal_pesan digunakan untuk menyimpan tanggal saat tamu melakukan pemesanan kamar. Informasi ini dapat digunakan untuk mengetahui riwayat pemesanan dan aktivitas reservasi hotel.

Atribut check_in digunakan untuk menyimpan tanggal kedatangan tamu, sedangkan atribut check_out digunakan untuk menyimpan tanggal keberangkatan tamu. Kedua atribut tersebut berfungsi sebagai dasar dalam menentukan lama masa menginap tamu. Atribut jumlah_tamu digunakan untuk menyimpan jumlah tamu yang akan menempati kamar sesuai dengan reservasi yang dilakukan. Data yang dimasukkan memiliki jumlah tamu antara dua hingga empat orang sesuai kapasitas kamar yang tersedia.

Atribut status_reservasi digunakan untuk menunjukkan kondisi reservasi. Pada implementasi ini digunakan dua status, yaitu:

- 1) Selesai, yang menunjukkan bahwa tamu telah menyelesaikan masa menginapnya.
- 2) Dipesan, yang menunjukkan bahwa reservasi telah dilakukan, namun masa menginap belum berlangsung atau belum selesai.

Atribut id_tamu merupakan *foreign key* yang menghubungkan tabel reservasi dengan tabel tamu. Relasi ini memastikan bahwa setiap reservasi harus dimiliki oleh tamu yang

telah terdaftar dalam sistem. Atribut `id_kamar` merupakan *foreign key* yang menghubungkan tabel reservasi dengan tabel kamar. Relasi ini digunakan untuk menunjukkan kamar yang dipesan oleh tamu.

Berdasarkan implementasi yang telah dilakukan, sebanyak 20 data reservasi berhasil ditambahkan ke dalam tabel reservasi. Data tersebut menggambarkan aktivitas pemesanan kamar yang dilakukan oleh berbagai tamu dengan jadwal menginap yang berbeda.

Data reservasi yang telah tersimpan menjadi pusat hubungan (*central transaction table*) dalam sistem reservasi hotel karena terhubung dengan tabel tamu, kamar, pembayaran, `check_in`, `check_out`, dan ulasan. Dengan demikian, tabel ini berperan penting dalam mendukung seluruh proses bisnis yang terjadi pada sistem reservasi hotel.

4.2.7 Implementasi Data Tabel Pembayaran

Setelah data reservasi berhasil ditambahkan ke dalam sistem, langkah berikutnya adalah melakukan pengisian data pada tabel pembayaran. Tabel ini digunakan untuk mencatat seluruh transaksi pembayaran yang dilakukan oleh tamu terhadap reservasi yang telah dibuat. Data pembayaran berisi informasi mengenai tanggal pembayaran, jumlah pembayaran, metode pembayaran, status pembayaran, nomor referensi transaksi, serta identitas reservasi yang terkait.

Tabel 36. *Script Implementasi Data Tabel Pembayaran*

```
-- PEMBAYARAN (20 baris)
INSERT INTO pembayaran (id_pembayaran, tanggal_bayar, jumlah_bayar,
metode_bayar, status_bayar, referensi, id_reservasi) VALUES
('PB001', '2026-06-01', 700000, 'Transfer', 'Lunas', 'TRF001', 'R001'),
('PB002', '2026-06-02', 1100000, 'QRIS', 'Lunas', 'QR001', 'R002'),
('PB003', '2026-06-03', 1400000, 'Transfer', 'Lunas', 'TRF003', 'R003'),
('PB004', '2026-06-04', 700000, 'Cash', 'Lunas', 'CASH004', 'R004'),
('PB005', '2026-06-05', 1900000, 'Transfer', 'Lunas', 'TRF005', 'R005'),
('PB006', '2026-06-06', 1100000, 'QRIS', 'Lunas', 'QR006', 'R006'),
('PB007', '2026-06-07', 550000, 'Transfer', 'Lunas', 'TRF007', 'R007'),
('PB008', '2026-06-08', 700000, 'Transfer', 'Lunas', 'TRF008', 'R008'),
('PB009', '2026-06-09', 350000, 'Cash', 'Lunas', 'CASH009', 'R009'),
('PB010', '2026-06-10', 1500000, 'Transfer', 'Lunas', 'TRF010', 'R010'),
('PB011', '2026-06-11', 700000, 'Transfer', 'Lunas', 'TRF011', 'R011'),
('PB012', '2026-06-12', 1100000, 'QRIS', 'Lunas', 'QR012', 'R012'),
('PB013', '2026-06-13', 1400000, 'Transfer', 'Lunas', 'TRF013', 'R013'),
('PB014', '2026-06-14', 700000, 'Cash', 'Lunas', 'CASH014', 'R014'),
('PB015', '2026-06-15', 1900000, 'Transfer', 'Lunas', 'TRF015', 'R015'),
('PB016', '2026-06-16', 1100000, 'QRIS', 'Lunas', 'QR016', 'R016'),
('PB017', '2026-06-17', 1100000, 'Transfer', 'Lunas', 'TRF017', 'R017'),
('PB018', '2026-06-18', 1400000, 'Transfer', 'Lunas', 'TRF018', 'R018'),
('PB019', '2026-06-19', 700000, 'Cash', 'Lunas', 'CASH019', 'R019'),
('PB020', '2026-06-20', 3000000, 'Transfer', 'Lunas', 'TRF020', 'R020');
```

Perintah INSERT INTO digunakan untuk menambahkan data baru ke dalam tabel pembayaran. Pada sintaks tersebut ditentukan atribut yang akan diisi, yaitu id_pembayaran, tanggal_bayar, jumlah_bayar, metode_bayar, status_bayar, referensi, dan id_reservasi. query VALUES digunakan untuk memasukkan beberapa data pembayaran sekaligus dalam satu perintah SQL sehingga proses pengisian data menjadi lebih cepat dan efisien.

Atribut id_pembayaran digunakan untuk menyimpan kode unik yang berfungsi sebagai identitas setiap transaksi pembayaran. Kode ini digunakan untuk membedakan satu

transaksi dengan transaksi lainnya. Atribut tanggal_bayar digunakan untuk menyimpan informasi mengenai tanggal ketika pembayaran dilakukan oleh tamu. Data ini dapat digunakan untuk kebutuhan pelaporan keuangan dan pencatatan transaksi.

Atribut jumlah_bayar digunakan untuk menyimpan nominal pembayaran yang dibayarkan oleh tamu. Besarnya pembayaran disesuaikan dengan jenis kamar yang dipesan dan lama masa menginap. Atribut metode_bayar digunakan untuk menyimpan metode pembayaran yang digunakan oleh tamu. Pada implementasi ini digunakan beberapa metode pembayaran, yaitu Transfer, QRIS, dan Cash.

Atribut status_bayar digunakan untuk menunjukkan status transaksi pembayaran. Seluruh data yang dimasukkan memiliki status Lunas, yang menunjukkan bahwa pembayaran telah berhasil diselesaikan oleh tamu. Atribut referensi digunakan untuk menyimpan nomor referensi transaksi pembayaran. Nomor referensi ini dapat berupa kode transfer bank, kode QRIS, maupun kode transaksi pembayaran tunai yang digunakan sebagai bukti transaksi.

Atribut id_reservasi merupakan *foreign key* yang menghubungkan tabel pembayaran dengan tabel reservasi. Relasi ini memastikan bahwa setiap pembayaran yang dilakukan harus berkaitan dengan data reservasi yang telah terdaftar dalam sistem.

Berdasarkan implementasi yang telah dilakukan, sebanyak 20 data pembayaran berhasil ditambahkan ke dalam tabel pembayaran. Setiap data pembayaran terhubung dengan satu data reservasi sehingga riwayat transaksi dapat ditelusuri dengan baik.

Dengan pengisian data tersebut, tabel pembayaran telah siap digunakan untuk mendukung proses pengelolaan transaksi keuangan pada sistem reservasi hotel. Data pembayaran yang tersimpan juga dapat dimanfaatkan untuk kebutuhan pelaporan pendapatan dan monitoring transaksi hotel.

4.2.8 Implementasi Data Tabel Check-in

Setelah data reservasi berhasil ditambahkan ke dalam sistem, langkah selanjutnya adalah melakukan pengisian data pada tabel check_in. Tabel ini digunakan untuk mencatat proses kedatangan tamu saat mulai menginap di hotel. Data yang tersimpan meliputi waktu check-in, identitas reservasi, serta pegawai yang bertugas melayani proses check-in. Pencatatan data check-in sangat penting untuk memastikan bahwa setiap tamu yang datang telah terdaftar dan sesuai dengan data reservasi yang dimiliki.

Tabel 37. *Script Implementasi Data Tabel Check-in*

```
-- CHECK_IN (20 baris)
INSERT INTO check_in (id_checkin, waktu_checkin, id_reservasi, id_pegawai)
VALUES
('CI001', '2026-06-05 13:00:00', 'R001', 'P001'),
('CI002', '2026-06-06 14:00:00', 'R002', 'P002'),
('CI003', '2026-06-10 13:15:00', 'R003', 'P001'),
('CI004', '2026-06-11 14:20:00', 'R004', 'P003'),
('CI005', '2026-06-15 13:30:00', 'R005', 'P004'),
('CI006', '2026-06-20 14:00:00', 'R006', 'P002'),
('CI007', '2026-06-25 14:30:00', 'R007', 'P001'),
('CI008', '2026-06-28 15:00:00', 'R008', 'P002'),
('CI009', '2026-07-01 15:30:00', 'R009', 'P001'),
('CI010', '2026-07-05 16:00:00', 'R010', 'P003'),
('CI011', '2026-07-08 16:30:00', 'R011', 'P004'),
('CI012', '2026-07-11 17:00:00', 'R012', 'P002'),
('CI013', '2026-07-14 17:30:00', 'R013', 'P001'),
('CI014', '2026-07-17 18:00:00', 'R014', 'P002'),
('CI015', '2026-07-20 18:30:00', 'R015', 'P001'),
('CI016', '2026-07-23 19:00:00', 'R016', 'P003'),
('CI017', '2026-07-26 19:30:00', 'R017', 'P004'),
('CI018', '2026-07-29 20:00:00', 'R018', 'P002'),
('CI019', '2026-08-01 20:30:00', 'R019', 'P001'),
('CI020', '2026-08-05 21:00:00', 'R020', 'P002');
```

Perintah INSERT INTO digunakan untuk menambahkan data baru ke dalam tabel check_in. Pada sintaks tersebut ditentukan atribut yang akan diisi, yaitu id_checkin, waktu_checkin, id_reservasi, dan id_pegawai. query VALUES digunakan untuk memasukkan beberapa data check-in sekaligus dalam satu perintah SQL sehingga proses pengisian data menjadi lebih efisien.

Atribut id_checkin digunakan untuk menyimpan kode unik yang berfungsi sebagai identitas setiap transaksi check-in. Kode ini digunakan untuk membedakan satu data check-in dengan data lainnya. Atribut waktu_checkin digunakan untuk menyimpan

informasi tanggal dan waktu kedatangan tamu secara lengkap. Penggunaan tipe data DATETIME memungkinkan sistem mencatat waktu check-in dengan tingkat ketelitian hingga satuan detik.

Atribut `id_reservasi` merupakan *foreign key* yang menghubungkan tabel `check_in` dengan tabel `reservasi`. Relasi ini memastikan bahwa setiap proses check-in hanya dapat dilakukan oleh tamu yang memiliki data reservasi yang valid. Atribut `id_pegawai` merupakan *foreign key* yang menghubungkan tabel `check_in` dengan tabel `pegawai`. Relasi ini digunakan untuk mencatat pegawai yang bertanggung jawab dalam melayani proses check-in tamu.

Data yang dimasukkan menunjukkan bahwa proses check-in dilakukan pada berbagai tanggal dan waktu yang berbeda sesuai dengan jadwal kedatangan tamu. Selain itu, proses check-in dilayani oleh beberapa pegawai yang memiliki tugas pada bagian resepsionis maupun *front office*.

Berdasarkan implementasi yang telah dilakukan, sebanyak 20 data check-in berhasil ditambahkan ke dalam tabel `check_in`. Seluruh data check-in terhubung dengan data reservasi dan data pegawai sehingga proses kedatangan tamu dapat terdokumentasi dengan baik.

Dengan pengisian data tersebut, tabel `check_in` telah siap digunakan untuk mendukung proses administrasi kedatangan tamu. Data yang tersimpan juga dapat dimanfaatkan untuk keperluan pelaporan operasional dan pemantauan aktivitas pelayanan hotel.

4.2.9 Implementasi Data Tabel Check-out

Setelah proses check-in dilakukan, sistem juga perlu mencatat proses keberangkatan tamu melalui tabel `check_out`. Tabel ini digunakan untuk menyimpan informasi terkait waktu tamu meninggalkan hotel setelah masa menginap berakhir. Selain itu, tabel ini juga mencatat pegawai yang bertugas menangani proses check-out sehingga seluruh aktivitas tamu dapat terdokumentasi dengan baik.

Tabel 38. *Script Implementasi Data Tabel Check-out*

```
-- CHECK_OUT (20 baris)
INSERT INTO check_out (id_checkout, waktu_checkout, id_reservasi, id_pegawai)
VALUES
('CO001', '2026-06-07 11:00:00', 'R001', 'P001'),
('CO002', '2026-06-08 11:30:00', 'R002', 'P002'),
```

```

('CO003', '2026-06-12 11:00:00', 'R003', 'P001'),
('CO004', '2026-06-13 11:15:00', 'R004', 'P003'),
('CO005', '2026-06-17 11:00:00', 'R005', 'P004'),
('CO006', '2026-06-22 11:00:00', 'R006', 'P002'),
('CO007', '2026-06-27 11:00:00', 'R007', 'P001'),
('CO008', '2026-06-30 11:00:00', 'R008', 'P002'),
('CO009', '2026-07-03 11:00:00', 'R009', 'P001'),
('CO010', '2026-07-07 11:00:00', 'R010', 'P003'),
('CO011', '2026-07-10 11:00:00', 'R011', 'P004'),
('CO012', '2026-07-13 11:00:00', 'R012', 'P002'),
('CO013', '2026-07-16 11:00:00', 'R013', 'P001'),
('CO014', '2026-07-19 11:00:00', 'R014', 'P002'),
('CO015', '2026-07-22 11:00:00', 'R015', 'P001'),
('CO016', '2026-07-25 11:00:00', 'R016', 'P003'),
('CO017', '2026-07-28 11:00:00', 'R017', 'P004'),
('CO018', '2026-07-31 11:00:00', 'R018', 'P002'),
('CO019', '2026-08-03 11:00:00', 'R019', 'P001'),
('CO020', '2026-08-07 11:00:00', 'R020', 'P002');

```

Perintah INSERT INTO digunakan untuk menambahkan data baru ke dalam tabel check_out. Pada sintaks tersebut ditentukan atribut yang akan diisi, yaitu id_checkout, waktu_checkout, id_reservasi, dan id_pegawai. query VALUES digunakan untuk memasukkan beberapa data check-out sekaligus dalam satu perintah SQL sehingga proses pengisian data menjadi lebih efisien.

Atribut id_checkout digunakan untuk menyimpan kode unik yang berfungsi sebagai identitas setiap transaksi check-out. Kode ini digunakan untuk membedakan setiap data check-out yang tersimpan dalam sistem. Atribut waktu_checkout digunakan untuk menyimpan informasi tanggal dan waktu keberangkatan tamu secara lengkap. Data ini dicatat menggunakan tipe data DATETIME sehingga sistem dapat merekam waktu check-out hingga tingkat ketelitian detik.

Atribut id_reservasi merupakan *foreign key* yang menghubungkan tabel check_out dengan tabel reservasi. Relasi ini memastikan bahwa setiap proses check-out harus terkait dengan data reservasi yang valid dan telah tercatat sebelumnya.

Atribut `id_pegawai` merupakan *foreign key* yang menghubungkan tabel `check_out` dengan tabel `pegawai`. Relasi ini digunakan untuk mencatat pegawai yang bertanggung jawab dalam melayani proses check-out tamu.

Data yang dimasukkan menunjukkan bahwa setiap reservasi memiliki satu data check-out yang dicatat setelah tamu menyelesaikan masa menginapnya. Sebagian besar proses check-out dilakukan pada pukul 11.00 sesuai dengan kebijakan jam check-out hotel, sedangkan beberapa transaksi memiliki waktu yang sedikit berbeda karena kebutuhan operasional tertentu.

Implementasi data pada tabel `check_out` berhasil dilakukan dengan menambahkan 20 data transaksi check-out yang terhubung dengan tabel reservasi dan pegawai. Data tersebut digunakan untuk mencatat waktu keberangkatan tamu serta pegawai yang melayani proses check-out. Dengan adanya tabel ini, sistem reservasi hotel dapat mengelola dan memantau riwayat keberangkatan tamu secara terstruktur sehingga mendukung kebutuhan administrasi dan pelaporan operasional hotel.

4.2.9 Implementasi Data Tabel Ulasan

Setelah tamu menyelesaikan masa menginapnya, sistem menyediakan fitur pemberian ulasan sebagai bentuk evaluasi terhadap kualitas layanan hotel. Data ulasan disimpan pada tabel ulasan yang berfungsi untuk mencatat penilaian dan komentar dari tamu berdasarkan pengalaman mereka selama menginap. Informasi ini dapat digunakan oleh pihak hotel sebagai bahan evaluasi dan peningkatan kualitas pelayanan.

Tabel 39. *Script* Implementasi Data Tabel Ulasan

```
-- ULASAN (20 baris)
INSERT INTO ulasan (id_ulasan, tanggal_ulasan, rating, komentar, id_reservasi)
VALUES
('U001', '2026-06-07', 5, 'Sangat puas, kamar bersih dan nyaman', 'R001'),
('U002', '2026-06-08', 4, 'Kamar nyaman, pelayanan ramah', 'R002'),
('U003', '2026-06-12', 5, 'Fasilitas lengkap, pasti balik lagi', 'R003'),
('U004', '2026-06-13', 4, 'Pelayanan baik, lokasi strategis', 'R004'),
('U005', '2026-06-17', 5, 'Cocok untuk keluarga, anak-anak senang', 'R005'),
('U006', '2026-06-22', 4, 'Bersih dan rapi, sarapan enak', 'R006'),
('U007', '2026-06-27', 4, 'Sangat puas dengan fasilitas gym', 'R007'),
('U008', '2026-06-30', 5, 'Kamar nyaman dan luas', 'R008'),
```

```

('U009', '2026-07-03', 4, 'Fasilitas lengkap, WiFi kencang', 'R009'),
('U010', '2026-07-07', 5, 'Pelayanan terbaik, kamar mewah', 'R010'),
('U011', '2026-07-10', 4, 'Cocok untuk keluarga, kolam renang bagus', 'R011'),
('U012', '2026-07-13', 4, 'Bersih dan rapi, harga terjangkau', 'R012'),
('U013', '2026-07-16', 5, 'Sangat puas, akan merekomendasikan', 'R013'),
('U014', '2026-07-19', 4, 'Kamar nyaman, dekat pusat kota', 'R014'),
('U015', '2026-07-22', 5, 'Fasilitas lengkap dan modern', 'R015'),
('U016', '2026-07-25', 4, 'Pelayanan baik dan profesional', 'R016'),
('U017', '2026-07-28', 3, 'Cukup hemat untuk budget traveler', 'R017'),
('U018', '2026-07-31', 3, 'Harga murah, fasilitas standar', 'R018'),
('U019', '2026-08-03', 4, 'Bagus dan nyaman untuk pasangan', 'R019'),
('U020', '2026-08-07', 5, 'Sangat rapi, layanan prima', 'R020');

```

Perintah INSERT INTO digunakan untuk menambahkan data baru ke dalam tabel ulasan. Pada sintaks tersebut ditentukan atribut yang akan diisi, yaitu id_ulasan, tanggal_ulasan, rating, komentar, dan id_reservasi. query VALUES digunakan untuk memasukkan beberapa data ulasan sekaligus dalam satu perintah SQL sehingga proses pengisian data menjadi lebih efisien.

Atribut id_ulasan digunakan untuk menyimpan kode unik yang berfungsi sebagai identitas setiap ulasan yang diberikan oleh tamu. Kode ini memastikan bahwa setiap ulasan dapat dibedakan satu sama lain. Atribut tanggal_ulasan digunakan untuk menyimpan tanggal ketika ulasan diberikan oleh tamu. Data ini berguna untuk mengetahui waktu pemberian penilaian dan melakukan analisis kepuasan pelanggan berdasarkan periode tertentu.

Atribut rating digunakan untuk menyimpan nilai penilaian yang diberikan oleh tamu terhadap layanan hotel. Nilai rating menggunakan skala 1 sampai 5, di mana nilai yang lebih tinggi menunjukkan tingkat kepuasan yang lebih baik. Pada implementasi ini, rating yang diberikan berada pada rentang 3 hingga 5.

Atribut komentar digunakan untuk menyimpan tanggapan atau pengalaman tamu selama menginap. Komentar yang diberikan mencakup berbagai aspek, seperti kebersihan kamar, kualitas pelayanan, fasilitas hotel, kenyamanan, dan lokasi hotel. Atribut id_reservasi merupakan *foreign key* yang menghubungkan tabel ulasan dengan tabel

reservasi. Relasi ini memastikan bahwa setiap ulasan hanya dapat diberikan untuk reservasi yang telah tercatat dalam sistem.

Implementasi data pada tabel ulasan berhasil dilakukan dengan menambahkan 20 data ulasan yang berasal dari tamu hotel. Data tersebut berisi informasi mengenai tanggal ulasan, rating, komentar, dan reservasi yang terkait. Dengan adanya tabel ulasan, sistem reservasi hotel dapat mengumpulkan umpan balik dari pelanggan sebagai dasar evaluasi dan peningkatan kualitas layanan di masa mendatang.

BAB V

PENGUJIAN QUERY

5.1 Query SELECT

5.1.1 Query 1 - Daftar Kamar Beserta Tipe (Sederhana)

Query ini digunakan untuk menampilkan informasi seluruh kamar yang tersedia di hotel beserta tipe kamar yang dimiliki oleh masing-masing kamar. Data diperoleh dari tabel kamar dan tabel tipe_kamar yang digabungkan menggunakan perintah JOIN berdasarkan atribut id_tipe. Penggabungan kedua tabel tersebut memungkinkan informasi kamar dan informasi tipe kamar ditampilkan dalam satu hasil query yang terintegrasi.

Kolom yang ditampilkan meliputi id_kamar, nomor_kamar, lantai, dan status_kamar dari tabel kamar, serta nama_tipe dan harga_dasar dari tabel tipe_kamar. Dengan demikian, pengguna dapat melihat informasi lengkap mengenai setiap kamar, termasuk jenis kamar dan tarif dasarnya tanpa perlu melakukan pencarian pada tabel yang berbeda.

Pada bagian JOIN tipe_kamar t ON k.id_tipe = t.id_tipe, sistem mencocokkan nilai id_tipe pada tabel kamar dengan id_tipe pada tabel tipe_kamar sehingga setiap kamar dapat dikaitkan dengan tipe yang sesuai. Selanjutnya, perintah ORDER BY k.lantai, k.nomor_kamar digunakan untuk mengurutkan data berdasarkan lantai terlebih dahulu, kemudian berdasarkan nomor kamar. Pengurutan ini membuat hasil yang ditampilkan menjadi lebih rapi dan mudah dipahami.

Hasil dari query ini berupa daftar seluruh kamar hotel yang telah dilengkapi dengan informasi tipe kamar dan harga dasar masing-masing. Informasi tersebut dapat dimanfaatkan oleh pihak hotel untuk memantau ketersediaan kamar, mengetahui status setiap kamar, serta melihat tarif yang berlaku sesuai dengan tipe kamar yang dimiliki.

Script:

Tabel 40. *Script* Query 1 - Daftar Kamar Beserta Tipe (Sederhana)

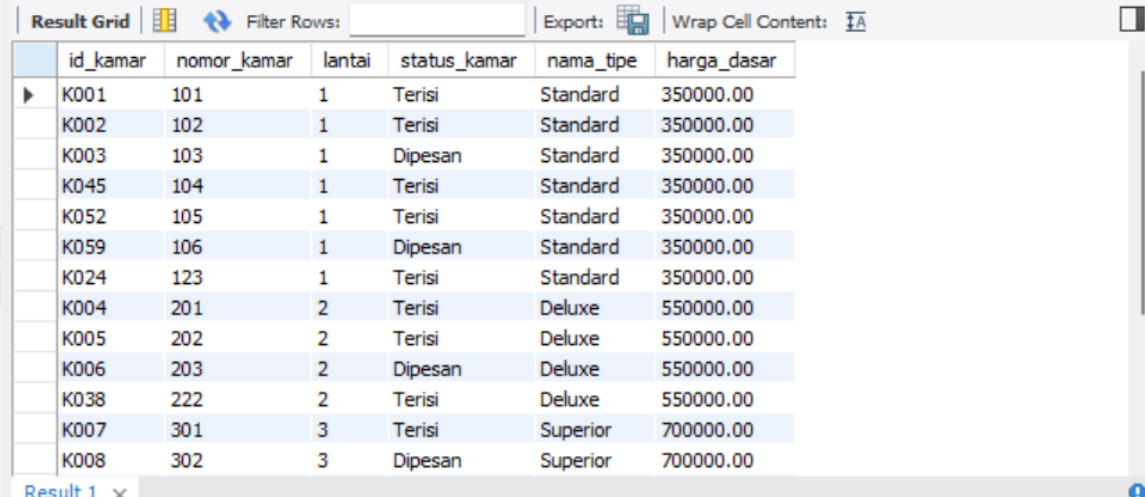
```
-- QUERY 1 (Sederhana) - Daftar semua kamar beserta tipe kamarnya
USE hotel_reservasi
SELECT
    k.id_kamar,
    k.nomor_kamar,
    k.lantai,
    k.status_kamar,
```

```

t.nama_tipe,
t.harga_dasar
FROM kamar k
JOIN tipe_kamar t ON k.id_tipe = t.id_tipe
ORDER BY k.lantai, k.nomor_kamar;

```

Output:



The screenshot shows a 'Result Grid' window with a table of room data. The table has columns: id_kamar, nomor_kamar, lantai, status_kamar, nama_tipe, and harga_dasar. The data is sorted by floor (lantai) and room number (nomor_kamar). The interface includes a 'Filter Rows' field, an 'Export' button, and a 'Wrap Cell Content' checkbox.

	id_kamar	nomor_kamar	lantai	status_kamar	nama_tipe	harga_dasar
▶	K001	101	1	Terisi	Standard	350000.00
	K002	102	1	Terisi	Standard	350000.00
	K003	103	1	Dipesan	Standard	350000.00
	K045	104	1	Terisi	Standard	350000.00
	K052	105	1	Terisi	Standard	350000.00
	K059	106	1	Dipesan	Standard	350000.00
	K024	123	1	Terisi	Standard	350000.00
	K004	201	2	Terisi	Deluxe	550000.00
	K005	202	2	Terisi	Deluxe	550000.00
	K006	203	2	Dipesan	Deluxe	550000.00
	K038	222	2	Terisi	Deluxe	550000.00
	K007	301	3	Terisi	Superior	700000.00
	K008	302	3	Dipesan	Superior	700000.00

Gambar 5.1.1 *Output Query 1*

5.1.2 Query 2 - Daftar Tamu (Sederhana)

Query ini digunakan untuk menampilkan seluruh data tamu yang telah terdaftar dalam sistem reservasi hotel. Data yang diambil berasal dari tabel tamu, yang menyimpan informasi identitas dan kontak setiap tamu yang pernah melakukan reservasi atau menginap di hotel. Query ini berguna untuk melihat daftar tamu secara lengkap sehingga memudahkan pengelolaan data pelanggan oleh pihak hotel.

Informasi yang ditampilkan meliputi id_tamu, nama_tamu, alamat, no_hp, dan email. Kolom id_tamu berfungsi sebagai identitas unik setiap tamu, sedangkan nama_tamu menyimpan nama lengkap tamu. Selain itu, alamat, no_hp, dan email digunakan sebagai informasi kontak yang dapat dimanfaatkan untuk keperluan komunikasi, konfirmasi reservasi, maupun penyampaian informasi terkait layanan hotel.

Pada bagian FROM tamu tm, tabel tamu ditetapkan sebagai sumber data utama yang digunakan dalam query. Alias tm diberikan untuk mempermudah penulisan nama tabel pada sintaks SQL. Selanjutnya, perintah ORDER BY tm.nama_tamu digunakan untuk

mengurutkan hasil berdasarkan nama tamu secara alfabetis. Pengurutan ini bertujuan agar data lebih rapi dan memudahkan pencarian tamu tertentu dalam daftar yang ditampilkan.

Hasil dari query ini berupa daftar seluruh tamu yang tersimpan dalam sistem beserta informasi identitas dan kontakannya. Informasi tersebut dapat digunakan oleh pihak hotel untuk mengelola data pelanggan, melakukan pencarian data tamu dengan lebih mudah, serta mendukung proses pelayanan dan administrasi hotel secara lebih efektif.

Script:

Tabel 41. *Script Query 2 - Daftar Tamu (Sederhana)*

```
-- QUERY 2 (Sederhana) - Daftar tamu yang pernah menginap
SELECT
    tm.id_tamu,
    tm.nama_tamu,
    tm.alamat,
    tm.no_hp,
    tm.email
FROM tamu tm
ORDER BY tm.nama_tamu;
```

Output:

Result Grid					
Filter Rows:					
Edit: Export/Import: Wrap Cell Content:					
	id_tamu	nama_tamu	alamat	no_hp	email
▶	TM003	Andi Wijaya	Semarang	081234567803	andi@gmail.com
	TM016	Anisa Rahma	Padang	081234567816	anisa@gmail.com
	TM001	Budi Santoso	Jakarta	081234567801	budi@gmail.com
	TM018	Cindy Oktavia	Banjarmasin	081234567818	cindy@gmail.com
	TM006	Dewi Lestari	Solo	081234567806	dewi@gmail.com
	TM005	Dimas Pratama	Surabaya	081234567805	dimas@gmail.com
	TM007	Fajar Nugroho	Malang	081234567807	fajar@gmail.com
	TM019	Farhan Akbar	Manado	081234567819	farhan@gmail.com
	TM015	Galih Saputra	Pekanbaru	081234567815	galih@gmail.com
	TM011	Hendra Kusuma	Medan	081234567811	hendra@gmail.com
	TM012	Laila Fitriani	Palembang	081234567812	laila@gmail.com
	TM008	Maya Sari	Bogor	081234567808	maya@gmail.com
	TM010	Nanda Putri	Purwokerto	081234567810	nanda@gmail.com

Gambar 5.1.2 *Output Query 2*

5.1.3 Query 3 - Pembayaran via Transfer (Sederhana)

Query ini digunakan untuk menampilkan data pembayaran yang dilakukan menggunakan metode Transfer. Data yang diambil berasal dari tabel pembayaran, yang

menyimpan seluruh informasi transaksi pembayaran dalam sistem reservasi hotel. Query ini berguna untuk memantau transaksi berdasarkan metode pembayaran tertentu sehingga memudahkan proses pencatatan dan verifikasi pembayaran oleh pihak hotel.

Informasi yang ditampilkan meliputi `id_pembayaran`, `tanggal_bayar`, `jumlah_bayar`, `metode_bayar`, `status_bayar`, dan `id_reservasi`. Kolom `id_pembayaran` berfungsi sebagai identitas unik setiap transaksi pembayaran, sedangkan `tanggal_bayar` menunjukkan waktu pembayaran dilakukan. Kolom `jumlah_bayar` berisi nominal pembayaran yang dibayarkan oleh tamu, `metode_bayar` menunjukkan metode yang digunakan, `status_bayar` menunjukkan status transaksi, dan `id_reservasi` menghubungkan pembayaran dengan data reservasi yang terkait.

Pada bagian `WHERE p.metode_bayar = 'Transfer'`, query melakukan proses penyaringan data sehingga hanya transaksi yang menggunakan metode pembayaran transfer yang ditampilkan. Dengan adanya kondisi ini, data pembayaran dengan metode lain seperti tunai atau kartu tidak akan muncul dalam hasil query.

Selanjutnya, perintah `ORDER BY p.tanggal_bayar DESC` digunakan untuk mengurutkan data berdasarkan tanggal pembayaran secara menurun (*descending*). Dengan demikian, transaksi pembayaran yang paling baru akan ditampilkan terlebih dahulu, sehingga memudahkan pengguna dalam melihat aktivitas pembayaran transfer terbaru.

Hasil dari query ini berupa daftar transaksi pembayaran yang menggunakan metode transfer beserta informasi detailnya. Informasi tersebut dapat dimanfaatkan oleh pihak hotel untuk melakukan pengecekan pembayaran, memverifikasi transaksi yang masuk ke rekening hotel, serta mendukung proses pelaporan keuangan berdasarkan metode pembayaran yang digunakan.

Script:

Tabel 42. *Script Query 3 - Pembayaran via Transfer (Sederhana)*

```
-- QUERY 3 (Sederhana) - Pembayaran yang menggunakan metode Transfer
SELECT
    p.id_pembayaran,
    p.tanggal_bayar,
    p.jumlah_bayar,
    p.metode_bayar,
    p.status_bayar,
```

```

p.id_reservasi
FROM pembayaran p
WHERE p.metode_bayar = 'Transfer'
ORDER BY p.tanggal_bayar DESC;

```

Output:

	id_pembayaran	tanggal_bayar	jumlah_bayar	metode_bayar	status_bayar	id_reservasi
▶	PB020	2026-06-20	3000000.00	Transfer	Lunas	R020
	PB018	2026-06-18	1400000.00	Transfer	Lunas	R018
	PB017	2026-06-17	1100000.00	Transfer	Lunas	R017
	PB015	2026-06-15	1900000.00	Transfer	Lunas	R015
	PB013	2026-06-13	1400000.00	Transfer	Lunas	R013
	PB011	2026-06-11	700000.00	Transfer	Lunas	R011
	PB010	2026-06-10	1500000.00	Transfer	Lunas	R010
	PB008	2026-06-08	700000.00	Transfer	Lunas	R008
	PB007	2026-06-07	550000.00	Transfer	Lunas	R007
	PB005	2026-06-05	1900000.00	Transfer	Lunas	R005
	PB003	2026-06-03	1400000.00	Transfer	Lunas	R003
	PB001	2026-06-01	700000.00	Transfer	Lunas	R001
*	NULL	NULL	NULL	NULL	NULL	NULL

Gambar 5.1.3 Output Query 3

5.1.4 Query 4 - Detail Lengkap Reservasi (JOIN >= 3 Tabel)

Query ini digunakan untuk menampilkan informasi lengkap mengenai setiap reservasi yang dilakukan oleh tamu. Data yang ditampilkan berasal dari empat tabel yang saling berhubungan, yaitu reservasi, tamu, kamar, dan tipe_kamar. Penggunaan beberapa operasi JOIN memungkinkan informasi yang tersimpan pada tabel yang berbeda dapat digabungkan menjadi satu tampilan yang lebih lengkap dan informatif.

Informasi yang ditampilkan meliputi id_reservasi, nama_tamu, nomor telepon tamu, nomor kamar, tipe kamar, tanggal check-in, tanggal check-out, lama menginap, jumlah tamu, dan status reservasi. Dengan adanya informasi tersebut, pihak hotel dapat melihat detail reservasi secara menyeluruh tanpa harus membuka tabel satu per satu.

Proses penggabungan data dilakukan melalui beberapa relasi. Tabel reservasi dihubungkan dengan tabel tamu melalui atribut id_tamu untuk memperoleh identitas tamu yang melakukan reservasi. Selanjutnya, tabel reservasi dihubungkan dengan tabel kamar melalui atribut id_kamar untuk mendapatkan informasi kamar yang dipesan. Kemudian, tabel kamar dihubungkan dengan tabel tipe_kamar melalui atribut id_tipe untuk menampilkan jenis kamar yang digunakan pada reservasi tersebut.

Pada query ini juga digunakan fungsi DATEDIFF(r.check_out, r.check_in) yang berfungsi untuk menghitung selisih hari antara tanggal check-out dan check-in. Hasil perhitungan tersebut ditampilkan sebagai kolom lama_menginap, sehingga pengguna dapat langsung mengetahui durasi menginap setiap reservasi tanpa perlu melakukan perhitungan secara manual.

Terakhir, perintah ORDER BY r.tanggal_pesan digunakan untuk mengurutkan data berdasarkan tanggal pemesanan reservasi. Pengurutan ini membantu pengguna dalam melihat urutan reservasi yang masuk sesuai waktu pemesanannya.

Hasil dari query ini berupa daftar lengkap reservasi yang mencakup informasi tamu, kamar yang dipesan, tipe kamar, periode menginap, jumlah tamu, serta status reservasi. Informasi tersebut sangat bermanfaat bagi pihak hotel untuk memantau aktivitas reservasi, mengelola data tamu dan kamar, serta mendukung proses operasional dan pelaporan secara lebih efektif.

Script :

Tabel 43. *Script Query 4 - Detail Lengkap Reservasi (JOIN $\geq$ 3 Tabel)*

```
-- QUERY 4 (JOIN  $\geq$  3 tabel) - Detail lengkap reservasi tamu
-- Gabung: reservasi + tamu + kamar + tipe_kamar
SELECT
    r.id_reservasi,
    tm.nama_tamu,
    tm.no_hp,
    k.nomor_kamar,
    t.nama_tipe,
    r.check_in,
    r.check_out,
    DATEDIFF(r.check_out, r.check_in) AS lama_menginap,
    r.jumlah_tamu,
    r.status_reservasi
FROM reservasi r
JOIN tamu tm ON r.id_tamu = tm.id_tamu
JOIN kamar k ON r.id_kamar = k.id_kamar
JOIN tipe_kamar t ON k.id_tipe = t.id_tipe
```

ORDER BY r.tanggal_pesana;

Output:

Result Grid		Filter Rows:		Export:	Wrap Cell Content:			
	id_reservasi	nama_tamu	no_hp	nomor_kamar	nama_tipe	check_in	check_out	lama_mei
▶	R001	Budi Santoso	081234567801	101	Standard	2026-06-05	2026-06-07	2
	R002	Siti Aminah	081234567802	201	Deluxe	2026-06-06	2026-06-08	2
	R003	Andi Wijaya	081234567803	301	Superior	2026-06-10	2026-06-12	2
	R004	Rina Putri	081234567804	102	Standard	2026-06-11	2026-06-13	2
	R005	Dimas Pratama	081234567805	401	Family	2026-06-15	2026-06-17	2
	R006	Dewi Lestari	081234567806	202	Deluxe	2026-06-20	2026-06-22	2
	R007	Fajar Nugroho	081234567807	203	Deluxe	2026-06-25	2026-06-27	2
	R008	Maya Sari	081234567808	302	Superior	2026-06-28	2026-06-30	2
	R009	Rizki Akbar	081234567809	103	Standard	2026-07-01	2026-07-03	2
	R010	Nanda Putri	081234567810	501	Suite	2026-07-05	2026-07-07	2
	R011	Budi Santoso	081234567801	509	Suite	2026-07-08	2026-07-10	2
	R012	Siti Aminah	081234567802	123	Standard	2026-07-11	2026-07-13	2

Gambar 5.1.4 Output Query 4

5.1.5 Query 5 - Riwayat Pembayaran Lengkap (JOIN >= 3 Tabel)

Query ini digunakan untuk menampilkan riwayat pembayaran secara lengkap dengan menggabungkan data dari empat tabel yang saling berhubungan, yaitu pembayaran, reservasi, tamu, dan kamar. Penggabungan beberapa tabel dilakukan agar informasi pembayaran tidak hanya menampilkan data transaksi, tetapi juga dilengkapi dengan informasi tamu dan kamar yang terkait dengan pembayaran tersebut.

Informasi yang ditampilkan meliputi id_pembayaran, nama_tamu, nomor_kamar, tanggal check-in, tanggal check-out, jumlah_bayar, metode_bayar, status_bayar, dan tanggal_bayar. Dengan adanya informasi tersebut, pihak hotel dapat mengetahui siapa tamu yang melakukan pembayaran, kamar yang digunakan, periode menginap, serta rincian transaksi yang telah dilakukan.

Proses penggabungan data dimulai dengan menghubungkan tabel pembayaran dengan tabel reservasi melalui atribut id_reservasi. Relasi ini digunakan untuk memperoleh informasi reservasi yang berkaitan dengan setiap transaksi pembayaran. Selanjutnya, tabel reservasi dihubungkan dengan tabel tamu menggunakan atribut id_tamu untuk mendapatkan identitas tamu yang melakukan reservasi. Tabel reservasi juga dihubungkan dengan tabel kamar melalui atribut id_kamar sehingga informasi nomor kamar dapat ditampilkan dalam hasil query.

Perintah ORDER BY pb.tanggal_bayar digunakan untuk mengurutkan data berdasarkan tanggal pembayaran. Pengurutan ini memudahkan pengguna dalam melihat urutan transaksi pembayaran sesuai waktu pembayaran yang dilakukan.

Hasil dari query ini berupa daftar riwayat pembayaran yang lengkap, mencakup informasi tamu, kamar yang digunakan, periode menginap, serta detail pembayaran seperti jumlah pembayaran, metode pembayaran, status pembayaran, dan tanggal pembayaran. Informasi tersebut dapat digunakan oleh pihak hotel untuk melakukan pemantauan transaksi, verifikasi pembayaran, penyusunan laporan keuangan, serta evaluasi aktivitas pembayaran pelanggan secara lebih efektif.

Script :

Tabel 44. *Script Query 5 - Riwayat Pembayaran Lengkap (JOIN >= 3 Tabel)*

```
-- QUERY 5 (JOIN ≥ 3 tabel) – Riwayat pembayaran lengkap
dengan info tamu
-- Gabung: pembayaran + reservasi + tamu + kamar
SELECT
    pb.id_pembayaran,
    tm.nama_tamu,
    k.nomor_kamar,
    r.check_in,
    r.check_out,
    pb.jumlah_bayar,
    pb.metode_bayar,
    pb.status_bayar,
    pb.tanggal_bayar
FROM pembayaran pb
JOIN reservasi r ON pb.id_reservasi = r.id_reservasi
JOIN tamu tm ON r.id_tamu = tm.id_tamu
JOIN kamar k ON r.id_kamar = k.id_kamar
ORDER BY pb.tanggal_bayar;
```

Output:

Result Grid		Filter Rows:	Export:	Wrap Cell Content:				
	id_pembayaran	nama_tamu	nomor_kamar	check_in	check_out	jumlah_bayar	metode_bayar	sta
▶	PB001	Budi Santoso	101	2026-06-05	2026-06-07	700000.00	Transfer	Luni
	PB002	Siti Aminah	201	2026-06-06	2026-06-08	1100000.00	QRIS	Luni
	PB003	Andi Wijaya	301	2026-06-10	2026-06-12	1400000.00	Transfer	Luni
	PB004	Rina Putri	102	2026-06-11	2026-06-13	700000.00	Cash	Luni
	PB005	Dimas Pratama	401	2026-06-15	2026-06-17	1900000.00	Transfer	Luni
	PB006	Dewi Lestari	202	2026-06-20	2026-06-22	1100000.00	QRIS	Luni
	PB007	Fajar Nugroho	203	2026-06-25	2026-06-27	550000.00	Transfer	Luni
	PB008	Maya Sari	302	2026-06-28	2026-06-30	700000.00	Transfer	Luni
	PB009	Rizki Akbar	103	2026-07-01	2026-07-03	350000.00	Cash	Luni
	PB010	Nanda Putri	501	2026-07-05	2026-07-07	1500000.00	Transfer	Luni
	PB011	Budi Santoso	509	2026-07-08	2026-07-10	700000.00	Transfer	Luni
	PB012	Siti Aminah	123	2026-07-11	2026-07-13	1100000.00	QRIS	Luni

Gambar 5.1.5 Output Query 5

5.1.6 Query 6 - Detail Check-In dan Pegawai (JOIN ≥ 3 Tabel)

Query ini digunakan untuk menampilkan informasi lengkap mengenai proses check-in tamu beserta data pegawai yang melayani proses tersebut. Untuk memperoleh informasi yang lengkap, query menggabungkan data dari enam tabel yang saling berhubungan, yaitu check_in, reservasi, tamu, kamar, tipe_kamar, dan pegawai. Penggunaan beberapa operasi JOIN memungkinkan data yang tersebar pada berbagai tabel dapat ditampilkan dalam satu hasil query yang terintegrasi.

Informasi yang ditampilkan meliputi id_checkin, waktu_checkin, nama_tamu, nomor_kamar, nama_tipe, petugas_checkin, dan jabatan. Data tersebut memberikan gambaran lengkap mengenai siapa tamu yang melakukan check-in, kamar yang ditempati, jenis kamar yang digunakan, serta pegawai yang bertugas melayani proses check-in tersebut.

Proses penggabungan data dimulai dengan menghubungkan tabel check_in dengan tabel reservasi melalui atribut id_reservasi. Selanjutnya, tabel reservasi dihubungkan dengan tabel tamu untuk memperoleh identitas tamu yang melakukan reservasi. Tabel reservasi juga dihubungkan dengan tabel kamar untuk mendapatkan informasi nomor kamar yang digunakan. Setelah itu, tabel kamar dihubungkan dengan tabel tipe_kamar melalui atribut id_tipe sehingga jenis kamar dapat ditampilkan. Terakhir, tabel check_in dihubungkan dengan tabel pegawai menggunakan atribut id_pegawai untuk memperoleh informasi pegawai yang menangani proses check-in.

Pada bagian pg.nama_pegawai AS petugas_checkin, digunakan alias petugas_checkin untuk memberikan nama kolom yang lebih mudah dipahami pada hasil query. Dengan

demikian, pengguna dapat langsung mengetahui bahwa kolom tersebut menunjukkan nama pegawai yang melayani proses check-in.

Perintah `ORDER BY ci.waktu_checkin` digunakan untuk mengurutkan data berdasarkan waktu check-in. Pengurutan ini memudahkan pengguna dalam melihat urutan aktivitas check-in sesuai waktu pelaksanaannya.

Hasil dari query ini berupa daftar aktivitas check-in yang lengkap, mencakup informasi tamu, kamar yang ditempati, tipe kamar, serta pegawai yang bertugas melayani proses check-in. Informasi tersebut sangat berguna bagi pihak hotel untuk memantau aktivitas kedatangan tamu, mengevaluasi kinerja pegawai, serta mendukung kebutuhan administrasi dan pelaporan operasional hotel secara lebih efektif.

Script :

Tabel 45. *Script Query 6 - Detail Check-In dan Pegawai (JOIN $\geq$ 3 Tabel)*

```
-- QUERY 6 (JOIN  $\geq$  3 tabel) - Detail check-in beserta pegawai
& tamu
-- Gabung: check_in + reservasi + tamu + pegawai
SELECT
    ci.id_checkin,
    ci.waktu_checkin,
    tm.nama_tamu,
    k.nomor_kamar,
    t.nama_tipe,
    pg.nama_pegawai AS petugas_checkin,
    pg.jabatan
FROM check_in ci
JOIN reservasi r ON ci.id_reservasi = r.id_reservasi
JOIN tamu tm ON r.id_tamu = tm.id_tamu
JOIN kamar k ON r.id_kamar = k.id_kamar
JOIN tipe_kamar t ON k.id_tipe = t.id_tipe
JOIN pegawai pg ON ci.id_pegawai = pg.id_pegawai
ORDER BY ci.waktu_checkin;
```

Output:

Result Grid							
		Filter Rows:		Export:		Wrap Cell Content:	
	id_checkin	waktu_checkin	nama_tamu	nomor_kamar	nama_tipe	petugas_checkin	jabatan
▶	CI001	2026-06-05 13:00:00	Budi Santoso	101	Standard	Ahmad Fauzi	Resepsionis
	CI002	2026-06-06 14:00:00	Siti Aminah	201	Deluxe	Yuni Astuti	Resepsionis
	CI003	2026-06-10 13:15:00	Andi Wijaya	301	Superior	Ahmad Fauzi	Resepsionis
	CI004	2026-06-11 14:20:00	Rina Putri	102	Standard	Dedi Saputra	Supervisor
	CI005	2026-06-15 13:30:00	Dimas Pratama	401	Family	Rika Maharani	Front Office
	CI006	2026-06-20 14:00:00	Dewi Lestari	202	Deluxe	Yuni Astuti	Resepsionis
	CI007	2026-06-25 14:30:00	Fajar Nugroho	203	Deluxe	Ahmad Fauzi	Resepsionis
	CI008	2026-06-28 15:00:00	Maya Sari	302	Superior	Yuni Astuti	Resepsionis
	CI009	2026-07-01 15:30:00	Rizki Akbar	103	Standard	Ahmad Fauzi	Resepsionis
	CI010	2026-07-05 16:00:00	Nanda Putri	501	Suite	Dedi Saputra	Supervisor
	CI011	2026-07-08 16:30:00	Budi Santoso	509	Suite	Rika Maharani	Front Office
	CI012	2026-07-11 17:00:00	Siti Aminah	123	Standard	Yuni Astuti	Resepsionis
	CI013	2026-07-14 17:30:00	Andi Wijaya	456	Family	Ahmad Fauzi	Resepsionis

Gambar 5.1.6 Output Query 6

5.1.7 Query 7 - Ulasan Tamu Beserta Detail Kamar (JOIN >= 3 Tabel)

Query ini digunakan untuk menampilkan data ulasan yang diberikan oleh tamu beserta informasi lengkap mengenai kamar yang ditempati. Untuk memperoleh informasi yang lebih detail, query menggabungkan data dari lima tabel yang saling berhubungan, yaitu ulasan, reservasi, tamu, kamar, dan tipe_kamar. Penggabungan beberapa tabel tersebut memungkinkan informasi ulasan dapat ditampilkan bersama identitas tamu dan detail kamar yang terkait.

Informasi yang ditampilkan meliputi id_ulasan, nama_tamu, nomor_kamar, nama_tipe, rating, komentar, dan tanggal_ulasan. Data tersebut memberikan gambaran mengenai penilaian yang diberikan oleh tamu terhadap pengalaman menginapnya, termasuk jenis kamar yang digunakan saat memberikan ulasan.

Proses penggabungan data dimulai dengan menghubungkan tabel ulasan dengan tabel reservasi melalui atribut id_reservasi. Relasi ini digunakan untuk mengetahui reservasi yang menjadi dasar pemberian ulasan. Selanjutnya, tabel reservasi dihubungkan dengan tabel tamu melalui atribut id_tamu untuk memperoleh identitas tamu yang memberikan ulasan. Tabel reservasi juga dihubungkan dengan tabel kamar melalui atribut id_kamar sehingga informasi nomor kamar dapat ditampilkan. Kemudian, tabel kamar dihubungkan dengan tabel tipe_kamar melalui atribut id_tipe untuk memperoleh informasi jenis kamar yang digunakan oleh tamu.

Pada bagian ORDER BY u.rating DESC, u.tanggal_ulasan, hasil query diurutkan berdasarkan nilai rating secara menurun (*descending*), sehingga ulasan dengan penilaian tertinggi akan ditampilkan terlebih dahulu. Jika terdapat beberapa ulasan dengan rating

yang sama, data akan diurutkan kembali berdasarkan tanggal_ulasan, sehingga urutan ulasan menjadi lebih terstruktur dan mudah dianalisis.

Hasil dari query ini berupa daftar ulasan tamu yang dilengkapi dengan informasi kamar dan tipe kamar yang digunakan selama menginap. Informasi tersebut dapat dimanfaatkan oleh pihak hotel untuk mengevaluasi kualitas layanan, mengetahui tingkat kepuasan tamu, mengidentifikasi kelebihan maupun kekurangan fasilitas kamar, serta mendukung pengambilan keputusan dalam upaya peningkatan kualitas pelayanan hotel.

Script :

Tabel 46. *Script Query 7 - Ulasan Tamu Beserta Detail Kamar (JOIN >= 3 Tabel)*

```
-- QUERY 7 (JOIN ≥ 3 tabel) – Ulasan tamu beserta detail
kamar
-- Gabung: ulasan + reservasi + tamu + kamar + tipe_kamar
SELECT
    u.id_ulasan,
    tm.nama_tamu,
    k.nomor_kamar,
    t.nama_tipe,
    u.rating,
    u.komentar,
    u.tanggal_ulasan
FROM ulasan u
JOIN reservasi r ON u.id_reservasi = r.id_reservasi
JOIN tamu tm ON r.id_tamu = tm.id_tamu
JOIN kamar k ON r.id_kamar = k.id_kamar
JOIN tipe_kamar t ON k.id_tipe = t.id_tipe
ORDER BY u.rating DESC, u.tanggal_ulasan;
```

Output:

Result Grid							
Filter Rows:							
Export:							
Wrap Cell Content:							
	id_ulasan	nama_tamu	nomor_kamar	nama_tipe	rating	komentar	tangga
▶	U001	Budi Santoso	101	Standard	5	Sangat puas, kamar bersih dan nyaman	2026-01
	U003	Andi Wijaya	301	Superior	5	Fasilitas lengkap, pasti balik lagi	2026-01
	U005	Dimas Pratama	401	Family	5	Cocok untuk keluarga, anak-anak senang	2026-01
	U008	Maya Sari	302	Superior	5	Kamar nyaman dan luas	2026-01
	U010	Nanda Putri	501	Suite	5	Pelayanan terbaik, kamar mewah	2026-01
	U013	Andi Wijaya	456	Family	5	Sangat puas, akan merekomendasikan	2026-01
	U015	Dimas Pratama	104	Standard	5	Fasilitas lengkap dan modern	2026-01
	U020	Nanda Putri	408	Family	5	Sangat rapi, layanan prima	2026-01
	U002	Siti Aminah	201	Deluxe	4	Kamar nyaman, pelayanan ramah	2026-01
	U004	Rina Putri	102	Standard	4	Pelayanan baik, lokasi strategis	2026-01
	U006	Dewi Lestari	202	Deluxe	4	Bersih dan rapi, sarapan enak	2026-01
	U007	Fajar Nugroho	203	Deluxe	4	Sangat puas dengan fasilitas gym	2026-01

Gambar 5.1.7 Output Query 7

5.1.8 Query 8 - Tamu di Atas Rata-rata Pembayaran (Subquery)

Query ini digunakan untuk menampilkan daftar tamu yang memiliki total pembayaran lebih besar daripada rata-rata total pembayaran seluruh tamu. Query ini memanfaatkan konsep subquery untuk menghitung nilai rata-rata terlebih dahulu, kemudian membandingkannya dengan total pembayaran masing-masing tamu. Dengan cara ini, pihak hotel dapat mengidentifikasi tamu yang memberikan kontribusi pembayaran di atas rata-rata sehingga dapat dijadikan dasar dalam analisis pelanggan atau program loyalitas.

Data yang digunakan berasal dari tabel tamu, reservasi, dan pembayaran. Ketiga tabel tersebut digabungkan menggunakan operasi JOIN sehingga informasi tamu dapat dihubungkan dengan data reservasi dan pembayaran yang dilakukan. Informasi yang ditampilkan meliputi nama_tamu, email, dan total_pembayaran. Nilai total_pembayaran diperoleh melalui fungsi agregasi SUM(pb.jumlah_bayar) yang menjumlahkan seluruh pembayaran yang pernah dilakukan oleh setiap tamu.

Pada bagian GROUP BY tm.id_tamu, tm.nama_tamu, tm.email, data dikelompokkan berdasarkan masing-masing tamu. Pengelompokan ini diperlukan agar fungsi SUM() dapat menghitung total pembayaran untuk setiap tamu secara terpisah.

Selanjutnya, query HAVING digunakan untuk melakukan penyaringan terhadap hasil agregasi. Kondisi pada HAVING membandingkan total pembayaran setiap tamu dengan nilai rata-rata total pembayaran seluruh tamu yang diperoleh dari sebuah subquery. Subquery tersebut terlebih dahulu menghitung total pembayaran masing-masing tamu menggunakan fungsi SUM(), kemudian hasilnya digunakan untuk menghitung rata-rata melalui fungsi AVG(). Dengan demikian, hanya tamu yang memiliki total pembayaran lebih besar daripada rata-rata keseluruhan yang akan ditampilkan pada hasil akhir.

Perintah ORDER BY total_pembayaran DESC digunakan untuk mengurutkan hasil berdasarkan total pembayaran secara menurun (*descending*), sehingga tamu dengan total pembayaran terbesar akan muncul di bagian atas hasil query.

Hasil dari query ini berupa daftar tamu yang memiliki kontribusi pembayaran di atas rata-rata seluruh tamu beserta total pembayaran yang telah dilakukan. Informasi tersebut dapat dimanfaatkan oleh pihak hotel untuk mengidentifikasi pelanggan bernilai tinggi (*high-value customers*), menyusun strategi pemasaran yang lebih tepat sasaran, memberikan program penghargaan atau loyalitas pelanggan, serta melakukan analisis terhadap pola transaksi pelanggan hotel.

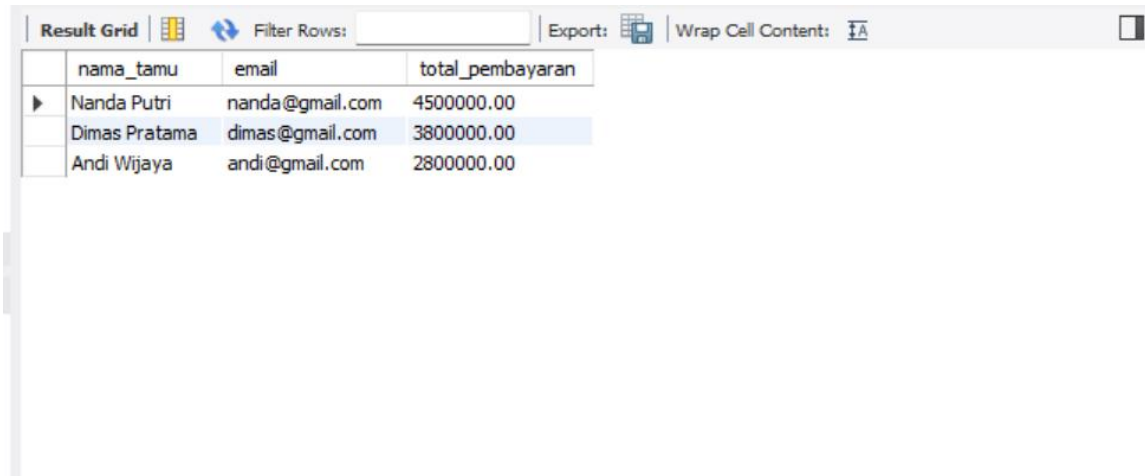
Script :

Tabel 47. *Script Query 8 - Tamu di Atas Rata-rata Pembayaran (Subquery)*

```
-- QUERY 8 (Subquery) - Tamu yang total pembayarannya
-- melebihi rata-rata pembayaran seluruh tamu
SELECT
    tm.nama_tamu,
    tm.email,
    SUM(pb.jumlah_bayar) AS total_pembayaran
FROM tamu    tm
JOIN reservasi r ON tm.id_tamu    = r.id_tamu
JOIN pembayaran pb ON r.id_reservasi = pb.id_reservasi
GROUP BY tm.id_tamu, tm.nama_tamu, tm.email -- Mengelompokkan data
berdasarkan tamu
HAVING SUM(pb.jumlah_bayar) > ( -- Memfilter hasil setelah agregasi, dan
menampilkan data dengan total pembayaran > rata-rata
    SELECT AVG(sub.total)
    FROM (
        SELECT SUM(pb2.jumlah_bayar) AS total
        FROM pembayaran pb2
        JOIN reservasi r2 ON pb2.id_reservasi = r2.id_reservasi
        GROUP BY r2.id_tamu
    ) sub
)
```

```
ORDER BY total_pembayaran DESC;
```

Output:



	nama_tamu	email	total_pembayaran
▶	Nanda Putri	nanda@gmail.com	4500000.00
	Dimas Pratama	dimas@gmail.com	3800000.00
	Andi Wijaya	andi@gmail.com	2800000.00

Gambar 5.1.8 Output Query 8

5.1.9 Query 9 - *Ranking Tipe Kamar* dengan CTE dan *Window Function*

Query ini digunakan untuk menentukan peringkat tipe kamar berdasarkan jumlah reservasi yang diterima oleh masing-masing tipe kamar. Query ini memanfaatkan *Common Table Expression* (CTE) melalui query *WITH* serta *Window Function* menggunakan fungsi *RANK()*. Penggunaan kedua fitur tersebut membuat proses analisis data menjadi lebih terstruktur dan mudah dipahami, terutama ketika melakukan perhitungan agregasi dan pemberian peringkat.

Pada bagian *WITH reservasi_per_tipe AS (...)*, dibuat sebuah CTE bernama *reservasi_per_tipe*. CTE berfungsi sebagai tabel sementara yang hanya tersedia selama query dijalankan. Tabel sementara ini digunakan untuk menyimpan hasil perhitungan jumlah reservasi dan total pendapatan untuk setiap tipe kamar sebelum data tersebut diproses lebih lanjut pada query utama.

Data dalam CTE diperoleh dari penggabungan tabel *tipe_kamar*, *kamar*, *reservasi*, dan *pembayaran*. Tabel *tipe_kamar* dihubungkan dengan tabel *kamar* melalui atribut *id_tipe*, kemudian tabel *kamar* dihubungkan dengan tabel *reservasi* melalui *id_kamar*, dan tabel *reservasi* dihubungkan dengan tabel *pembayaran* melalui *id_reservasi*. Penggabungan ini memungkinkan sistem memperoleh informasi lengkap mengenai jumlah reservasi dan total pendapatan yang dihasilkan oleh setiap tipe kamar.

Fungsi COUNT(r.id_reservasi) digunakan untuk menghitung jumlah reservasi pada setiap tipe kamar, sedangkan fungsi SUM(pb.jumlah_bayar) digunakan untuk menghitung total pendapatan yang diperoleh dari reservasi tersebut. Seluruh data kemudian dikelompokkan berdasarkan id_tipe, nama_tipe, dan harga_dasar menggunakan query GROUP BY.

Setelah data agregasi diperoleh melalui CTE, query utama menampilkan informasi berupa nama_tipe, harga_dasar, jumlah_reservasi, dan total_pendapatan. Selain itu, digunakan fungsi Window Function RANK() OVER (ORDER BY jumlah_reservasi DESC) untuk memberikan peringkat pada setiap tipe kamar berdasarkan jumlah reservasi. Tipe kamar dengan jumlah reservasi terbanyak akan memperoleh peringkat pertama, sedangkan tipe kamar dengan jumlah reservasi yang lebih sedikit akan mendapatkan peringkat berikutnya. Jika terdapat dua atau lebih tipe kamar dengan jumlah reservasi yang sama, fungsi RANK() akan memberikan peringkat yang sama pada tipe kamar tersebut.

Terakhir, perintah ORDER BY ranking_populer digunakan untuk mengurutkan hasil berdasarkan peringkat yang telah dihasilkan, sehingga tipe kamar paling populer akan ditampilkan di bagian atas.

Hasil dari query ini berupa daftar tipe kamar yang telah dilengkapi dengan informasi harga dasar, jumlah reservasi, total pendapatan, dan peringkat popularitasnya. Informasi tersebut dapat dimanfaatkan oleh pihak hotel untuk mengetahui tipe kamar yang paling diminati pelanggan, mengevaluasi kontribusi pendapatan dari setiap tipe kamar, serta mendukung pengambilan keputusan dalam strategi pemasaran, pengelolaan fasilitas, dan pengembangan layanan hotel.

Script :

Tabel 48. *Script Query 9 - Ranking Tipe Kamar dengan CTE dan Window Function*

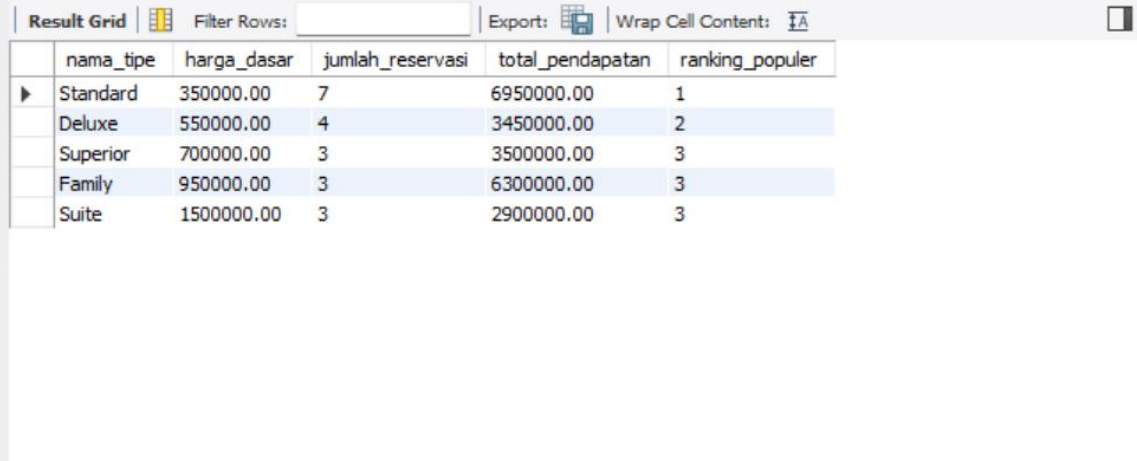
```
-- QUERY 9 (CTE) - Ranking tipe kamar berdasarkan jumlah reservasi
-- menggunakan Common Table Expression (CTE)
WITH reservasi_per_tipe AS ( -- WITH digunakan untuk membuat tabel sementara,
tabel tsb hanya akan ada selama query dijalankan
SELECT
    t.id_tipe,
    t.nama_tipe,
    t.harga_dasar,
```

```

COUNT(r.id_reservasi) AS jumlah_reservasi,
SUM(pb.jumlah_bayar) AS total_pendapatan
FROM tipe_kamar t
JOIN kamar k ON t.id_tipe = k.id_tipe
JOIN reservasi r ON k.id_kamar = r.id_kamar
JOIN pembayaran pb ON r.id_reservasi = pb.id_reservasi
GROUP BY t.id_tipe, t.nama_tipe, t.harga_dasar
)
SELECT
nama_tipe,
harga_dasar,
jumlah_reservasi,
total_pendapatan,
RANK() OVER (ORDER BY jumlah_reservasi DESC) AS ranking_populer
FROM reservasi_per_tipe
ORDER BY ranking_populer;

```

Output:



	nama_tipe	harga_dasar	jumlah_reservasi	total_pendapatan	ranking_populer
▶	Standard	350000.00	7	6950000.00	1
	Deluxe	550000.00	4	3450000.00	2
	Superior	700000.00	3	3500000.00	3
	Family	950000.00	3	6300000.00	3
	Suite	1500000.00	3	2900000.00	3

Gambar 5.1.9 Output Query 9

5.1.10 Query 10 - Pegawai dengan Transaksi Terbanyak (Agregat + GROUP BY + HAVING)

Query ini digunakan untuk menampilkan data pegawai yang menangani lebih dari tiga proses check-in beserta jumlah transaksi yang telah ditangani. Query ini memanfaatkan

fungsi agregat COUNT(), query GROUP BY, dan HAVING untuk melakukan perhitungan serta penyaringan data berdasarkan aktivitas pegawai. Tujuan dari query ini adalah untuk mengidentifikasi pegawai yang memiliki tingkat aktivitas tinggi dalam melayani tamu hotel.

Data yang digunakan berasal dari tabel pegawai, check_in, dan check_out. Tabel pegawai berisi informasi identitas pegawai, sedangkan tabel check_in dan check_out menyimpan data transaksi kedatangan dan keberangkatan tamu. Ketiga tabel tersebut dihubungkan menggunakan LEFT JOIN, sehingga seluruh data pegawai tetap ditampilkan meskipun terdapat pegawai yang belum memiliki catatan check-in atau check-out.

Informasi yang ditampilkan meliputi id_pegawai, nama_pegawai, jabatan, jumlah_checkin, jumlah_checkout, dan total_transaksi. Nilai jumlah_checkin dihitung menggunakan fungsi COUNT(ci.id_checkin) untuk mengetahui berapa banyak proses check-in yang ditangani oleh setiap pegawai. Sementara itu, jumlah_checkout dihitung menggunakan fungsi COUNT(co.id_checkout) untuk mengetahui jumlah proses check-out yang ditangani. Kedua nilai tersebut kemudian dijumlahkan untuk menghasilkan total_transaksi, yang menunjukkan keseluruhan aktivitas pelayanan yang dilakukan oleh pegawai.

Pada bagian GROUP BY pg.id_pegawai, pg.nama_pegawai, pg.jabatan, data dikelompokkan berdasarkan masing-masing pegawai. Pengelompokan ini diperlukan agar fungsi agregat dapat menghitung jumlah transaksi untuk setiap pegawai secara terpisah.

Selanjutnya, query HAVING COUNT(ci.id_checkin) > 3 digunakan untuk menyaring hasil setelah proses agregasi dilakukan. Kondisi ini memastikan bahwa hanya pegawai yang menangani lebih dari tiga proses check-in yang akan ditampilkan dalam hasil query. Berbeda dengan query WHERE yang menyaring data sebelum proses agregasi, HAVING digunakan untuk menyaring hasil berdasarkan nilai yang diperoleh dari fungsi agregat.

Terakhir, perintah ORDER BY total_transaksi DESC digunakan untuk mengurutkan hasil berdasarkan jumlah total transaksi secara menurun (*descending*). Dengan demikian, pegawai yang menangani transaksi terbanyak akan ditampilkan pada urutan teratas.

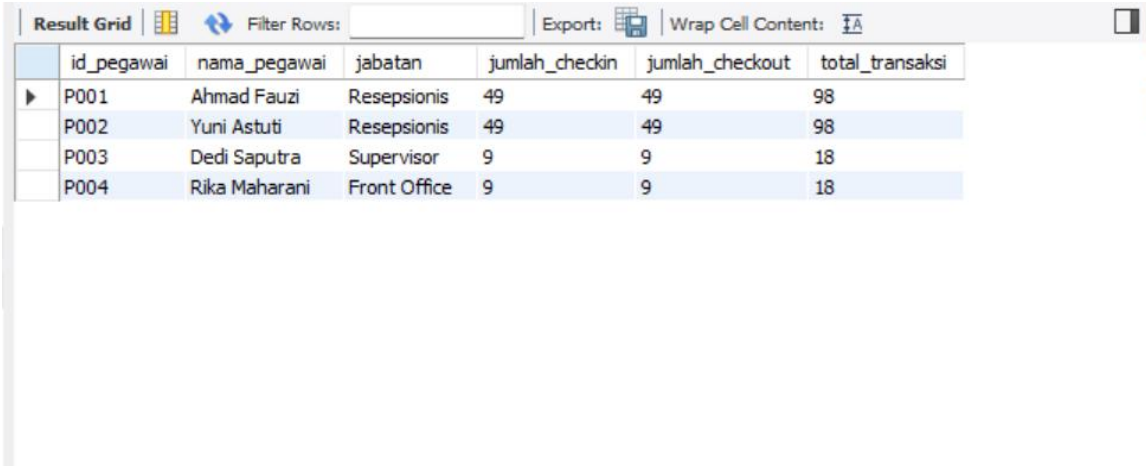
Hasil dari query ini berupa daftar pegawai yang aktif menangani proses check-in beserta jumlah check-in, jumlah check-out, dan total transaksi yang telah ditangani. Informasi tersebut dapat digunakan oleh manajemen hotel untuk mengevaluasi kinerja pegawai, mengetahui tingkat produktivitas staf operasional, serta mendukung pengambilan keputusan terkait pembagian tugas, penilaian kinerja, maupun pemberian penghargaan kepada pegawai yang berprestasi.

Script :

Tabel 49. Script Query 10 - Pegawai dengan Transaksi Terbanyak (Agregat + GROUP BY + HAVING)

```
-- QUERY 10 (Agregat + GROUP BY + HAVING) - Pegawai yang menangani
-- lebih dari 3 proses check-in, beserta total transaksi
SELECT
    pg.id_pegawai,
    pg.nama_pegawai,
    pg.jabatan,
    COUNT(ci.id_checkin) AS jumlah_checkin,
    COUNT(co.id_checkout) AS jumlah_checkout,
    COUNT(ci.id_checkin) + COUNT(co.id_checkout) AS total_transaksi
FROM pegawai pg
LEFT JOIN check_in ci ON pg.id_pegawai = ci.id_pegawai
LEFT JOIN check_out co ON pg.id_pegawai = co.id_pegawai
GROUP BY pg.id_pegawai, pg.nama_pegawai, pg.jabatan
HAVING COUNT(ci.id_checkin) > 3
ORDER BY total_transaksi DESC;
```

Output:



The screenshot shows a database query result grid with the following columns: id_pegawai, nama_pegawai, jabatan, jumlah_checkin, jumlah_checkout, and total_transaksi. The results are sorted by total_transaksi in descending order. The first two rows (P001 and P002) have a total of 98 transactions, while the last two (P003 and P004) have a total of 18 transactions.

	id_pegawai	nama_pegawai	jabatan	jumlah_checkin	jumlah_checkout	total_transaksi
▶	P001	Ahmad Fauzi	Resepsionis	49	49	98
	P002	Yuni Astuti	Resepsionis	49	49	98
	P003	Dedi Saputra	Supervisor	9	9	18
	P004	Rika Maharani	Front Office	9	9	18

Gambar 5.1.10 Output Query 10

5.2 VIEW

5.2.1 VIEW v_reservasi_lengkap

View v_reservasi_lengkap dibuat untuk menyajikan informasi reservasi secara lengkap dalam satu tampilan sehingga pengguna tidak perlu melakukan penggabungan beberapa tabel setiap kali ingin melihat data reservasi. View ini mengintegrasikan data dari tabel reservasi, tamu, kamar, tipe_kamar, dan pembayaran sehingga menghasilkan informasi yang lebih komprehensif dan mudah digunakan untuk kebutuhan operasional hotel.

Informasi yang ditampilkan mencakup data reservasi seperti id_reservasi, tanggal_pesan, check_in, check_out, lama_menginap, jumlah_tamu, dan status_reservasi. Selain itu, view ini juga menampilkan informasi tamu berupa id_tamu, nama_tamu, nomor telepon, dan email, sehingga resepsionis dapat dengan mudah mengetahui identitas pelanggan yang melakukan reservasi.

Pada bagian informasi kamar, view menampilkan nomor_kamar, lantai, nama_tipe, kapasitas, dan harga_dasar. Informasi ini memberikan gambaran mengenai kamar yang dipesan beserta karakteristik dan tarif dasarnya. Untuk melengkapi data transaksi, view juga menampilkan informasi pembayaran seperti id_pembayaran, jumlah_bayar, metode_bayar, dan status_bayar.

View ini memanfaatkan fungsi DATEDIFF() untuk menghitung lama menginap tamu berdasarkan selisih antara tanggal check-out dan check-in. Hasil perhitungan tersebut ditampilkan pada kolom lama_menginap. Selain itu, view juga melakukan perhitungan tagihan_normal, yaitu estimasi biaya menginap yang diperoleh dari hasil perkalian lama menginap dengan harga dasar tipe kamar. Perhitungan ini memberikan gambaran awal mengenai biaya yang seharusnya dibayarkan sebelum memperhitungkan fasilitas tambahan atau biaya lainnya.

Penggabungan data dilakukan menggunakan beberapa operasi JOIN. Tabel reservasi dihubungkan dengan tabel tamu, kamar, dan tipe_kamar untuk memperoleh informasi identitas tamu serta detail kamar yang dipesan. Sementara itu, tabel pembayaran dihubungkan menggunakan LEFT JOIN, sehingga data reservasi tetap akan ditampilkan meskipun belum memiliki catatan pembayaran. Hal ini penting untuk memantau reservasi yang masih menunggu proses pembayaran.

Keuntungan utama penggunaan view ini adalah mempermudah akses data karena seluruh informasi penting mengenai reservasi telah tersedia dalam satu objek basis data. Resepsionis maupun manajemen hotel dapat menggunakan view ini untuk memantau

reservasi aktif, memeriksa status pembayaran, melihat detail tamu dan kamar, serta memperoleh informasi tagihan dengan lebih cepat dan efisien. Sebagai contoh, view ini dapat digunakan untuk menampilkan seluruh reservasi yang masih berstatus "Dipesan" sehingga memudahkan proses pemantauan reservasi yang belum melakukan check-in.

Script :

Tabel 50. *Script* VIEW v_reservasi_lengkap

```
-- VIEW 1: v_reservasi_lengkap
-- Menampilkan ringkasan reservasi secara lengkap, berguna bagi
-- resepsionis untuk melihat seluruh data tamu, kamar, dan status.
CREATE VIEW v_reservasi_lengkap AS
SELECT
    r.id_reservasi,
    r.tanggal_pesan,
    r.check_in,
    r.check_out,
    DATEDIFF(r.check_out, r.check_in) AS lama_menginap,
    r.jumlah_tamu,
    r.status_reservasi,
    -- Info tamu
    tm.id_tamu,
    tm.nama_tamu,
    tm.no_hp AS hp_tamu,
    tm.email AS email_tamu,
    -- Info kamar
    k.nomor_kamar,
    k.lantai,
    t.nama_tipe,
    t.kapasitas,
    t.harga_dasar,
    -- Info pembayaran
    pb.id_pembayaran,
    pb.jumlah_bayar,
```

```

pb.metode_bayar,
pb.status_bayar,
-- Kalkulasi total tagihan berdasar lama menginap × harga dasar
(DATEDIFF(r.check_out, r.check_in) * t.harga_dasar) AS tagihan_normal
FROM reservasi r
JOIN tamu tm ON r.id_tamu = tm.id_tamu
JOIN kamar k ON r.id_kamar = k.id_kamar
JOIN tipe_kamar t ON k.id_tipe = t.id_tipe
LEFT JOIN pembayaran pb ON r.id_reservasi = pb.id_reservasi;
-- Contoh penggunaan:
-- SELECT * FROM v_reservasi_lengkap WHERE status_reservasi = 'Dipesan';

```

Output:

id_reservasi	tanggal_pesan	check_in	check_out	lama_menginap	jumlah_tamu	status_reservasi	id_tam
R009	2026-06-09	2026-07-01	2026-07-03	2	2	Dipesan	TM009
R017	2026-06-17	2026-07-26	2026-07-28	2	2	Dipesan	TM007
R007	2026-06-07	2026-06-25	2026-06-27	2	2	Dipesan	TM007
R008	2026-06-08	2026-06-28	2026-06-30	2	3	Dipesan	TM008
R018	2026-06-18	2026-07-29	2026-07-31	2	3	Dipesan	TM008
R010	2026-06-10	2026-07-05	2026-07-07	2	4	Dipesan	TM010
R019	2026-06-19	2026-08-01	2026-08-03	2	2	Dipesan	TM009

Gambar 5.2.1 Output VIEW v_reservasi_lengkap

5.2.2 VIEW v_statistik_kamar

View v_statistik_kamar digunakan untuk menampilkan statistik kinerja setiap tipe kamar berdasarkan data reservasi, pembayaran, dan ulasan tamu. View ini menggabungkan tabel tipe_kamar, kamar, reservasi, pembayaran, dan ulasan sehingga menghasilkan informasi yang dapat digunakan manajemen hotel untuk mengevaluasi performa masing-masing tipe kamar.

Informasi yang ditampilkan meliputi id_tipe, nama_tipe, kapasitas, harga_dasar, total_reservasi, total_pendapatan, rata_pendapatan_per_res, rata_rating, dan jumlah_ulasan. Nilai total_reservasi dihitung menggunakan fungsi COUNT(DISTINCT) untuk menghitung jumlah reservasi unik, sedangkan total_pendapatan diperoleh dari

penjumlahan seluruh pembayaran menggunakan fungsi SUM(). Selain itu, fungsi AVG() digunakan untuk menghitung rata-rata pendapatan per reservasi dan rata-rata rating ulasan, yang kemudian dibulatkan menggunakan fungsi ROUND().

View ini menggunakan LEFT JOIN agar seluruh tipe kamar tetap ditampilkan meskipun belum memiliki data reservasi, pembayaran, atau ulasan. Selanjutnya, data dikelompokkan berdasarkan tipe kamar menggunakan query GROUP BY sehingga perhitungan statistik dapat dilakukan untuk masing-masing tipe kamar secara terpisah.

Hasil dari view ini berupa ringkasan performa setiap tipe kamar yang mencakup jumlah reservasi, total pendapatan, dan tingkat kepuasan tamu. Informasi tersebut dapat digunakan oleh manajemen hotel untuk mengetahui tipe kamar yang paling diminati, paling menguntungkan, serta sebagai bahan evaluasi dalam meningkatkan kualitas layanan dan strategi bisnis hotel.

Script :

Tabel 51. *Script VIEW v_statistik_kamar*

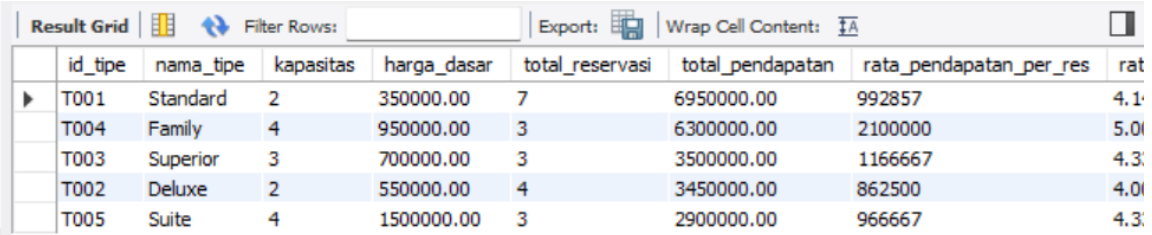
```
-- VIEW 2: v_statistik_kamar
-- Menampilkan statistik kinerja setiap tipe kamar: jumlah
-- reservasi, total pendapatan, dan rata-rata rating ulasan.
-- Berguna bagi manajemen untuk evaluasi performa kamar.
CREATE VIEW v_statistik_kamar AS
SELECT
    t.id_tipe,
    t.nama_tipe,
    t.kapasitas,
    t.harga_dasar,
    COUNT(DISTINCT r.id_reservasi) AS total_reservasi,
    SUM(pb.jumlah_bayar) AS total_pendapatan,
    ROUND(AVG(pb.jumlah_bayar), 0) AS rata_pendapatan_per_res, -- ROUND untuk
membulatkan angka dari desimal ke int, misal 60000,75 menjadi 60001
    ROUND(AVG(u.rating), 2) AS rata_rating,
    COUNT(DISTINCT u.id_ulasan) AS jumlah_ulasan -- DISTINT untuk
menghilangkan data yang duplikat sehingga hanya menampilkan nilai unik
FROM tipe_kamar t
```

```

LEFT JOIN kamar k ON t.id_tipe = k.id_tipe
LEFT JOIN reservasi r ON k.id_kamar = r.id_kamar
LEFT JOIN pembayaran pb ON r.id_reservasi = pb.id_reservasi
LEFT JOIN ulasan u ON r.id_reservasi = u.id_reservasi
GROUP BY t.id_tipe, t.nama_tipe, t.kapasitas, t.harga_dasar;
-- Contoh penggunaan:
-- SELECT * FROM v_statistik_kamar ORDER BY total_pendapatan DESC;

```

Output:



	id_tipe	nama_tipe	kapasitas	harga_dasar	total_reservasi	total_pendapatan	rata_pendapatan_per_res	rata_pendapatan_per_kamar
▶	T001	Standard	2	350000.00	7	6950000.00	992857	4.14
	T004	Family	4	950000.00	3	6300000.00	2100000	5.00
	T003	Superior	3	700000.00	3	3500000.00	1166667	4.33
	T002	Deluxe	2	550000.00	4	3450000.00	862500	4.00
	T005	Suite	4	1500000.00	3	2900000.00	966667	4.33

Gambar 5.2.2 Output VIEW v_statistik_kamar

5.3 STORED PROCEDURE

5.3.1 sp_buat_reservasi

Stored procedure sp_buat_reservasi digunakan untuk membuat data reservasi baru secara otomatis sekaligus memperbarui status kamar yang dipesan. Procedure ini menerima enam parameter input, yaitu id_reservasi, id_tamu, id_kamar, check_in, check_out, dan jumlah_tamu yang diperlukan dalam proses pemesanan kamar.

Saat procedure dijalankan, sistem akan menambahkan data baru ke tabel reservasi menggunakan perintah INSERT INTO. Tanggal pemesanan diisi secara otomatis menggunakan fungsi CURDATE(), sedangkan status reservasi akan diset menjadi 'Dipesan'. Setelah data reservasi berhasil ditambahkan, procedure akan menjalankan perintah UPDATE pada tabel kamar untuk mengubah status_kamar menjadi 'Dipesan' sesuai kamar yang dipilih oleh tamu.

Pada bagian akhir, procedure menggunakan fungsi CONCAT() untuk menampilkan pesan konfirmasi bahwa reservasi berhasil dibuat. Sebagai contoh, jika procedure dipanggil dengan ID reservasi R021, maka sistem akan menampilkan pesan *"Reservasi R021 berhasil dibuat"*.

Dengan adanya stored procedure ini, proses pembuatan reservasi menjadi lebih cepat, terstruktur, dan konsisten karena penyimpanan data reservasi serta perubahan status kamar dilakukan secara otomatis dalam satu perintah. Hal ini juga membantu mengurangi kesalahan yang dapat terjadi apabila proses dilakukan secara manual.

Script :

Tabel 52. *Script* Stored Procedure sp_buat_reservasi

```
-- STORED PROCEDURE

USE hotel_reservasi;

DELIMITER //

-- PROCEDURE: sp_buat_reservasi
-- Fungsi : Membuat reservasi baru dan mengubah status kamar menjadi 'Dipesan'.
-- Parameter:
-- p_id_reservasi - ID reservasi baru (mis: 'R021')
-- p_id_tamu      - ID tamu yang memesan
-- p_id_kamar    - ID kamar yang dipesan
-- p_check_in    - Tanggal check-in
-- p_check_out   - Tanggal check-out
-- p_jumlah_tamu - Jumlah tamu

CREATE PROCEDURE sp_buat_reservasi(
    IN p_id_reservasi VARCHAR(5),
    IN p_id_tamu VARCHAR(7),
    IN p_id_kamar VARCHAR(5),
    IN p_check_in DATE,
    IN p_check_out DATE,
    IN p_jumlah_tamu TINYINT
)
BEGIN
    -- Insert reservasi baru
    INSERT INTO reservasi
        (id_reservasi, tanggal_pesan, check_in, check_out,
        jumlah_tamu, status_reservasi, id_tamu, id_kamar)
    VALUES
        (p_id_reservasi, CURDATE(), p_check_in, p_check_out,
        p_jumlah_tamu, 'Dipesan', p_id_tamu, p_id_kamar);
```

```

-- Update status kamar menjadi Dipesan
UPDATE kamar -- Mengupdate tabel
SET status_kamar = 'Dipesan' -- Mengubah status kamar setelah ada yang reservasi
WHERE id_kamar = p_id_kamar; -- Hanya kamar yang dipilih

SELECT CONCAT('Reservasi ', p_id_reservasi, ' berhasil dibuat.') AS hasil; --
CONCAT untuk menggabungkan teks (Reservasi R021 berhasil dibuat)
END //

DELIMITER ;
CALL sp_buat_reservasi('R021','TM003','K045','2026-09-01','2026-09-04',2);

```

Output:

hasil
Reservasi R021 berhasil dibuat.

Gambar 5.3.1 Output PROCEDURE sp_buat_reservasi

5.4 TRIGGER

5.4.1 trg_after_checkout_update_kamar

Trigger trg_after_checkout_update_kamar digunakan untuk mengotomatisasi proses yang terjadi setelah tamu melakukan check-out. Trigger ini akan aktif secara otomatis setiap kali terdapat penambahan data (INSERT) pada tabel check_out. Dengan adanya trigger ini, sistem dapat memperbarui status kamar dan reservasi tanpa perlu dilakukan secara manual oleh pengguna.

Ketika trigger dijalankan, sistem terlebih dahulu mengambil informasi `id_kamar` dan `nama_tamu` yang terkait dengan reservasi yang baru melakukan check-out. Data tersebut diperoleh dari tabel reservasi dan tamu menggunakan perintah `SELECT ... INTO`, kemudian disimpan ke dalam variabel lokal `v_id_kamar` dan `v_nama_tamu`.

Selanjutnya, trigger akan mengubah status_kamar pada tabel kamar menjadi 'Tersedia'. Perubahan ini menandakan bahwa kamar yang sebelumnya ditempati tamu sudah dapat digunakan kembali untuk reservasi berikutnya. Selain itu, status pada tabel reservasi juga diperbarui menjadi 'Selesai' untuk menunjukkan bahwa proses menginap telah berakhir.

Setelah proses pembaruan data selesai, trigger akan mencatat aktivitas tersebut ke dalam tabel `audit_log`. Pencatatan dilakukan menggunakan perintah `INSERT` yang menyimpan informasi mengenai ID check-out, ID reservasi, nama tamu, kamar yang digunakan, perubahan status kamar, serta waktu check-out. Fungsi `CONCAT()` digunakan untuk menggabungkan seluruh informasi tersebut menjadi satu keterangan yang mudah dibaca.

Dengan adanya trigger ini, proses check-out menjadi lebih efisien karena pembaruan status kamar, penyelesaian reservasi, dan pencatatan aktivitas dilakukan secara otomatis dalam satu proses. Selain membantu menjaga konsistensi data, trigger ini juga memudahkan proses audit dan pelacakan aktivitas yang terjadi dalam sistem reservasi hotel.

Script :

Tabel 53. *Script Trigger trg_after_checkout_update_kamar*

```
-- TRIGGER 1: trg_after_checkout_update_kamar
-- Fungsi : Setelah baris baru masuk ke tabel check_out,
--           status kamar terkait otomatis diubah menjadi
--           'Tersedia', dan status reservasi diubah 'Selesai'.
--           Kejadian ini juga dicatat ke tabel audit_log.
DELIMITER //

CREATE TRIGGER trg_after_checkout_update_kamar
AFTER INSERT ON check_out
FOR EACH ROW
BEGIN
```

```

DECLARE v_id_kamar VARCHAR(5);
DECLARE v_nama_tamu VARCHAR(100);

-- Ambil id_kamar dari reservasi terkait
SELECT r.id_kamar, tm.nama_tamu
INTO v_id_kamar, v_nama_tamu
FROM reservasi r
JOIN tamu tm ON r.id_tamu = tm.id_tamu
WHERE r.id_reservasi = NEW.id_reservasi;

-- Update status kamar menjadi Tersedia
UPDATE kamar
SET status_kamar = 'Tersedia'
WHERE id_kamar = v_id_kamar;

-- Update status reservasi menjadi Selesai
UPDATE reservasi
SET status_reservasi = 'Selesai'
WHERE id_reservasi = NEW.id_reservasi;

-- Catat ke audit_log
INSERT INTO audit_log (nama_tabel, operasi, id_record, keterangan)
VALUES (
    'check_out',
    'INSERT',
    NEW.id_checkout,
    CONCAT(
        'Check-out ', NEW.id_checkout,
        '| Reservasi: ', NEW.id_reservasi,
        '| Tamu: ', v_nama_tamu,
        '| Kamar ', v_id_kamar, ' -> Tersedia',
        '| Waktu: ', NEW.waktu_checkout
    )
)

```

```
);  
END//  
DELIMITER ;
```

5.5 FUNCTION

5.5.1 fn_hitung_total_tagihan

Function `fn_hitung_total_tagihan` digunakan untuk menghitung total tagihan suatu reservasi berdasarkan lama menginap dan harga dasar kamar yang dipesan. Function ini menerima satu parameter input berupa `id_reservasi`, kemudian mengembalikan nilai total tagihan dalam bentuk `DECIMAL(12,2)`. Dengan adanya function ini, perhitungan biaya menginap dapat dilakukan secara otomatis dan konsisten tanpa perlu menuliskan rumus perhitungan berulang kali pada query lain.

Saat function dijalankan, sistem akan mengambil informasi `harga_dasar` kamar serta menghitung lama menginap menggunakan fungsi `DATEDIFF()`, yaitu selisih antara tanggal `check_out` dan `check_in`. Data tersebut diperoleh dari tabel reservasi, kamar, dan tipe_kamar yang dihubungkan melalui operasi `JOIN`. Nilai yang diperoleh kemudian disimpan ke dalam variabel lokal `v_harga_dasar` dan `v_lama`.

Selanjutnya, function melakukan perhitungan total tagihan dengan mengalikan lama menginap dengan harga dasar kamar menggunakan perintah `SET v_total = v_lama * v_harga_dasar`. Hasil perhitungan tersebut disimpan pada variabel `v_total` dan dikembalikan melalui perintah `RETURN`.

Sebagai contoh, apabila function dipanggil menggunakan perintah `SELECT fn_hitung_total_tagihan('R001')`, sistem akan menghitung total biaya menginap untuk reservasi dengan ID R001 dan menampilkan hasilnya sebagai `total_tagihan`.

Dengan adanya function ini, proses perhitungan biaya reservasi menjadi lebih sederhana, cepat, dan konsisten. Function juga dapat digunakan kembali pada berbagai query, view, maupun laporan yang membutuhkan informasi total tagihan berdasarkan data reservasi yang tersimpan dalam sistem.

Script :

Tabel 54. *Script Function fn_hitung_total_tagihan*

```
-- FUNCTION : fn_hitung_total_tagihan  
-- Fungsi : Menghitung total tagihan sebuah reservasi berdasarkan lama menginap
```

(selisih check_out - check_in) dikalikan harga dasar tipe kamar yang dipesan, lalu ditambah biaya fasilitas tambahan yang dipilih tamu.

-- Parameter:

-- p_id_reservasi - ID reservasi yang ingin dihitung

-- Return : DECIMAL(12,2) - Total tagihan dalam Rupiah

DELIMITER //

CREATE FUNCTION fn_hitung_total_tagihan(p_id_reservasi VARCHAR(5))

RETURNS DECIMAL(12,2)

DETERMINISTIC

READS SQL DATA

BEGIN

DECLARE v_harga_dasar DECIMAL(12,2);

DECLARE v_lama INT;

DECLARE v_total DECIMAL(12,2);

SELECT

t.harga_dasar,

DATEDIFF(r.check_out, r.check_in)

INTO v_harga_dasar, v_lama

FROM reservasi r

JOIN kamar k ON r.id_kamar = k.id_kamar

JOIN tipe_kamar t ON k.id_tipe = t.id_tipe

WHERE r.id_reservasi = p_id_reservasi;

SET v_total = v_lama * v_harga_dasar;

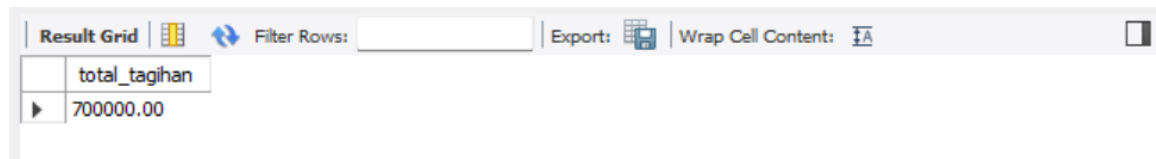
RETURN v_total;

END//

DELIMITER ;

SELECT fn_hitung_total_tagihan('R001') AS total_tagihan;

Output:



Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	total_tagihan			
▶	700000.00			

Gambar 5.5.1 *Output* FUNCTION fn_hitung_total_tagihan

BAB VI

PENUTUP

6.1 Kesimpulan

Berdasarkan hasil perancangan dan implementasi yang telah dilakukan, dapat ditarik beberapa kesimpulan sebagai berikut.

Pertama, perancangan *Entity Relationship Diagram* (ERD) dan skema relasional sistem reservasi hotel telah berhasil dilakukan secara komprehensif. ERD yang dirancang mencakup sepuluh entitas utama, yaitu Tamu, Kamar, Tipe_Kamar, Reservasi, Pembayaran, Check_In, Check_Out, Pegawai, Fasilitas, dan Ulasan, beserta relasi dan kardinalitas yang merepresentasikan seluruh proses bisnis hotel secara tepat. Pemetaan ERD ke dalam skema relasional menghasilkan sepuluh tabel yang saling terhubung melalui primary key dan foreign key sehingga integritas referensial antar tabel dapat terjaga dengan baik.

Kedua, implementasi basis data relasional sistem reservasi hotel menggunakan DDL dan DML telah berhasil dilaksanakan sesuai dengan kaidah normalisasi hingga Bentuk Normal Ketiga (3NF). Proses normalisasi dimulai dari data tidak normal (0NF), kemudian secara bertahap mencapai 1NF, 2NF, dan akhirnya 3NF sehingga seluruh atribut *non-key* bergantung penuh pada *primary key* masing-masing tabel. Penerapan berbagai constraint seperti PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK, dan DEFAULT pada setiap tabel berhasil menjaga validitas, konsistensi, dan integritas data dalam sistem reservasi hotel.

Ketiga, pembuatan query, view, stored procedure, trigger, dan function telah berhasil dilakukan untuk mendukung kebutuhan operasional sistem reservasi hotel. Sebanyak sepuluh query SELECT dirancang dengan berbagai tingkat kompleksitas mulai dari query sederhana, JOIN lebih dari tiga tabel, subquery, hingga penggunaan CTE dan window function. Dua view (*v_reservasi_lengkap* dan *v_statistik_kamar*) dibuat untuk mempermudah akses data yang terintegrasi. Stored procedure *sp_buat_reservasi* mengotomatisasi proses pembuatan reservasi sekaligus pembaruan status kamar, trigger *trg_after_checkout_update_kamar* mengotomatisasi pembaruan status kamar dan reservasi setelah tamu check-out sekaligus mencatat aktivitas pada tabel audit log, serta function *fn_hitung_total_tagihan* menyederhanakan perhitungan biaya menginap berdasarkan lama menginap dan harga dasar kamar.

6.2 Saran

Berdasarkan hasil perancangan dan implementasi sistem basis data reservasi hotel yang telah dilakukan, terdapat beberapa saran yang dapat dijadikan bahan pertimbangan untuk pengembangan lebih lanjut.

Pertama, sistem basis data yang telah dirancang dapat dikembangkan lebih lanjut dengan menambahkan fitur autentikasi dan otorisasi pengguna secara langsung pada level basis data, misalnya dengan memanfaatkan fitur manajemen hak akses (GRANT dan REVOKE) pada MySQL agar setiap pengguna hanya dapat mengakses data sesuai dengan peran dan wewenangnya masing-masing.

Kedua, basis data reservasi hotel ini dapat diintegrasikan dengan antarmuka aplikasi berbasis web atau mobile sehingga proses reservasi, check-in, check-out, dan pembayaran dapat dilakukan secara daring oleh tamu maupun petugas hotel. Pengembangan ini akan meningkatkan kemudahan akses dan memperluas jangkauan layanan hotel kepada pelanggan yang lebih luas.

Ketiga, perlu ditambahkan mekanisme pencadangan data (backup) dan pemulihan data (recovery) secara berkala agar data operasional hotel tetap aman dari risiko kehilangan atau kerusakan data. Penerapan strategi replikasi basis data juga dapat dipertimbangkan untuk meningkatkan ketersediaan dan keandalan sistem, terutama apabila sistem digunakan dalam lingkungan produksi yang menuntut ketersediaan tinggi.

Keempat, desain basis data yang telah dibuat dapat diperluas dengan menambahkan entitas-entitas baru sesuai dengan kebutuhan operasional hotel yang sesungguhnya, seperti entitas promosi atau diskon, paket menginap, layanan kamar (room service), serta integrasi dengan sistem keuangan dan akuntansi hotel agar pengelolaan data dapat menjadi lebih lengkap dan terintegrasi.

LAMPIRAN

Link AI : <https://chatgpt.com/share/6a38e002-d45c-83ec-bea0-a1c713f39b77>

Link Github : https://github.com/ameliaawulandari710-a11y/PROJECT-BASISDATA_KELOMPOK-D